arXiv:2601.10611v4 [cs.CV] 2 Apr 2026

Molmo2

Open Weights and Data for Vision-Language Models with Video Understanding and Grounding

Christopher Clark[♥1*] Jieyu Zhang[♥1,2*] Zixian Ma[♥1,2*] Jae Sung Park[♥1,2*] Mohammadreza Salehi[♥1,2] Rohun Tripathi[♥1] Sangho Lee[♥1]

Zhongzheng Ren[1,2] Chris Dongjoo Kim[1] Yinuo Yang[2] Vincent Shao[2] Yue Yang[1] Weikai Huang[2] Ziqi Gao[1] Taira Anderson[1] Jianrui Zhang[1] Jitesh Jain[1] George Stoica[1] Winson Han[1]

Ali Farhadi[1,2] Ranjay Krishna[♥1,2]

[1]Allen Institute for AI, [2]University of Washington

[*]denotes equal contribution. ♥ marks core contributors, who were all integral to the project
See full author contributions here.

🤗 **Models:** Molmo2-4B Molmo2-8B Molmo2-O-7B

Data: Molmo2 Data

Code: https://github.com/allenai/molmo2

Demo: playground.allenai.org

Contact: molmo@allenai.org

Abstract

Today's strongest video-language models (VLMs) remain proprietary. The strongest open-weight models either rely on synthetic data from proprietary VLMs, effectively distilling from them, or do not disclose their training data or recipe. As a result, the open-source community lacks the foundations needed to improve on the state-of-the-art video (and image) language models. Crucially, many downstream applications require more than just high-level video understanding; they require grounding—either by pointing or by tracking in pixels. Even proprietary models lack this capability. We present Molmo2, a new family of VLMs that are state-of-the-art among open-source models and demonstrate exceptional new capabilities in point-driven grounding in single image, multi-image, and video tasks. Our key contribution is a collection of 7 new video datasets and 2 multi-image datasets, including a dataset of highly detailed video captions for pre-training, a free-form video Q&A dataset for fine-tuning, a new object tracking dataset with complex queries, and an innovative new video pointing dataset, all collected without the use of closed VLMs. We also present a training recipe for this data utilizing an efficient packing and message-tree encoding scheme, and show bi-directional attention on vision tokens and a novel token-weight strategy improves performance. Our best-in-class 8B model outperforms others in the class of open weight and data models on short videos, counting, and captioning, and is competitive on long-videos. On video-grounding Molmo2 significantly outperforms existing open-weight models like Qwen3-VL (35.5 vs 29.6 accuracy on video counting) and surpasses proprietary models like Gemini 3 Pro on some tasks (38.4 vs 20.0 F1 on video pointing and 56.2 vs 41.1 $\mathcal{J}\&\mathcal{F}$ on video tracking).

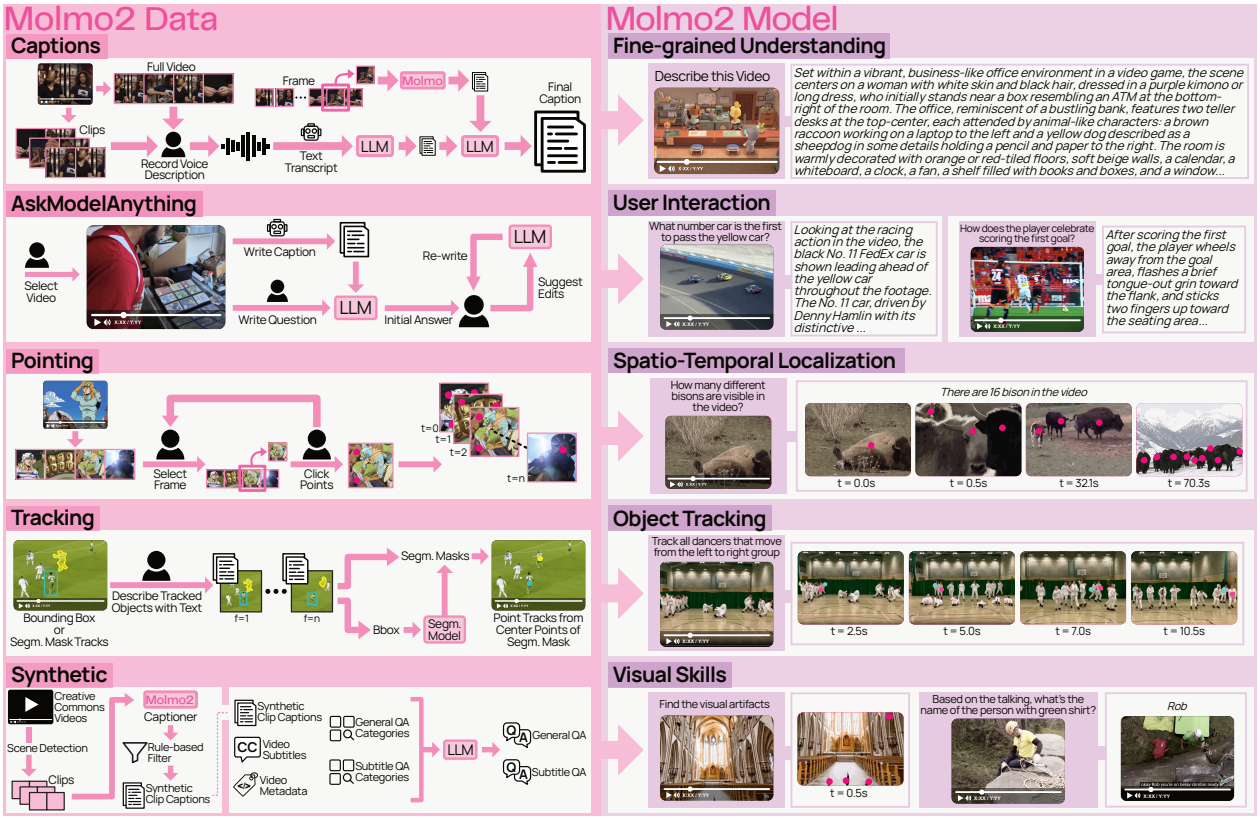


Figure 1 Molmo2 is trained on one of the largest fully open video-centric multimodal corpus to date, including nine new datasets for dense video captioning, long-form and long-video QA, and open-vocabulary pointing and tracking over images, multi-images, and videos. Molmo2 accepts single images, image sets, and videos as input and can produce both free-form language and grounded outputs such as spatio-temporal points, object tracks, and grounded chain-of-thoughts that localize objects and events over time. Across diverse video-language and grounding benchmarks, Molmo2 matches or surpasses prior open models, approaches proprietary systems, and remains fully open.

1 Introduction

Visual data (especially videos) is now ubiquitous, streaming continuously from phones, home cameras, social media, autonomous systems, and industrial sensors [34]. Understanding this video is fundamental for applications such as video search, household and industrial robotics, assistive technologies, sports analytics, security and traffic monitoring, and autonomous driving [83, 84, 87]. Yet the strongest video–language models remain proprietary [135, 113, 17, 145], with closed weights, data, and training recipes.

A key missing capability in current video–language models is *grounding*. Grounding would allow models to answer "How many times does the robot grasp the red block?", by emitting *points* for each grasp event in space and time. It would identify "When did the cup fall off the table?" by returning a *track* of the cup so users can precisely locate the event. Although image grounding is now standard [20], video grounding is only supported in some proprietary systems, and even there in a limited form.

We present the **Molmo2** (**M**ultimodal **O**pen **L**anguage **Mo**del), a family of *fully open* state-of-the-art vision-language models. Molmo2 supports single image, multi-image, as well as video, bridging the aforementioned gap by bringing grounding capabilities to video understanding. To promote open research, *we release our training data, model weights, and training code.* To ensure our work is transparent and fully open, all our data is constructed without distilling from proprietary models.

A core contribution of this work is *a suite of 9 novel datasets* targeting crucial skills underrepresented in existing open data for video and multi-image inputs. This includes: (1) two open-vocabulary video pointing and tracking datasets (520k instances), enabling models to pinpoint when and where events or objects occur

in videos; (2) a dense video captioning corpus (104k videos) with captions far longer and more detailed than in any prior work (e.g., GPT-generated video captions in LLaVA-Video [184] and ShareGPT4Video [19]); (3) two long-form QA datasets (212k instances), including user questions on multi-image/video inputs with rich human-crafted answers (without distilling proprietary models); and (4) two long-video question answering datasets (around 1.3M instances) that tackle videos longer than those in current benchmarks (addressing a known weakness of open models on long-duration content [118]); (5) two multi-image datasets to improve multi-image pointing and document understanding.

Our data collection uses multiple innovative pipelines (Figure 1). For *dense video captioning*, we devised a multi-stage process: human annotators first narrate each video clip in detail via spoken descriptions (allowing much more detail than text typing), which are transcribed and then enriched with frame-level visual details sourced from Molmo [29] to ensure no detail is overlooked.

Because existing large-scale datasets for video or multi-image are largely distilled from proprietary models [99, 93, 59, 76, 19], we develop a human-and-LLM collaboration pipeline to create high-quality, long-form QA data from scratch. To add more data for medium (1-3 minutes) length videos, we introduce a synthetic data generator that uses our own captioning model to summarize and annotate extended videos (segmented into clips) and then formulates questions from those captions and the video's transcript.

Grounding capabilities are vital. We extend the 2D pointing paradigm popularized in image-based VLMs [29, 60, 178] into the temporal domain. Our models can not only point to objects in a frame, but also identify the moment an action happens or continuously track an object across a video. We created dedicated datasets for both video-pointing in space and time (*e.g.* "click the moment and location where X occurs"), and video-tracking (continuously indicating an object's position whenever it appears).

Existing video grounding datasets tend to be narrow in scope or vocabulary, which is insufficient for training general models that can respond to arbitrary user input [3, 111]. We address this by generating large-scale video grounding data covering diverse actions and objects (including many high-frequency everyday objects and complex referring expressions), and we complement it with data converted from several academic sources (*e.g.* reference video segmentation benchmarks) to ensure broad coverage. Finally, we construct a multi-image pointing dataset using PixMo-Points [29], enabling our model to output points on multiple images.

All Molmo2 variants are trained in a three-stage pipeline: (1) an image-captioning and image-pointing pre-training stage, (2) a joint supervised fine-tuning stage on our integrated multimodal dataset mixture (images, videos, and multi-image inputs), and (3) a short long-context training stage on the same data. We introduce several training innovations that further boost performance: a *novel token-weighting scheme* during fine-tuning to balance learning from diverse tasks, as well as efficient training techniques like *sequence packing* and a *message-tree schedule* that dramatically increase training throughput. We also show that enabling *bi-directional attention* between visual tokens yields notable gains.

We evaluate Molmo2 across a broad spectrum of established benchmarks, and also propose new evaluation sets for the less-explored capabilities we target (such as dense video captioning and open-vocabulary video pointing). On short-video understanding, Molmo2 achieves results on par with or better than existing models; for example, it outperforms previous open models on benchmarks like MVBench [78] and MotionBench [54], and even challenges some proprietary models' performance on these tasks. In tasks like visual counting and captioning, Molmo2 (even at 4B scale) is only outperformed by the strongest closed-source systems (*e.g.* Gemini 3.0 [45]), demonstrating the benefits of our fine-grained grounding data. Molmo2 also establishes new state-of-the-art results in video grounding (both tracking and pointing), substantially ahead of prior open models [3, 111], all while maintaining strong performance on traditional image and multi-image benchmarks [59, 93]. A human preference evaluation ranks Molmo2 as equal or better than existing open-weight models and ahead of a few proprietary models, including GPT-5 [114] and Claude Sonnet 4.5 [5], showing its general-purpose capabilities.

We release three versions of Molmo2: 4B and 8B models based on the Qwen3 LLMs [169], and a 7B model based on the OLMo LLM [112], to demonstrate what can be achieved with a fully-open language model. All our code, data, and models will be made open source.

Dataset Group	Description	Rate(%)	Datasets	Examples
Captions/Long QA	Captioning and long-form question answering data on images and videos, including Molmo2-Cap, -AskModelAnything, -MultiImageQA and PixMo-Cap, -AskModelAnything and -CapQA.	13.6	6	1.2m
Image QA	Multiple-choice and short answer image QA data, including Molmo2-SynMultiImageQA, open-source image datasets [48, 130, 101, 102, 103, 105, 65, 125, 94, 95, 14, 2, 61, 62, 107] following Molmo with CoSyn [172] instead of PixMo-Docs, and open-source multi-image datasets [132, 89, 58].	22.7	32	2.4m
Video QA	Multiple-choice and short answer video QA, including Molmo2-CapQA, -SubtitleQA, and various open video datasets [80, 74, 154, 184, 115, 160, 57, 167, 163, 173, 155, 156, 138, 38, 86, 54, 46, 109, 64, 42, 134, 188, 15, 49, 26, 141, 75, 77, 161, 123, 159]. Downsampled since video-benchmarks converge quickly.	18.2	32	2.4m
Image Pointing	PixMo-Points and PixMo-Count, CoSyn-Point [172], and Molmo2-MultiImagePoint. PixMo-Points is weighted to emphasize high counts. Downsampled since it was seen during pre-training.	9.1	4	1.1m
Video Pointing	Molmo2-VideoPoint and AcademicVideoPoint. Upsampled since this task is slow to converge.	13.6	7	0.37m
Video Tracking	Molmo2-VideoTrack and AcademicVideoTrack. Re-weighted to emphasize tail concepts.	13.6	22	0.80m
NLP	Text-only SFT data from Tulu [71] to preserve performance on natural language understanding.	9.1	1	0.99m

Table 1 We create nine new datasets (in pink) to train Molmo2. We also include a suite of image and language data from academic datasets into our training mix. We categorize all datasets into categories and show each categories' sampling rate, dataset count, and total training examples after filtering and formatting the data into message trees. See Section 2 and the appendix for details.

2 Data

We create five human-annotated datasets and four synthetic datasets, and additionally curate two datasets by repurposing existing open-source data. We summarize their design and collection pipelines below; see the appendix for details.

Molmo2-Cap (human). We collect 104k video-level and 431k clip-level dense captions from annotators, targeting both high detail and broad diversity. Videos are drawn from multiple large-scale sources [180, 147, 153, 184], starting from a pool of over 10M clips, then filtered for informativeness and sampled for diversity to obtain a balanced subset.

Obtaining dense video captions is challenging because annotators must describe dynamic events alongside fine-grained visual details [69]. We use a two-stage pipeline: annotators first describe short clips, then summarize the entire video. As in PixMo-Cap [29], annotators speak their descriptions, which are transcribed with Whisper-1 [120] and then rewritten by a text-only LLM for coherence. We condition annotators to describe dynamic visual details (*e.g.* object or event changes over time) by prompting them with a set of predefined questions. To add any missing low-level details, we use Molmo to generate frame-level captions and an LLM to merge the clip and frame captions into a single long caption. This produces the densest video caption dataset to date, averaging 924 words per video, compared to 75 words in Video Localized Narratives [141], 89 and 100 in RCap and RDCap [22], 280 in ShareGPT4-Video [19], and 547 in LLaVA-Video-178K [184].

Molmo2-AskModelAnything (human). We collect 140k human-authored video QA pairs. Using video captions, we cluster videos into 31 categories and sample them evenly to promote data diversity. Annotators then write specific, fine-grained questions (*e.g.* about text, actions, or temporal relations), while we discourage counting questions (handled separately by pointing data), overly generic prompts, or questions requiring expert knowledge. For each question, we first obtain an initial answer from an LLM (Claude Sonnet 4.5) conditioned on a caption generated by an early Molmo2 captioner. Annotators either accept the answer or

iteratively refine it through dialogue with the LLM. Finally, we post-process all QA pairs with an LLM filter to remove non-English, mismatched, or counting questions. We remove counting questions since the model should point for those questions instead of producing a pure text response.

Molmo2-CapQA and -SubtitleQA (synthetic). To build large-scale synthetic video QA, we use a video captioner trained on Molmo2-Cap to caption videos from YT-Temporal [180] and YouTube keyword search. We segment each video into multiple scenes and caption each scene instead of the entire video to encourage detailed descriptions. An LLM then uses these captions and video metadata to generate 1M QA pairs (200k videos, 5 QA per video). For SubtitleQA, we transcribe the video audio with Whisper-1 and additionally prompt the LLM with the transcript to create 300k QA pairs (100k videos, 3 QA per video) that require reasoning over both visual content and language.

Molmo2-VideoPoint (human). To improve Molmo2's counting and spatial-temporal localization, we collect over 650k video pointing queries on 280k videos, with an average of 6 points per video, targeting eight diverse categories: objects, animals, actions/events, referring expressions, indirect references, spatial references, comparative references, and visual artifacts/anomalies (for generative videos only). We generate queries by using LLM on video captions from an early version of Molmo2. Annotators first identify the frame where an object appears and then click on its exact location in the frame. Frames were obtained at 2 fps.

Molmo2-VideoTrack (human). We collect point-based object-tracking data covering 3.6k video clips and 15k complex natural language queries, with an average of 2.28 objects per query. Our dataset collection follows Ref-VOS [12] by asking users to re-label existing tracking annotations. For each video, we display either segmentation or bounding box object tracks, and ask annotators to craft non-trivial text queries that apply to a subset of objects. The queries are then validated in a separate validation round. We source videos and tracks from diverse open-source segmentation tracks [12, 33, 108, 122] and bounding-box tracks [133, 183, 126, 144, 44, 30, 186, 37, 140, 174].

AcademicVideoPoint and AcademicVideoTrack (curated). For pointing, we convert existing object tracking annotations from six datasets [6, 143, 117, 12, 66, 31] into 49k pointing and counting QAs. We first obtain the timestamp of the first frame in which an object appears and then randomly sample a point in the object's mask with a Gaussian distribution around the mask center. For tracking, we repurpose 7 existing Ref-VOS datasets [66, 127, 31, 6, 143, 166, 7] to obtain point tracking supervision data. In addition, we process 11 bounding-box based tracking datasets [182, 55, 116, 110, 53, 39, 181, 72, 151, 152, 189] by using SAM-2 to generate segmentation masks and corresponding point tasks.

Molmo2-MultiImageQA (human). We collect QA data on semantically related image sets to support real-world multi-image queries. We form image sets by grouping images whose captions (generated by a PixMo-Cap–trained model) have high sentence-level similarity; each set contains 2–5 images (2.73 on average). Human annotators then write questions over each set, and answers are refined through the same human–LLM loop as above. In total, we construct 45k image sets from 96k unique images and 72k QA pairs.

Molmo2-MultiImagePoint and -SynMultiImageQA (synthetic). To improve multi-image grounding, we construct a dataset of over 470k pointing and counting examples by applying soft clustering over images in PixMo-Points. Image sets are formed using a combination of single-token and sentence-level label embedding similarities, producing sets of 2–5 semantically related images (mean set size: 3.24). For each image set, we first normalize all human-provided labels via lowercasing, punctuation, and whitespace normalization, and synonym consolidation. We then use a large language model to resolve these normalized labels into a single canonical description that is semantically consistent across the set. This canonical label defines the shared entity or concept to be pointed to and counted across all images in the set. During training, we stochastically sample from the original (pre-canonicalized) human annotations rather than always using the canonical label, thereby preserving lexical diversity and improving robustness to annotation variability.

For Molmo2-SynMultiImageQA, we adapt CoSyn [172] to create 188k synthetic multi-image examples with text-rich images such as charts, tables, and documents.

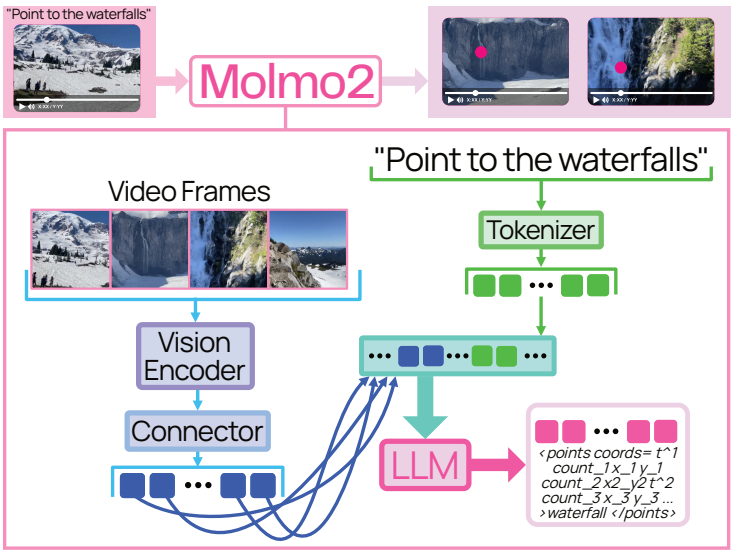


Figure 2 Molmo2 follows the standard design of connecting a vision encoder and a language model to process video inputs.

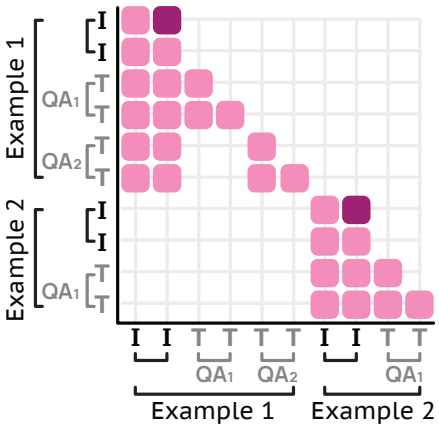


Figure 3 Attention mask for a *packed* sequence with two examples. The first contains two QA pairs for one image. Frame tokens (dark pink) have forward attention, while masking blocks cross-attention between different examples (lower-left empty block) and between distinct QA pairs within the same example (upper empty block).

3 Training

This section provides an overview of our model and training pipeline. See the appendix for additional details.

3.1 Architecture

Our model architecture follows the common design of combining a pre-trained LLM and a vision transformer (ViT) [36] via a connector module [29, 89]. Visual inputs are split or resized into fixed-size crops, which are encoded into patch-level features by the ViT. The patch-level features are then pooled, projected by the connector, and passed as visual tokens, along with any text inputs, to the LLM. Figure 2 provides an overview.

Cropping. For input images, we use a single crop of the down-scaled image as well as up to K overlapping crops tiling the image to allow higher-resolution processing [29]. Images that cannot be tiled by K crops are downscaled. We use $K = 8$ during training and $K = 24$ during inference. For videos, we sample frames at $S = 2$ fps as single crops (downscaling if needed) to reduce computational costs when processing long videos. We set a maximum of $F = 128$ frames (or $F = 384$ for long-context training). If the video length is longer than F/S, we uniformly sample F frames. In both cases, the last frame is always included since most video players will display the last frame after the video finishes playing, and it therefore might have special importance to users.

Vision-language connector. The connector uses features from the third-to-last and ninth-from-last ViT layers, following [29]. For images, 2×2 patch windows are pooled into a single vector using a multi-headed attention layer, where the mean of the patches serves as the query. For video frames, a 3×3 patch window is used instead to reduce the token count. We use the same shared parameters for the connector for both image and video frame pooling. Finally, the pooled features are projected using a shared MLP.

LLM. The LLM takes as input the visual tokens interleaved with text timestamps (for videos) or image indices (for multi-image input). For multi-crop images, we include column tokens [29] to indicate the image's aspect ratio. We do not include column-tokens for single-crop images since they are always square. We also add image and frame start tokens and include subtitles (marked with text timestamps) as text after the visual input if available. We allow image tokens (even if they are from different frames/images) to forward-attend to one another [43, 136], which we find can increase performance.

3.2 Training

We use a simple three-stage design: a light-weight image-only pre-training stage, a joint video/image supervised fine-tuning (SFT) stage, and then a short long-context SFT stage. We train on the Molmo2 data, image data from PixMo, and various open-source datasets. We review those stages and additional training details here, but leave most details to the appendix.

Pre-training. Our pre-training stage includes dense captioning with length conditioning and transcript prediction using PixMo-Cap, following [29]. We add NLP data using the supervised fine-tuning data from Tulu [71], filtered to remove non-English content and code, to better preserve language capabilities. Additionally, we add pointing data from PixMo-Points, PixMo-Count, and CoSyn-Point [172]. We find that adding pointing data during pre-training leads to better and more stable pointing performance. We use 60% captioning, 30% image pointing, and 10% natural language for the mixing ratios. We train for 32k steps with a batch size of 128, which results in about 4 epochs of training on PixMo-Cap. All parameters are fine-tuned, and we use separate learning rates for the ViT, connector, and LLM following [29].

SFT. Our data mixture combines PixMo [29], the Molmo2 datasets, Tulu, and other open-source video and image datasets. We divide these datasets into categories and manually assign each category a sampling rate based on empirical tests; see Table 1. Within each category, we sample datasets proportionally to the square root of each dataset size, with the addition of some manual rebalancing, such as downsampling large synthetic datasets. We train for 30k steps with a batch size of 128 and a max sequence length of 16,384.

Long-context SFT. Finally we do a third stage of training with a longer context length [17, 135] on the same SFT data mixture. During this stage we increase the sequence length to 36,864, set $F = 384$, train for 2k steps, and use context parallelism (CP) on the LLM so each example is processed by a group of 8 GPUs. We employ Ulysses attention [56] for the LLM context parallelism as its all-gather offers flexibility with the custom attention masks used by our packing and message tree system [4]. We also distribute video frame processing by the vision encoder and the attentional pooling after that across each context parallel group and find it very effective in reducing the memory footprint of the model. We only do long-context training as a short final training stage since its adds significant overhead to the training.

Pointing and tracking. We represent point coordinates with a compressed plain-text format that includes normalized x and y coordinates, a timestamp (for video) or an image index (for images), and an integer ID that is unique for each distinct object to enable tracking and counting. Points are sorted based on time/image index, then x, y coordinates. During SFT, we use a maximum of 24 crops instead of 8 for 30% of images with pointing annotations to ensure that pointing can generalize to high-resolution images. For video pointing, we train with examples with up to 60 points annotated. Additionally, we construct and train on multi-turn conversations with multiple pointing or counting queries for the same videos. For tracking, we also add auxiliary tasks of predicting only the first and last frames in which the objects appear, or tracking from an input query and point.

Token weighting. Our data includes both multiple choice questions with a single output token and long video captions with 4,000+ output tokens. These long-output examples can easily become the large majority of loss tokens even if they are sampled rarely, which can cause degradation on short-answer or multiple-choice tasks. As a solution, we adjust the weighting of some examples when they are used with the loss. We use a fixed weight of 0.1 for video captions and 0.2 for pointing, since both of these tasks can have very long, dense outputs. For other tasks we follow the heuristic of $\frac{4}{\sqrt{n}}$ where n is the number of answer tokens, which better balances long and short output training examples.

Packing. Examples can have anywhere from hundreds (pure-text or small images) to 16k+ (videos with subtitles or long videos during long-context training) of tokens. To avoid wasteful padding when creating training batches, we use packing to merge multiple short examples into a single long sequence. Packing is non-trivial for vision-language models due to the need to efficiently pack both crops for the ViT and tokens for the LLM, and the need to support models with different approaches to converting images/videos into tokens. We develop an on-the-fly packing algorithm that builds maximally efficient packed sequences from a small pool of in-memory examples and can be integrated into standard PyTorch data loaders.

Message trees. We encode videos and images with multiple annotations as *message-trees*. The visual input is

encoded as the first message, and each annotation becomes a different branch. The tree is linearized as a single sequence with a custom attention mask to prevent branches from cross-attending to each other. On average, examples in our data have 4 annotations, and packing is able to fit 3.8 examples into a 16348 token sequence during SFT, leading to 15x training efficiency. Figure 3 shows the attention masking.

4 Evaluation

We evaluate Molmo2 on standard video academic benchmarks and on our new benchmarks for video captioning, counting, and pointing, as well as a large-scale human-preference study. Then we report results for ablations, task-specific Molmo2 variants, and test-time scaling. See the appendix for details, additional ablations, evaluations on NLP benchmarks, and additional discussion.

4.1 Overall results

Model	NextQA test [160]	PerceptionTest test [115]	MVBench test [78]	Tomato test [128]	MotionBench val [54]	TempCompass test MCQ [91]	Video-MME test [40]	Video-MME-Sub test [40]	LongVideoBench val [157]	MLVU test MCQ [187]	LVBench test [150]	VideoEvalPro test [98]	Ego Schema test [100]	Molmo2 Caption test F1 Score	Molmo2 Count val accuracy	Short QA avg.	Long QA avg.	Average	Elo Score	Elo Rank
API call only																				
GPT-5 [114]	86.3	79.4	74.1	53.0	65.4	80.4	83.3	86.9	72.6	77.7	65.2	68.8	75.6	50.1	35.8	73.1	76.3	70.6	1031	10
GPT-5 mini [114]	83.2	72.0	66.5	44.1	59.9	74.9	77.3	82.3	69.7	69.1	54.7	60.1	70.9	56.6	29.8	66.8	69.8	65.0	1076	4
Gemini 3 Pro [45]	84.3	77.6	70.4	48.3	62.6	82.8	88.6	87.5	75.9	75.7	77.0	78.0	68.9	36.0	37.1	71.0	78.8	70.0	1082	3
Gemini 2.5 Pro [25]	85.3	78.4	70.6	48.6	62.0	81.9	87.8	87.8	76.8	81.5	75.7	78.4	72.2	42.1	35.8	71.1	80.4	71.2	1096	1
Gemini 2.5 Flash [25]	81.8	74.7	67.0	39.1	59.3	80.2	84.2	84.2	73.1	75.1	64.9	69.6	70.2	46.0	31.9	67.0	74.5	66.7	1084	2
Claude Sonnet 4.5 [5]	79.2	64.3	62.1	39.6	58.5	72.8	74.2	80.5	65.1	64.0	50.5	50.5	73.1	26.0	27.2	62.8	66.4	59.6	1008	12
Open weights only																				
InternVL3.5-4B [149]	80.3	68.1	71.2	26.8	56.5	68.8	65.4	68.6	60.8	52.0	43.2	46.5	58.9	7.7	26.3	62.0	56.5	53.4	935	18
InternVL3.5-8B [149]	81.7	72.7	72.1	24.6	56.6	70.3	66.0	68.6	62.1	53.2	43.4	48.1	58.6	7.8	26.1	63.0	57.1	54.1	941	19
Qwen3-VL-4B [169]	81.4	70.7	68.9	31.8	58.6	70.8	69.3	74.0	62.8	58.4	56.2	49.8	68.4	25.2	25.3	63.7	62.7	58.1	1048	7
Qwen3-VL-8B [169]	83.4	72.7	68.7	35.7	56.9	74.3	71.4	75.2	62.4	57.6	**58.0**	50.3	69.8	26.7	29.6	65.3	63.5	59.5	1054	6
Keye-VL-1.5-8B [170]	75.8	64.2	56.9	33.0	55.1	**75.5**	**73.0**	**76.2**	66.0	53.8	42.8	54.9	56.3	25.4	27.2	60.1	60.4	55.7	952	17
GLM-4.1V-9B [137]	81.3	74.2	68.4	30.0	59.0	72.3	68.2	75.6	65.7	56.6	44.0	51.1	62.6	18.4	26.6	64.2	60.5	56.9	962	14
MiniCPM-V-4.5-8B [176]	78.8	70.9	60.5	29.8	59.7	72.7	67.9	73.5	63.9	60.6	50.4	54.9	49.6	29.3	26.3	62.1	60.1	56.6	975	13
Eagle2.5-8B [17]	85.0	81.0	74.8	31.0	55.7	74.4	72.4	75.7	66.4	60.4	50.9	58.6	**72.2**	22.8	28.9	67.0	**65.2**	60.7	1019	11
Open models																				
PLM-3B [22]	83.4	79.3	74.7	30.9	60.4	69.3	54.9	59.4	57.9	48.4	40.4	46.2	66.9	12.3	24.4	66.3	53.5	53.9	841	20
PLM-8B [22]	84.1	**82.7**	**77.1**	33.2	61.4	72.7	58.3	65.4	56.9	52.6	44.5	47.2	68.8	10.9	26.6	68.5	56.2	56.2	853	21
LLaVA-Video-7B [184]	83.2	68.8	58.6	24.9	54.2	66.6	63.3	69.7	58.2	52.8	44.2	47.8	57.3	19.9	21.4	59.4	56.2	52.7	959	15
VideoChat-Flash-7B [79]	85.5	76.5	74.0	32.5	60.6	69.4	65.3	69.7	64.7	56.0	48.2	51.2	51.3	14.8	21.6	66.4	58.1	56.1	956	16
Molmo2 family: Open weights, Open data (no distillation), Open code																				
Molmo2-4B	85.5	81.3	75.1	**39.8**	61.6	72.8	69.6	75.7	**68.0**	**63.0**	53.9	59.9	61.2	39.9	34.3	69.3	64.5	62.8	1041	8
Molmo2-8B	**86.2**	82.1	75.9	39.6	**62.2**	73.4	69.9	75.8	67.5	60.2	52.8	**60.4**	62.0	**43.2**	**35.5**	**69.9**	64.1	**63.1**	**1057**	**5**
Molmo2-O-7B	84.3	79.6	74.8	36.2	60.6	73.0	64.9	69.2	63.7	55.2	49.6	55.1	56.8	40.1	33.2	68.1	59.2	59.7	1033	9

Table 2 Video benchmark results for a range of proprietary APIs, open-weight baselines, video-specialized models, and our Molmo2 family across video understanding, captioning, and counting benchmarks. The result of the best-performing open-weight model is in **bold**, and the second best is underlined.

We evaluate captioning by constructing Molmo2-CapTest, an eval set of 693 Creative Commons-licensed videos with at least four human-annotated captions. We use an LLM-as-a-judge to compute precision, recall, and F1 for statements made in the model's caption relative to statements from the annotator's captions, similar to Molmo's image captioning metric [29]. For counting, we construct Molmo2-VideoCount by using our Molmo2-VideoPoint pipeline to collect 533 diverse examples that cover object, action, and animal queries with up to 60 points.

For the human preference study, we collect questions from human annotators and manually filter them to prioritize open-ended questions over straightforward ones, resulting in 450 questions. We added another 51 videos for captioning queries. We sample two model outputs and gather pairwise preferences on them from annotators. We collect over 105K ratings (501 per model pair). From this data, we calculate an Elo ranking using the Bradley-Terry model [21].

We obtain results for all models on all tasks. We prioritize author-published results but fill in missing results with the best previously reported values from technical reports or papers. If data is still missing, we compute it ourselves. We try to follow the author's eval setup, but note that eval details (*e.g.*, prompting or number of frames) are sometimes not public, so results should be interpreted carefully.

During inference, we use 384 frames and greedy decoding. For human evaluations and video captioning, we use top_p=0.95, temperature=0.7, and frequency_penalty=0.1 instead, which produces more natural results when generating long outputs.

Results are in Table 2; we highlight a few key takeaways:

- Molmo2 is SoTA on short video benchmarks, captioning, and counting among non-proprietary models
- Molmo2 outperforms previous fully-open models but lags behind the best open-weight models. We believe this is due to a lack of open-source long (10+ minutes) training data and computational limitations that made it challenging to run extensive ultra-long context training.
- Molmo2 ranks equal to or better than other open-weight models on human preference, and is far ahead of previous fully-open models.

4.2 Grounding results

	BURST [6] VC (test)		Molmo2-VC		Molmo2-VP		
Model	Acc.	Close acc.	Acc.	Close acc.	F1	Recall	Precision
API call only							
GPT-5 [114]	43.1	73.7	<u>35.8</u>	50.3	4.1	4.4	4.2
GPT-5 mini [114]	46.0	73.0	29.8	49.3	2.2	2.2	2.2
Gemini 3 Pro [45]	44.0	71.7	**37.1**	53.1	20.0	27.4	19.8
Gemini 2.5 Pro [25]	41.6	70.0	<u>35.8</u>	**56.5**	13.0	14.5	13.6
Gemini 2.5 Flash [25]	38.7	70.0	31.9	48.2	11.1	11.2	12.2
Claude Sonnet 4.5 [5]	42.4	72.6	27.2	45.1	3.5	3.7	4.3
Open weights only							
Qwen3-VL-4B [10]	38.9	74.7	25.3	44.3	0.0	0.0	0.0
Qwen3-VL-8B [10]	42.0	74.4	29.6	47.7	1.5	1.5	1.5
Molmo2 family: Open weights, Open data (no distillation), Open code							
Molmo2-4B	<u>61.5</u>	**76.1**	34.3	<u>56.1</u>	**39.9**	**42.7**	**39.4**
Molmo2-8B	60.8	75.0	35.5	53.3	<u>38.4</u>	<u>39.3</u>	<u>38.7</u>
Molmo2-O-7B	**61.6**	<u>76.0</u>	33.2	50.5	35.8	35.8	37.9

Table 3 Video counting and pointing results. Molmo2 scores highest on BURST-VC and Molmo2-VP and second highest on Molmo2-VC's close accuracy, slightly behind Gemini 2.5 Pro.

Video counting and pointing. For counting, we also evaluate on BURST-VideoCount, a counting benchmark of 2.2k examples derived from the ground-truth tracks in the BURST test set [6]. We report the close accuracy metric (correct if $|pred - gt| \leq \Delta$, where $\Delta = 1 + \lfloor 0.05 \times gt \rfloor$), which rewards being close to the correct answer. For pointing, we build Molmo2-VideoPointVal (Molmo2-VP) by running SAM 2 [122] to gather object segmentation masks within a 3-second window centered around the annotated spatial-temporal points in Molmo2-VideoPoint, and manually filter out examples with incorrect masks, leaving a total of 181 examples. For video pointing, we report the F1, recall, and prediction metrics, measuring how well the generated points match the ground-truth masks.

Model	MeViS [31] valid $\mathcal{J}\&\mathcal{F}$	MeViS [31] valid-u $\mathcal{J}\&\mathcal{F}$	MeViS [31] valid-u F1	MeViS [31] valid-u HOTA	Ref-YT-VOS [127] valid $\mathcal{J}\&\mathcal{F}$	Ref-YT-VOS [127] valid F1	Ref-YT-VOS [127] valid HOTA	Ref-Davis [66] valid $\mathcal{J}\&\mathcal{F}$	Ref-Davis [66] valid F1	Ref-Davis [66] valid HOTA	ReasonVOS [11] test $\mathcal{J}\&\mathcal{F}$	ReasonVOS [11] test F1	ReasonVOS [11] test HOTA
API call only													
GPT-5 [114]	23.4	26.5	17.3	14.0	30.9	21.0	18.4	25.2	17.0	11.6	24.7	13.6	10.7
GPT-5 mini [114]	15.7	15.4	8.5	6.8	16.2	7.4	6.2	8.4	3.4	2.3	14.6	4.2	3.4
Gemini 3 Pro [45]	42.5	51.1	42.3	36.0	55.0	49.1	45.5	66.6	60.8	55.7	52.6	48.5	42.1
Gemini 2.5 Pro [25]	40.7	52.8	41.2	35.0	45.1	44.5	40.5	45.6	62.7	56.6	44.0	50.2	42.4
Gemini 2.5 Flash [25]	27.6	31.8	24.0	19.9	36.0	32.8	30.0	31.6	36.7	30.0	26.5	25.8	21.0
Open weights only													
Qwen3-VL-4B [169]	29.7	30.6	23.3	18.7	32.1	29.0	26.5	44.4	33.1	26.9	26.5	17.0	13.5
Qwen3-VL-8B [169]	35.1	34.4	30.1	23.8	48.3	42.1	37.6	41.0	41.6	33.2	24.9	22.3	17.5
Specialized open models													
VideoLISA [11]	44.4	53.2	–	–	63.7	–	–	68.8	–	–	47.5	–	–
VideoGLaMM [121]	45.2	50.6	–	–	66.8	–	–	69.5	–	–	33.9	–	–
Sa2VA-8B [177]	46.9	57.0	–	–	**70.7**	–	–	75.2	–	–	55.5	–	–
Sa2VA-Qwen3-VL-4B [177]	36.7	57.1	–	–	68.1	–	–	**76.0**	–	–	50.0	–	–
Molmo [29] + SAM 2 [122]	46.9	51.5	53.8	–	64.6	71.1	–	65.2	74.5	–	45.7	50.3	–
VideoMolmo-7B [3]	53.9	57.0	59.4	–	67.3	73.7	–	72.5	75.4	–	51.1	50.3	–
Molmo2 family: Open weights, Open data (no distillation), Open code													
Molmo2-4B	**63.3**	70.0	75.5	72.4	70.2	**80.4**	**78.8**	73.5	**83.1**	**81.1**	61.9	66.5	64.0
Molmo2-8B	62.3	**70.8**	75.9	**72.6**	70.2	78.7	77.3	72.7	81.3	78.7	**65.8**	**70.8**	**68.6**
Molmo2-O-7B	58.4	69.7	**76.1**	72.3	67.9	77.7	76.1	70.4	79.2	76.0	62.6	67.5	65.1

Table 4 Tracking Results on Academic Benchmark. $\mathcal{J}\&\mathcal{F}$ is reported for specialized segmentation or points-to-segmentation models. F1 is the point accuracy measured for VLMs that can generate points per frame. HOTA [97] is the tracking accuracy that accounts for association accuracy for models that provide tracking IDs.

Model	Animals $\mathcal{J}\&\mathcal{F}$	Animals F1	Animals HOTA	Person $\mathcal{J}\&\mathcal{F}$	Person F1	Person HOTA	Sports $\mathcal{J}\&\mathcal{F}$	Sports F1	Sports HOTA	Dancers $\mathcal{J}\&\mathcal{F}$	Dancers F1	Dancers HOTA	Misc $\mathcal{J}\&\mathcal{F}$	Misc F1	Misc HOTA	Overall $\mathcal{J}\&\mathcal{F}$	Overall F1	Overall HOTA
API call only																		
GPT-5 [114]	41.4	20.6	20.3	16.5	4.5	4.2	14.4	2.0	2.5	33.8	11.7	11.5	14.6	2.2	1.6	23.5	7.5	7.5
GPT-5 mini [114]	21.7	7.8	8.0	8.6	1.6	1.5	10.7	0.6	0.8	15.6	2.1	2.0	13.5	0.6	0.4	12.7	2.1	2.1
Gemini 3 Pro [25]	70.4	62.3	60.0	44.5	30.7	29.2	23.4	10.3	8.8	55.6	44.3	37.8	35.3	18.3	14.4	44.6	32.2	29.1
Gemini 2.5 Pro [25]	69.3	56.8	53.2	50.0	33.6	31.9	29.7	10.8	8.9	55.9	39.4	32.2	34.7	17.6	18.3	47.9	31.2	27.8
Gemini 2.5 Flash [25]	58.0	46.6	44.4	38.9	21.4	20.1	13.2	6.2	5.5	48.0	29.0	25.1	21.9	5.7	4.6	36.2	21.8	19.8
Open weights only																		
Qwen3-VL-4B [169]	57.2	11.5	12.3	35.1	12.0	11.2	3.8	0.4	0.4	34.6	6.9	5.7	17.5	6.2	4.2	28.5	7.2	6.7
Qwen3-VL-8B [169]	63.8	52.3	50.2	35.4	20.3	18.9	5.2	1.7	1.4	31.3	19.0	16.7	16.3	6.2	4.2	28.7	18.0	16.5
Specialized open video models																		
VideoLISA [11]	67.8	–	–	35.8	–	–	32.9	–	–	53.6	–	–	25.8	–	–	43.3	–	–
VideoGLaMM [121]	63.9	–	–	26.2	–	–	34.3	–	–	46.0	–	–	22.3	–	–	37.9	–	–
Sa2VA-8B [177]	74.3	–	–	45.5	–	–	30.7	–	–	53.3	–	–	**49.1**	–	–	46.9	–	–
Sa2VA-Qwen3-VL-4B [177]	73.3	–	–	48.6	–	–	31.6	–	–	50.1	–	–	31.4	–	–	46.7	–	–
SAM 3 [16]	41.1	–	–	35.2	–	–	43.3	–	–	29.2	–	–	36.8	–	–	36.3	–	–
Molmo [29] + SAM 2 [122]	71.8	76.0	–	**52.7**	7.0	–	52.8	2.6	–	51.7	7.55	–	40.9	37.5	–	54.2	14.0	–
VideoMolmo-7B [3]	68.4	69.5	–	51.1	6.3	–	43.2	2.1	–	53.8	7.2	–	39.9	30.8	–	51.3	12.7	–
Molmo2 family: Open weights, Open data (no distillation), Open code																		
Molmo2-4B	**81.0**	**83.0**	**83.7**	43.7	**48.3**	47.7	59.7	53.1	54.3	**60.4**	**64.4**	**64.4**	43.1	35.1	31.3	**56.7**	**57.5**	**57.6**
Molmo2-8B	80.1	82.0	83.0	43.1	47.9	**48.0**	**59.8**	**53.3**	**54.8**	59.9	63.9	63.5	41.6	31.5	29.7	56.2	57.1	57.5
Molmo2-O-7B	80.1	81.9	82.8	41.5	45.5	45.4	54.1	47.6	48.6	57.7	61.0	60.3	45.0	**37.6**	**34.7**	53.7	54.2	54.2

Table 5 Tracking results on Molmo2-Track by video domain. Overall is the accuracy across all samples.

Results are shown in Table 3. Molmo2 is strong on the close metric, outperforming GPT 5. For Molmo2-VP, we carefully tune the prompts and try both point and bounding-box formats for our baseline models; however, we were unable to find a formulation that achieved very strong performance. Gemini Pro 3.0 reached the best score, but Molmo2 still significantly outperforms it.

Video object tracking. We evaluate video tracking on referring video object segmentation (VOS) benchmarks, where a point is considered correct if it lies within the ground truth segmentation mask. We additionally introduce Molmo2-Track, a benchmark covering more diverse domains with complex object movements and occlusions, to evaluate Molmo2 on more challenging and realistic tracking tasks (see the appendix). Following [3], we use SAM 2 to convert point predictions to segmentation masks for evaluation. We report the Jaccard and F-measure ($\mathcal{J}\&\mathcal{F}$) metrics for measuring segmentation quality across all frames, and the F1 score for the points at 1 fps. For API models, we generate the bounding box and extract their center points as they fail to generate accurate points. Tables 4–5 show the results: 1) Molmo2 outperforms all baselines, including specialized segmentation models (in gray), across all benchmarks, particularly excelling on ReasonVOS and Molmo2-Track, which require complex reasoning and occlusion handling skills. 2) Gemini 2.5 Pro is the strongest API model, but it still struggles to generate accurate object tracks.

Model	AI2D test [65]	ChartQA test [102]	DocVQA test [104]	InfoQA test [105]	TextVQA val [130]	VQA v2.0 val [47]	RWQA [158]	MMMU val [179]	MathVista testmini [96]	CountBench [13]	PixMoCount test [29]	MuirBench [142]	MMIU [106]	Blink val [41]	Img QA avg.	MultiImg QA avg.	Average
API call only																	
GPT-5 [114]	97.1	89.6	88.9	83.0	78.7	79.7	80.8	81.8	82.7	90.8	67.2	78.6	71.0	66.5	83.7	72.1	81.2
GPT-5 mini [114]	95.8	88.2	86.7	82.2	79.1	72.1	77.0	78.7	79.2	87.1	74.4	71.4	64.5	68.7	81.9	68.2	78.9
Gemini 3 Pro [45]	98.7	93.7	87.1	86.9	74.1	74.1	73.6	85.2	89.1	96.1	90.0	86.1	72.1	87.4	86.2	81.9	85.3
Gemini 2.5 Pro [25]	94.3	82.7	91.5	82.0	70.3	67.1	77.4	79.6	84.6	90.8	73.8	74.5	68.9	73.7	81.3	72.4	79.4
Gemini 2.5 Flash [25]	95.9	76.8	91.1	80.9	73.0	69.4	74.5	79.0	81.2	86.7	63.9	73.5	61.2	70.2	79.3	68.3	76.9
Claude Sonnet 4.5 [5]	91.5	88.1	91.7	65.9	67.2	77.0	61.1	77.8	73.1	87.3	58.3	59.6	54.1	64.8	76.3	59.5	72.7
Open weights only																	
InternVL3.5-4B [149]	82.6	86.0	92.4	78.0	77.9	78.1	66.3	66.6	77.1	82.2	62.4	53.1	49.2	58.1	77.2	53.5	72.1
InternVL3.5-8B [149]	84.0	86.7	92.3	79.1	78.2	79.5	67.5	**73.4**	78.4	79.6	61.9	55.8	49.4	59.5	78.2	54.9	73.2
Qwen3-VL-4B [169]	84.1	84.6	95.3	80.3	81.0	81.7	70.9	67.4	73.7	85.5	58.0	63.8	43.2	65.8	78.4	57.6	73.9
Qwen3-VL-8B [169]	85.7	89.6	**96.1**	**83.1**	82.8	82.3	71.5	69.6	77.2	90.4	65.0	64.4	35.3	**69.1**	81.2	56.3	75.9
Keye-VL-1.5-8B [170]	89.5	**94.1**	93.4	74.9	81.5	79.3	73.5	71.4	**81.2**	81.6	57.4	51.2	50.3	54.9	79.8	52.1	73.9
GLM-4.1V-9B [137]	87.9	70.0	93.3	80.3	79.6	68.3	70.7	68.0	80.7	88.0	60.7	**74.7**	**62.4**	65.1	77.0	**67.4**	75.0
MiniCPM-V-4.5-8B [176]	86.5	87.4	94.7	73.4	82.2	64.1	72.1	67.7	79.9	83.9	62.8	53.3	46.5	42.0	77.7	47.3	71.2
Eagle2.5-8B [17]	84.5	87.5	94.1	80.4	83.7	82.4	76.7	55.8	67.8	90.2	90.2	61.8	48.4	45.8	81.2	52.0	75.0
Open models																	
PLM-3B [22]	90.9	84.3	93.8	74.6	84.3	84.4	72.4	41.2	59.1	87.1	63.0	25.7	40.6	55.4	75.9	40.6	68.3
PLM-8B [22]	92.7	85.5	94.6	80.0	**86.5**	85.6	75.0	46.1	59.9	91.8	68.0	23.5	27.4	56.0	78.7	35.7	69.5
Molmo1 family: Open weights, Open data (no distillation), Open code																	
MolmoE-1B [29]	86.4	78.0	77.7	53.9	78.8	83.9	60.4	34.9	34.0	87.2	79.6	-	-	-	68.6	-	-
Molmo-7B-O [29]	90.7	80.4	90.8	70.0	80.4	85.3	67.5	39.3	44.5	89.0	83.3	-	-	-	74.6	-	-
Molmo-7B-D [29]	93.2	84.1	92.2	72.6	81.7	85.6	70.7	45.3	51.6	88.5	84.8	-	-	-	77.3	-	-
Molmo-72B [29]	**96.3**	87.3	93.5	81.9	83.1	86.5	75.2	54.1	58.6	91.2	85.2	-	-	-	81.2	-	-
Molmo2 family: Open weights, Open data (no distillation), Open code																	
Molmo2-4B	95.6	86.1	87.8	78.6	85.0	86.6	75.4	50.9	56.7	93.9	88.1	60.5	55.5	57.5	80.4	57.8	75.6
Molmo2-8B	95.8	86.0	93.2	80.1	85.7	**87.0**	**77.6**	53.0	58.9	93.7	88.5	63.7	54.2	51.3	**81.7**	56.4	**76.3**
Molmo2-O-7B	93.7	84.9	90.4	77.9	84.7	86.6	73.6	45.8	54.2	**95.1**	**88.9**	58.4	51.7	50.5	79.7	53.5	74.1

Table 6 Image benchmark results for a range of proprietary APIs, open-weight baselines, and our Molmo2 family across image understanding and counting benchmarks. The result of the best-performing open-weight model is in **bold**. The Molmo1 models do not support multi-image input, so those evaluations are left blank.

4.3 Image results

We present image and multi-image benchmark results in Table 6. We follow the evaluation protocol from Molmo [29] and report the same 11-benchmark average for single-image benchmarks. As with videos, we collect results for all models by testing them ourselves if needed.

Generally, Molmo2 robustly outperforms previous open-data models. Molmo2 is a bit behind the best open-weight model on OCR-heavy benchmarks (such as DocVQA or InfoQA) but performs well on general QA tasks, including state-of-the-art performance on VQA v2.0 and RealWorldQA (RWQA). Counting is also a strength, most notably on the challenging PixMo-Count test set. However, Molmo2 is behind on open-weight reasoning benchmarks (MathVista, MMMU), possibly due to the lack of multi-modal reasoning training data. On multi-image tasks, Molmo2 performs competitively with most open-weight models, with the exception of GLM-4.1V-9B, which is notably ahead of all other models.

Model	Affordance	Spatial	Reasoning	Steerability	Counting	Average
Human	92.3	83.6	87.8	86.3	95.6	89.1
API call only						
Gemini-Robotics-ER-1.5 [1]	69.7	69.7	60.1	**67.5**	68.5	67.1
Gemini-2.5-Pro [25]	72.7	70.3	71.0	41.0	59.2	62.8
Open weights only						
Poivre-7B [171]	-	-	-	-	-	67.5
Qwen2.5-VL-32B-Instruct [168]	76.8	60.0	54.4	46.5	57.1	59.0
Qwen2.5-VL-72B-Instruct [168]	76.8	60.0	54.4	46.5	57.1	59.0
Qwen3VL [169]	81.3	65.6	60.6	23.5	61.2	58.5
Qwen3-VL-235B-A22B-Instruct [169]	-	-	-	-	-	58.3
Open models						
VisionReasoner-7B [92]	-	-	-	-	-	64.7
Molmo1 family: Open weights, Open data (no distillation), Open code						
Molmo-7B-D [29]	82.8	67.7	70.5	28.5	58.7	61.6
Molmo-72B [29]	**87.9**	70.3	69.4	37.0	54.6	63.8
Molmo-7B-O [29]	84.9	63.1	63.2	45.5	59.7	63.3
Molmo2 family: Open weights, Open data (no distillation), Open code						
Molmo2-4B	82.3	**71.8**	**72.0**	41.0	71.4	67.7
Molmo2-8B	84.8	71.3	71.5	44.5	71.4	**68.7**
Molmo2-O-7B	81.8	69.7	69.4	39.0	**72.4**	66.5

Table 7 Point-Bench results[1] baseline scores taken from the Point-Bench leaderboard. Qwen3-VL-235B-A22B-Instruct and VisionReasoner-7B scores were taken from their evaluation in Poivre [171], which did not include sub-category scores.

We evaluate image pointing on Point-Bench [20], results are in Table 7. Molmo2 surpasses all other models on the Point-Bench leaderboard[2] and the recent dedicated pointing model Poivre [171]. We attribute the gain on pointing compared to Molmo to the improved vision encoder, pointing pre-training, and token-weighting.

4.4 Ablations and specialized models

Next, we present ablations on our model, training strategy, and data. To avoid the high compute cost of training the full model, we train specialized 4B models on subsets of our data and use them for ablations. These tables use Gray rows to show specialized models with default settings; key takeaways are in the captions.

[1] An older version of this report include higher scores that were the result of an evaluation bug.
[2] As of 12/15/25

Data	QA avg.	Cap. F1
Video-Only	64.8	39.5
Molmo2-Cap Only	-	35.8

(a) Caption Specialization. Joint training with other video data improves the video caption performance.

Model	QA avg.	Cap. F1
Video-Only	**64.8**	39.5
No bidir	64.4	38.5
No token weighting	64.0	**40.0**
No time tokens	64.5	37.4
Video pool size 3x3 to 4x4	64.3	37.0

(b) Modeling. Bidirectional attention, token weighting, and time tokens significantly improve performance, while a larger pool size degrades video captioning.

Data	QA avg.	Cap. F1
Academic	62.9	5.0
+ QA	64.5	17.2
+ Cap	**65.3**	38.4
+ Cap/QA	64.8	**39.5**

(c) Video SFT data. Both Molmo2-Cap and Molmo2-QA improve performance compared to academic datasets only.

Data	Cap. R	Cap. P	Cap. F1
V	13.3	**66.7**	22.1
VF	25.4	59.5	35.5
VF+V	**25.6**	59.6	**35.8**
VF + F	22.4	59.4	35.6
VF + V + F	22.6	57.3	35.7

(d) Caption data. Using the video and frame merged caption (VF) is critical, but adding video (V) and/or frame (F) captions does not bring improvements.

Table 8 Video ablations. For ablations (a)(b)(c) we train models on only video data; ablation (d) has models with only video captions.

Strategy	BVC	MVC
Count	61.3	28.1
Point then count	**61.5**	**34.5**

(a) Counting strategy. Pointing is the key ingredient in Molmo2's counting abilities.

Data	BVC	MVC	MVP
Both	61.5	**34.5**	31.8
Molmo2-VP	60.0	34.3	**35.0**
Academic-VP	**61.6**	9.0	9.0

(b) Data source. Including both Molmo2- and AcademicVideoPoints performs the best overall.

Upsampling	BVC	MVC	MVP
Med-high	61.5	**34.5**	**31.8**
No	**62.4**	32.1	28.1

(c) Sampling strategy. Upsampling medium and high-count examples helps on MVC and MVP.

Table 9 Counting and pointing ablations. BVC represents Burst-VideoCount accuracy; and MVC and MVP are Molmo2-VideoCount accuracy and Molmo2-VideoPoint F1 on the validation sets.

Video ablations. Table 8 shows results and ablations with video-only and video-captioning-only data. We see that video QA data transfers positively to captioning (Table 8a) and vice versa (Table 8c). Table 8b shows bi-directional attention and token-weighting both boost QA performance, although token-weighting can slightly degrade caption performance. Meanwhile, removing frame timestamps diminishes both metrics, indicating that including temporal information is important, especially for captioning. Increasing the video pool size from 3x3 to 4x4 slightly lowers QA performance but causes a significant drop in captioning quality. We believe that this is because the video benchmarks are relatively high-level and do not require understanding small details, so decreasing the pooling size is not very harmful. This illustrates the importance of tracking the captioning metric in addition to the other benchmarks, which requires a much more fine-grained understanding

Model	$\mathcal{J}\&\mathcal{F}$	F1	HOTA
Tracking only	64.9	70.0	68.4
Tracking + Pointing	**65.7**	**71.1**	**69.4**

(a) Adding pointing. Training with pointing tasks helps tracking performance.

Data	$\mathcal{J}\&\mathcal{F}$	F1	HOTA
Academic (VOS)	64.3	68.8	66.7
+ Academic (bbox)	63.9	69.3	67.5
+ Molmo2 (VideoTrack)	**64.9**	**70.0**	**68.4**

(b) Tracking data source. We see progressive improvements from academic VOS, bounding box (bbox) tracks, to Molmo2 data.

Strategy	$\mathcal{J}\&\mathcal{F}$	F1	HOTA
Tracking	64.2	68.4	66.2
+ Temporal grounding	**64.8**	**69.4**	**67.2**
+ Single-point object tracking	64.3	68.8	66.7

(c) Tracking sub-tasks ablated on Academic VOS only. Temporal grounding helps, while single-point object tracking slightly degrades performance.

Table 10 Tracking ablations. We report average metrics across the five tracking benchmarks (the valid-u split for MeViS). HOTA [97] measures association accuracy.

of the video. Finally, captioning models based solely on human transcripts (V) produce worse results than those that include frame-level captions (VF), but training on a mixture of these captions does not lead to improvements (8d).

Video counting and pointing. Table 9 reports the performance of a specialized pointing model and ablating counting strategy, data, and data sampling. We observe that our two sources of pointing are complementary (Table 9b), that pointing before counting is much better than directly predicting the count (Table 9a), and that upsampling high-frequency points improves both counting and pointing (Table 9c).

Video object tracking. Table 10 shows ablations on task mixtures and data sources for tracking with a model trained only on our tracking data. Including our video pointing data improves performance, showing a moderate transfer from pointing to tracking (Table 10a). Using bounding box tracks and the Molmo2-VideoTrack dataset also leads to improvements (Table 10b). Supporting temporal grounding helps, while adding point-based single object tracking causes a slight degradation (Table 10c).

Post-training	Short video QA	Long video QA	Molmo2 Video Cap.	Image QA
With long-context SFT	69.4	67.4	39.9	80.6
No long-context SFT	69.6	64.4	42.3	80.5

Table 11 Long-context SFT ablation. Columns show the average of our 12 video benchmarks divided by short/long video benchmarks, using validation sets for EgoSchema, PerceptionText, and MLVU, video captioning F1, the average of the 11 image benchmarks using validation sets for InfoQA, DocQA, ChartQA, VQA v2, and AI2D.

Long context SFT. We compare the Molmo2-4B performance before and after long-context post-training in Table 11. We find that long-context post-training significantly improves model performance on long video QA benchmarks, while the video caption performance drops and performance on short video QA benchmarks and image QA benchmarks do not significantly change.

5 Related works

Multimodal LLMs. Multimodal LLM models have become popular in the last few years for image understanding and grounding tasks [29, 70, 136]. A common strategy for multimodal LLMs is to use CLIP-style image encoders and align image embeddings with the LLM input space via a connector module [29, 90]. Video LLMs also commonly extend the CLIP-style image encoding and use image embedders to individually embed each frame in a video [17, 22, 99]. Some have explored using pretrained video encoders in combination with

[2],

per-frame encoding or encoding 2 frames together [190, 146, 137], but using video encoders with more frames lags behind using image encoders (such as SigLIP 2 [139]). However, when encoding each frame of a video individually, the number of visual tokens increases linearly with the frame sampling rate and the length of the video. This leads to a high compute cost and has led to a rise in works exploring efficient video encodings [129, 164, 170, 79, 149].

The best performing video LLMs [114, 5, 25] are closed-source proprietary models. While they are very capable, not much is known about how these models are trained and what data they use. By contrast, while some open weight models have been released [146, 149, 170, 190, 17], most don't release their training recipes or don't release their training data. A few projects do release all the training details and data [22, 184], but use biased data generated by proprietary VLMs (such as GPT4 and LLaMA3 [18, 4]). Hence, there is a need for a fully open SoTA training pipeline for Video LLMs that does not use previously trained multimodal LLMs to generate data.

Video-language instruction tuning datasets. The popularity of Video LLMs has also led to an increase in methods to develop instruction-tuning data for them. The current dominant paradigm involves generating synthetic instruction data by first segmenting videos into clips, generating descriptive captions for each clip, and then using a powerful LLM to synthesize video-level captions and QA pairs [184, 17, 22, 19]. However, a critical limitation of these approaches is their reliance on closed-source Video-Language Models (VLMs) for the initial clip captioning step. This introduces an inherent, often proprietary, bias into the generated data, as the underlying VLM's training data and biases are inaccessible to the research community.

Our Molmo2-CapQA dataset is generated through a similar pipeline but utilizes a video captioner trained on our fully open Molmo2-Cap to generate video captions. We segment each video into multiple scenes, caption each scene, and then provide these to an LLM along with the video metadata to generate 1M QA pairs. Another strategy used for generating QA pairs is to have annotators work with an LLM provided with an image caption when generating QA pairs [29], and we extend the same to video data to generate our Molmo2-AskModelAnything.

Video tracking. Early video tracking focused on bounding boxes for a closed set of objects [110, 30]. Since then, the field has branched into specific subtasks, including track any point (TAP) [63, 35] and tracking object segmentations [53, 6]. Object segmentations improved accuracy and granularity, but tracking was still limited to a closed set of objects. Moving beyond a closed set of objects to an open vocabulary has led to a rise in language-guided video object segmentation (VOS) [166]. A variety of new specialized models have been trained to track object [11, 81, 3]. Unlike Molmo2, these models are specialized and do not support other capabilities.

Previous methods, like Ref-VOS [12] and MeVis [31], support the language-guided VOS task by augmenting existing tracking datasets with complex referring expressions. However, we noticed a lack of language prompts referring to multiple objects or diverse actions. For our Molmo2-VideoTrack dataset, we similarly add to existing datasets by asking annotators to craft non-trivial text queries that apply to object tracks, with a focus on queries that describe multiple objects. For segmentation masks, we source videos and tracks from diverse open-source segmentation tracks [12, 33, 108, 122] and use a data pipeline to produce masks from bounding-box tracks [133, 183, 126, 144, 44, 30, 186, 37, 140, 174].

Video pointing. Multimodal LLMs that support point grounding in an image have recently become quite common [29, 171, 25, 1, 10, 20]. The training data used in these works is collected using automated object detectors, using existing referring expression datasets [175, 88, 68] or through manual human annotation [29]. We extend the human annotation pipeline approach to videos by adding a frame-selection phase. We also propose generating some queries through an LLM based on the caption to ensure the queries are complex and diverse.

6 Conclusion

Open research needs open-source. Molmo2 supports open science by closing the gap between proprietary VLMs and the rest of the community.

Author Contributions

Christopher Clark, Jieyu Zhang, Zixian Ma, JaeSung Park, Rohun Tripathi, Sangho Lee and Mohammadreza Salehi collectively contributed to dataset construction, model training, and conducted numerous exploratory experiments for this project.

Christopher Clark led the project and focused on video modeling and training strategies, including experiments with the SFT mixture, the pre-training approach, and video modeling. He also wrote much of the core training code and implemented the packing and message tree systems.
Jieyu Zhang co-led the data effort on video datasets. He collected and filtered raw videos for Molmo2 video caption, video QA, and video pointing datasets, and contributed to the curation of these datasets. He helped the integration of other training/evaluation datasets and ran evaluations for many baseline models. He also helped add subtitle understanding to the model and ablations of the video SFT/caption models.
Zixian Ma co-led the data effort on video datasets. She designed human data collection interfaces and implemented them with help from Yinuo Yang. She collected the Molmo2-Cap, Molmo2-AskModelAnything, and Molmo2-VideoPoint datasets via Prolific. She led the training ablations on video counting and pointing and helped integrate academic training datasets. She ran the human preference and NLP evaluations.
Jae Sung Park led the effort to add tracking capability to Molmo2 as points. Together with Zhongzheng Ren and Vincent Shao, he designed the Molmo2-Track human annotation collection, curated existing academic tracking datasets for training, and built the pipeline to extract accurate point tracks. He introduced auxiliary grounding and single-point tracking objectives and performed ablations on mixtures of video tracking tasks. He and Zhongzheng Ren designed tracking evaluations across diverse VLMs and segmentation models.
Mohammadreza Salehi led the long-context post-training and co-led sourcing videos for training. He also contributed to training dataset construction, training on a mixture of images and videos, and evaluation of Molmo and API models.
Rohun Tripathi primarily worked on efficient modeling strategies. He developed learned and training free solutions to token allocation for different frames, with and without the input query. He implemented the initial training pipeline and details such as 3D position encoding and time tokens. He helped with training/evaluation set integrations, with a focus on long video understanding.
Sangho Lee led improvements to image modeling and training strategies and extended them to the multi-image setting. He also supported and directly conducted extensive ablation studies to develop effective training strategies for video modeling. In addition, he implemented the Hugging Face model and processor code and vLLM integrations.
Chris Dongjoo Kim led the data effort for multi-image datasets. In collaboration with Weikai Huang and Sangho Lee, he curated the MultiImageQA dataset. He also held full responsibility for the multi-image pointing capability, including dataset curation algorithms and model training.
Yue Yang led data curation for text-rich multi-image datasets, synthetically generating diverse question-answer pairs grounded in images such as charts, tables, and documents.
Zhongzheng Ren, **Yinuo Yang**, **Vincent Shao**, **Weikai Huang**, and **Ziqi Gao** all made significant dataset contributions.
Jitesh Jain, **Jianrui Zhang**, and **George Stoica** contributed to research discussions throughout the project and did exploratory experiments based on Molmo2.
Taira Anderson managed the project.
Winson Han designed the figures in this report.
Ali Farhadi advised the project.
Ranjay Krishna was the PI for the project.

Acknowledgements

This work would not be possible without the support of our colleagues at Ai2.

- We thank David Albright, Erin Bransom, Kristin Cha, Yvonne Chou, Karen Goodfellow, Malachi Hamada, Stephen Kelman, Ryan Kiskis, Sophie Lebrecht, Kelsey MacMillan, Crystal Nam, Lauren Olvera, Carissa Schoenick, Jeremy Tryba, Tina Weiss, Kyle Lo, Kyle Wiggers, and Will Smith for their important work for the Molmo2 public release.

- We thank the Ai2 Playground team, including Taylor Blanton, Byron Bischoff, Jon Borchardt, David Everhart, Michal Guerquin, Paul Laskowski, Caleb Ouellette, and Michael Schmitz, for constructing the excellent Molmo2 demo.
- We thank other members of the PRIOR team, including Maximilian Argus, Jaemin Cho, Jiafei Duan, Rose Hendrix, Amita Kamath, Yejin Kim, Tanmay Gupta, Peter Sushko, Eli VanderBilt, and Piper Wolters, for providing advice and feedback on various aspects of Molmo2.
- We thank the Prolific team for their support and our annotators on Prolific for providing us with high-quality data that is crucial to Molmo2.

This material is based upon work supported by the National Science Foundation under Award No. 2413244.

References

[1] A. Abdolmaleki, S. Abeyruwan, J. Ainslie, J.-B. Alayrac, M. G. Arenas, A. Balakrishna, N. Batchelor, A. Bewley, J. Bingham, M. Bloesch, et al. Gemini robotics 1.5: Pushing the frontier of generalist robots with advanced embodied reasoning, thinking, and motion transfer. *arXiv preprint arXiv:2510.03342*, 2025.

[2] M. Acharya, K. Kafle, and C. Kanan. TallyQA: Answering complex counting questions. In *AAAI*, 2019.

[3] G. S. Ahmad, A. Heakl, H. Gani, A. Shaker, Z. Shen, F. S. Khan, and S. Khan. Videomolmo: Spatio-temporal grounding meets pointing. *arXiv preprint arXiv:2506.05336*, 2025.

[4] M. AI. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[5] Anthropic. Claude sonnet 4.5 system card, 2025. URL https://assets.anthropic.com/m/12f214efcc2f457a/original/Claude-Sonnet-4-5-System-Card.pdf.

[6] A. Athar, J. Luiten, P. Voigtlaender, T. Khurana, A. Dave, B. Leibe, and D. Ramanan. Burst: A benchmark for unifying object recognition, segmentation and tracking in video. In *WACV*, 2023.

[7] A. Athar, X. Deng, and L.-C. Chen. Vicas: A dataset for combining holistic and pixel-level video understanding using captions with grounded segmentation. In *CVPR*, 2025.

[8] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[9] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. In *NeurIPS Deep Learning Symposium*, 2016.

[10] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, W. Ding, C. Gao, C. Ge, W. Ge, Z. Guo, Q. Huang, J. Huang, F. Huang, B. Hui, S. Jiang, Z. Li, M. Li, M. Li, K. Li, Z. Lin, J. Lin, X. Liu, J. Liu, C. Liu, Y. Liu, D. Liu, S. Liu, D. Lu, R. Luo, C. Lv, R. Men, L. Meng, X. Ren, X. Ren, S. Song, Y. Sun, J. Tang, J. Tu, J. Wan, P. Wang, P. Wang, Q. Wang, Y. Wang, T. Xie, Y. Xu, H. Xu, J. Xu, Z. Yang, M. Yang, J. Yang, A. Yang, B. Yu, F. Zhang, H. Zhang, X. Zhang, B. Zheng, H. Zhong, J. Zhou, F. Zhou, J. Zhou, Y. Zhu, and K. Zhu. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.

[11] Z. Bai, T. He, H. Mei, P. Wang, Z. Gao, J. Chen, L. Liu, Z. Zhang, and M. Z. Shou. One token to seg them all: Language instructed reasoning segmentation in videos. In *NeurIPS*, 2024.

[12] M. Bellver, C. Ventura, C. Silberer, I. Kazakos, J. Torres, and X. Giro-i Nieto. Refvos: a closer look at referring expressions for video object segmentation. *arXiv preprint arXiv:2010.00263*, 2020.

[13] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov, X. Wang, D. Salz, M. Neumann, I. Alabdulmohsin, M. Tschannen, E. Bugliarello, T. Unterthiner, D. Keysers, S. Koppula, F. Liu, A. Grycner, A. Gritsenko, N. Houlsby, M. Kumar, K. Rong, J. Eisenschlos, R. Kabra, M. Bauer, M. Bošnjak, X. Chen, M. Minderer, P. Voigtlaender, I. Bica, I. Balazevic, J. Puigcerver, P. Papalampidi, O. Henaff, X. Xiong, R. Soricut, J. Harmsen, and X. Zhai. PaliGemma: A versatile 3B VLM for transfer. *arXiv preprint arXiv:2407.07726*, 2024.

[14] A. F. Biten, R. Tito, A. Mafla, L. Gomez, M. Rusinol, E. Valveny, C. Jawahar, and D. Karatzas. Scene text visual question answering. In *ICCV*, 2019.

[15] F. Caba Heilbron, V. Escorcia, B. Ghanem, and J. Carlos Niebles. Activitynet: A large-scale video benchmark for human activity understanding. In *CVPR*, 2015.

[16] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris, C. Ryali, K. V. Alwala, H. Khedr, A. Huang, et al. Sam 3: Segment anything with concepts. *arXiv preprint arXiv:2511.16719*, 2025.

[17] G. Chen, Z. Li, S. Wang, J. Jiang, Y. Liu, L. Lu, D.-A. Huang, W. Byeon, M. Le, M. Ehrlich, T. Lu, L. Wang, B. Catanzaro, J. Kautz, A. Tao, Z. Yu, and G. Liu. Eagle 2.5: Boosting long-context post-training for frontier vision-language models. In *NeurIPS*, 2025.

[18] L. Chen, J. Li, X. Dong, P. Zhang, C. He, J. Wang, F. Zhao, and D. Lin. ShareGPT4V: Improving large multi-modal models with better captions. *arXiv preprint arXiv:2311.12793*, 2023.

[19] L. Chen, X. Wei, J. Li, X. Dong, P. Zhang, Y. Zang, Z. Chen, H. Duan, B. Lin, Z. Tang, L. Yuan, Y. Qiao, D. Lin, F. Zhao, and J. Wang. Sharegpt4video: Improving video understanding and generation with better captions. In *NeurIPS Track on Datasets and Benchmarks*, 2024.

[20] L. Cheng, J. Duan, Y. R. Wang, H. Fang, B. Li, Y. Huang, E. Wang, A. Eftekhar, J. Lee, W. Yuan, et al. Pointarena: Probing multimodal grounding through language-guided pointing. *arXiv preprint arXiv:2505.09990*, 2025.

[21] W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, H. Zhang, B. Zhu, M. Jordan, J. E. Gonzalez, and I. Stoica. Chatbot arena: An open platform for evaluating LLMs by human preference. In *ICML*, 2024.

[22] J. H. Cho, A. Madotto, E. Mavroudi, T. Afouras, T. Nagarajan, M. Maaz, Y. Song, T. Ma, S. Hu, H. Rasheed, P. Sun, P.-Y. Huang, D. Bolya, S. Jain, M. Martin, H. Wang, N. Ravi, S. Jain, T. Stark, S. Moon, B. Damavandi, V. Lee, A. Westbury, S. Khan, P. Krähenbühl, P. Dollár, L. Torresani, K. Grauman, and C. Feichtenhofer. Perceptionlm: Open-access data and models for detailed visual understanding. *arXiv preprint arXiv:2504.13180*, 2025.

[23] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

[24] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[25] G. Comanici, E. Bieber, M. Schaekermann, I. Pasupat, N. Sachdeva, I. Dhillon, M. Blistein, O. Ram, D. Zhang, E. Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.

[26] D. Damen, H. Doughty, G. M. Farinella, S. Fidler, A. Furnari, E. Kazakos, D. Moltisanti, J. Munro, T. Perrett, W. Price, and M. Wray. Scaling egocentric vision: The epic-kitchens dataset. In *ECCV*, 2018.

[27] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *ICLR*, 2024.

[28] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022.

[29] M. Deitke, C. Clark, S. Lee, R. Tripathi, Y. Yang, J. S. Park, M. Salehi, N. Muennighoff, K. Lo, L. Soldaini, J. Lu, T. Anderson, E. Bransom, K. Ehsani, H. Ngo, Y. Chen, A. Patel, M. Yatskar, C. Callison-Burch, A. Head, R. Hendrix, F. Bastani, E. VanderBilt, N. Lambert, Y. Chou, A. Chheda, J. Sparks, S. Skjonsberg, M. Schmitz, A. Sarnat, B. Bischoff, P. Walsh, C. Newell, P. Wolters, T. Gupta, K.-H. Zeng, J. Borchardt, D. Groeneveld, C. Nam, S. Lebrecht, C. Wittlif, C. Schoenick, O. Michel, R. Krishna, L. Weihs, N. A. Smith, H. Hajishirzi, R. Girshick, A. Farhadi, and A. Kembhavi. Molmo and pixmo: Open weights and open data for state-of-the-art vision-language models. In *CVPR*, 2025.

[30] P. Dendorfer, H. Rezatofighi, A. Milan, J. Shi, D. Cremers, I. Reid, S. Roth, K. Schindler, and L. Leal-Taixé. Mot20: A benchmark for multi object tracking in crowded scenes. *arXiv preprint arXiv:2003.09003*, 2020.

[31] H. Ding, C. Liu, S. He, X. Jiang, and C. C. Loy. Mevis: A large-scale benchmark for video segmentation with motion expressions. In *ICCV*, 2023.

[32] H. Ding, C. Liu, S. He, X. Jiang, P. H. Torr, and S. Bai. MOSE: A new dataset for video object segmentation in complex scenes. In *ICCV*, 2023.

[33] H. Ding, K. Ying, C. Liu, S. He, X. Jiang, Y.-G. Jiang, P. H. Torr, and S. Bai. Mosev2: A more challenging dataset for video object segmentation in complex scenes. *arXiv preprint arXiv:2508.05630*, 2025.

[34] T.-T.-T. Do, Q.-T. Huynh, K. Kim, and V.-Q. Nguyen. A survey on video big data analytics: architecture, technologies, and open research challenges. *Applied Sciences*, 2025.

[35] C. Doersch, A. Gupta, L. Markeeva, A. Recasens, L. Smaira, Y. Aytar, J. a. Carreira, A. Zisserman, and Y. Yang. Tap-vid: a benchmark for tracking any point in a video. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, 2022.

[36] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.

[37] D. Du, Y. Qi, H. Yu, Y. Yang, K. Duan, G. Li, W. Zhang, Q. Huang, and Q. Tian. The unmanned aerial vehicle benchmark: Object detection and tracking. In *ECCV*, 2018.

[38] D. Dwibedi, Y. Aytar, J. Tompson, P. Sermanet, and A. Zisserman. Counting out time: Class agnostic video repetition counting in the wild. In *CVPR*, 2020.

[39] H. Fan, L. Lin, F. Yang, P. Chu, G. Deng, S. Yu, H. Bai, Y. Xu, C. Liao, and H. Ling. Lasot: A high-quality benchmark for large-scale single object tracking. In *CVPR*, 2019.

[40] C. Fu, Y. Dai, Y. Luo, L. Li, S. Ren, R. Zhang, Z. Wang, C. Zhou, Y. Shen, M. Zhang, et al. Video-mme: The first-ever comprehensive evaluation benchmark of multi-modal llms in video analysis. In *CVPR*, 2025.

[41] X. Fu, Y. Hu, B. Li, Y. Feng, H. Wang, X. Lin, D. Roth, N. A. Smith, W.-C. Ma, and R. Krishna. Blink: Multimodal large language models can see but not perceive. In *ECCV*, 2024.

[42] J. Gao, C. Sun, Z. Yang, and R. Nevatia. Tall: Temporal activity localization via language query. In *ICCV*, 2017.

[43] M. Gao, J. Liu, M. Li, J. Xie, Q. Liu, B. Zhao, X. Chen, and H. Xiong. Tc-llava: Rethinking the transfer from image to video understanding with temporal considerations. In *AAAI*, 2025.

[44] S. Giancola, M. Amine, T. Dghaily, and B. Ghanem. Soccernet: A scalable dataset for action spotting in soccer videos. In *CVPR Workshop on Computer Vision in Sports*, 2018.

[45] Google. Gemini 3 Pro model card, 2025. URL https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Pro-Model-Card.pdf.

[46] R. Goyal, S. Ebrahimi Kahou, V. Michalski, J. Materzynska, S. Westphal, H. Kim, V. Haenel, I. Fruend, P. Yianilos, M. Mueller-Freitag, et al. The" something something" video database for learning and evaluating visual common sense. In *ICCV*, 2017.

[47] Y. Goyal, T. Khot, D. Summers-Stay, D. Batra, and D. Parikh. Making the V in VQA matter: Elevating the role of image understanding in visual question answering. In *CVPR*, 2017.

[48] Y. Goyal, T. Khot, D. Summers-Stay, D. Batra, and D. Parikh. Making the V in VQA matter: Elevating the role of image understanding in visual question answering. In *CVPR*, 2017.

[49] K. Grauman, A. Westbury, E. Byrne, Z. Chavis, A. Furnari, R. Girdhar, J. Hamburger, H. Jiang, M. Liu, X. Liu, et al. Ego4d: Around the world in 3,000 hours of egocentric video. In *CVPR*, 2022.

[50] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. In *ICLR*, 2021.

[51] J. R. Hermans, G. Spanakis, and R. Möckel. Accumulated gradient normalization. In *ACML*, 2017.

[52] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*, 2019.

[53] L. Hong, W. Chen, Z. Liu, W. Zhang, P. Guo, Z. Chen, and W. Zhang. Lvos: A benchmark for long-term video object segmentation. In *ICCV*, 2023.

[54] W. Hong, Y. Cheng, Z. Yang, W. Wang, L. Wang, X. Gu, S. Huang, Y. Dong, and J. Tang. Motionbench: Benchmarking and improving fine-grained video motion understanding for vision language models. In *CVPR*, 2025.

[55] L. Huang, X. Zhao, and K. Huang. Got-10k: A large high-diversity benchmark for generic object tracking in the wild. *TPAMI*, 2019.

[56] S. A. Jacobs, M. Tanaka, C. Zhang, M. Zhang, R. Y. Aminadabi, S. L. Song, S. Rajbhandari, and Y. He. System optimizations for enabling training of extreme long sequence transformer models. In *PODC*, 2024.

[57] S. Jahagirdar, M. Mathew, D. Karatzas, and C. Jawahar. Watching the news: Towards videoqa models that can read. In *CVPR*, 2023.

[58] H. Jhamtani and T. Berg-Kirkpatrick. Learning to describe differences between pairs of similar images. In *EMNLP*, 2018.

[59] D. Jiang, X. He, H. Zeng, C. Wei, M. Ku, Q. Liu, and W. Chen. MANTIS: Interleaved multi-image instruction tuning. *TMLR*, 2024.

[60] Q. Jiang, J. Huo, X. Chen, Y. Xiong, Z. Zeng, Y. Chen, T. Ren, J. Yu, and L. Zhang. Detect anything via next point prediction. *arXiv preprint arXiv:2510.12798*, 2025.

[61] K. Kafle, B. Price, S. Cohen, and C. Kanan. DVQA: Understanding data visualizations via question answering. In *CVPR*, 2018.

[62] S. E. Kahou, V. Michalski, A. Atkinson, Á. Kádár, A. Trischler, and Y. Bengio. FigureQA: An annotated figure dataset for visual reasoning. *arXiv preprint arXiv:1710.07300*, 2017.

[63] N. Karaev, I. Makarov, J. Wang, N. Neverova, A. Vedaldi, and C. Rupprecht. CoTracker3: Simpler and better point tracking by pseudo-labelling real videos. In *arxiv*, 2024.

[64] W. Kay, J. Carreira, K. Simonyan, B. Zhang, C. Hillier, S. Vijayanarasimhan, F. Viola, T. Green, T. Back, P. Natsev, et al. The kinetics human action video dataset. *arXiv preprint arXiv:1705.06950*, 2017.

[65] A. Kembhavi, M. Salvato, E. Kolve, M. Seo, H. Hajishirzi, and A. Farhadi. A diagram is worth a dozen images. In *ECCV*, 2016.

[66] A. Khoreva, A. Rohrbach, and B. Schiele. Video object segmentation with language referring expressions. In *ACCV*, 2018.

[67] D. P. Kingma. Adam: A method for stochastic optimization. In *ICLR*, 2015.

[68] R. Krishna, Y. Zhu, O. Groth, J. Johnson, K. Hata, J. Kravitz, S. Chen, Y. Kalantidis, L.-J. Li, D. A. Shamma, M. S. Bernstein, and L. Fei-Fei. Visual genome: Connecting language and vision using crowdsourced dense image annotations. *International Journal of Computer Vision*, 123:32 – 73, 2016.

[69] R. Krishna, K. Hata, F. Ren, L. Fei-Fei, and J. Carlos Niebles. Dense-captioning events in videos. In *ICCV*, 2017.

[70] X. Lai, Z. Tian, Y. Chen, Y. Li, Y. Yuan, S. Liu, and J. Jia. Lisa: Reasoning segmentation via large language model. *arXiv preprint arXiv:2308.00692*, 2023.

[71] N. Lambert, J. Morrison, V. Pyatkin, S. Huang, H. Ivison, F. Brahman, L. J. V. Miranda, A. Liu, N. Dziri, S. Lyu, Y. Gu, S. Malik, V. Graf, J. D. Hwang, J. Yang, R. L. Bras, O. Tafjord, C. Wilhelm, L. Soldaini, N. A. Smith, Y. Wang, P. Dasigi, and H. Hajishirzi. Tülu 3: Pushing frontiers in open language model post-training. In *COLM*, 2025.

[72] H. Lamdouar, C. Yang, W. Xie, and A. Zisserman. Betrayed by motion: Camouflaged object discovery via motion segmentation. In *ACCV*, 2020.

[73] J. Lee, J. Duan, H. Fang, Y. Deng, S. Liu, B. Li, B. Fang, J. Zhang, Y. R. Wang, S. Lee, W. Han, W. Pumacay, A. Wu, R. Hendrix, K. Farley, E. VanderBilt, A. Farhadi, D. Fox, and R. Krishna. Molmoact: Action reasoning models that can reason in space. *arXiv preprint arXiv:2508.07917*, 2025.

[74] J. Lei, L. Yu, M. Bansal, and T. L. Berg. Tvqa: Localized, compositional video question answering. In *EMNLP*, 2018.

[75] J. Lei, T. L. Berg, and M. Bansal. Detecting moments and highlights in videos via natural language queries. In *NeurIPS*, 2021.

[76] A. Li, R. Thapa, R. Chalamala, Q. Wu, K. Chen, and J. Zou. SMIR: Efficient synthetic data pipeline to improve multi-image reasoning. *arXiv preprint arXiv:2501.03675*, 2025.

[77] J. Li, P. Wei, W. Han, and L. Fan. Intentqa: Context-aware video intent reasoning. In *CVPR*, 2023.

[78] K. Li, Y. Wang, Y. He, Y. Li, Y. Wang, Y. Liu, Z. Wang, J. Xu, G. Chen, P. Luo, et al. Mvbench: A comprehensive multi-modal video understanding benchmark. In *CVPR*, 2024.

[79] X. Li, Y. Wang, J. Yu, X. Zeng, Y. Zhu, H. Huang, J. Gao, K. Li, Y. He, C. Wang, Y. Qiao, Y. Wang, and L. Wang. Videochat-flash: Hierarchical compression for long-context video modeling. *arXiv preprint arXiv:2501.00574*, 2024.

[80] Y. Li, Y. Song, L. Cao, J. Tetreault, L. Goldberg, A. Jaimes, and J. Luo. Tgif: A new dataset and benchmark on animated gif description. In *CVPR*, 2016.

[81] Y. Li, J. Zhang, X. Teng, H. Zhang, X. Liu, and L. Lan. Refsam: Efficiently adapting segmenting anything model for referring video object segmentation. *Neural Networks*, 2025.

[82] Z. Li, M. Ganti, Z. Ma, H. Vasconcelos, Q. He, and R. Krishna. Rethinking (human) preference evaluation of llm rationales. In *COLM Workshop on the Application of LLM Explainability to Reasoning and Planning*, 2025.

[83] L. Liang, H. Ma, L. Zhao, X. Xie, C. Hua, M. Zhang, and Y. Zhang. Vehicle detection algorithms for autonomous driving: A review. *Sensors*, 2024.

[84] J. T. Licardo, M. Domjan, and T. Orehovački. Intelligent robotics—a systematic review of emerging technologies and trends. *Electronics*, 2024.

[85] J. Lin, W. Peng, B. Zi, Y. Gao, X. Qi, X. Ma, and Y.-G. Jiang. Brokenvideos: A benchmark dataset for fine-grained artifact localization in ai-generated videos. In *MM*, 2025.

[86] Z. Lin, S. Cen, D. Jiang, J. Karhade, H. Wang, C. Mitra, T. Ling, Y. Huang, S. Liu, M. Chen, et al. Towards understanding camera motions in any video. *arXiv preprint arXiv:2504.15376*, 2025.

[87] H. LinLin, L. Sangheang, and S. GuanTing. Cam-vtrans: real-time sports training utilizing multi-modal robot data. *Frontiers in Neurorobotics*, 2024.

[88] C. Liu, H. Ding, and X. Jiang. GRES: Generalized referring expression segmentation. In *CVPR*, 2023.

[89] H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. In *NeurIPS*, 2023.

[90] H. Liu, C. Li, Y. Li, and Y. J. Lee. Improved baselines with visual instruction tuning. In *CVPR*, 2024.

[91] Y. Liu, S. Li, Y. Liu, Y. Wang, S. Ren, L. Li, S. Chen, X. Sun, and L. Hou. Tempcompass: Do video llms really understand videos? In *ACL*, 2024.

[92] Y. Liu, T. Qu, Z. Zhong, B. Peng, S. Liu, B. Yu, and J. Jia. Visionreasoner: Unified visual perception and reasoning via reinforcement learning. *arXiv preprint arXiv:2505.12081*, 2025.

[93] Z. Liu, T. Chu, Y. Zang, X. Wei, X. Dong, P. Zhang, Z. Liang, Y. Xiong, Y. Qiao, D. Lin, and J. Wang. MMDU: A multi-turn multi-image dialog understanding benchmark and instruction-tuning dataset for LVLMs. In *NeurIPS Track on Datasets and Benchmarks*, 2024.

[94] P. Lu, S. Mishra, T. Xia, L. Qiu, K.-W. Chang, S.-C. Zhu, O. Tafjord, P. Clark, and A. Kalyan. Learn to explain: Multimodal reasoning via thought chains for science question answering. In *NeurIPS*, 2022.

[95] P. Lu, L. Qiu, K.-W. Chang, Y. N. Wu, S.-C. Zhu, T. Rajpurohit, P. Clark, and A. Kalyan. Dynamic prompt learning via policy gradient for semi-structured mathematical reasoning. In *ICLR*, 2023.

[96] P. Lu, H. Bansal, T. Xia, J. Liu, C. Li, H. Hajishirzi, H. Cheng, K.-W. Chang, M. Galley, and J. Gao. MathVista: Evaluating mathematical reasoning of foundation models in visual contexts. In *ICLR*, 2024.

[97] J. Luiten, A. Osep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe. Hota: A higher order metric for evaluating multi-object tracking. *IJCV*, 2021.

[98] W. Ma, W. Ren, Y. Jia, Z. Li, P. Nie, G. Zhang, and W. Chen. Videoeval-pro: Robust and realistic long video understanding evaluation. *arXiv preprint arXiv:2505.14640*, 2025.

[99] M. Maaz, H. Rasheed, S. Khan, and F. S. Khan. Video-ChatGPT: Towards detailed video understanding via large vision and language models. In *ACL*, 2024.

[100] K. Mangalam, R. Akshulakov, and J. Malik. Egoschema: A diagnostic benchmark for very long-form video language understanding. In *NeurIPS Track on Datasets and Benchmarks*, 2023.

[101] K. Marino, M. Rastegari, A. Farhadi, and R. Mottaghi. OK-VQA: A visual question answering benchmark requiring external knowledge. In *CVPR*, 2019.

[102] A. Masry, D. Long, J. Q. Tan, S. Joty, and E. Hoque. ChartQA: A benchmark for question answering about charts with visual and logical reasoning. In *ACL*, 2022.

[103] M. Mathew, D. Karatzas, and C. Jawahar. DocVQA: A dataset for VQA on document images. In *WACV*, 2021.

[104] M. Mathew, D. Karatzas, and C. Jawahar. DocVQA: A dataset for VQA on document images. In *WACV*, 2021.

[105] M. Mathew, V. Bagal, R. Tito, D. Karatzas, E. Valveny, and C. Jawahar. InfographicVQA. In *WACV*, 2022.

[106] F. Meng, J. Wang, C. Li, Q. Lu, H. Tian, J. Liao, X. Zhu, J. Dai, Y. Qiao, P. Luo, K. Zhang, and W. Shao. Mmiu: Multimodal multi-image understanding for evaluating large vision-language models. In *ICLR*, 2025.

[107] N. Methani, P. Ganguly, M. M. Khapra, and P. Kumar. PlotQA: Reasoning over scientific plots. In *WACV*, 2020.

[108] J. Miao, X. Wang, Y. Wu, W. Li, X. Zhang, Y. Wei, and Y. Yang. Large-scale video panoptic segmentation in the wild: A benchmark. In *CVPR*, 2022.

[109] M. Monfort, A. Andonian, B. Zhou, K. Ramakrishnan, S. A. Bargal, T. Yan, L. Brown, Q. Fan, D. Gutfreund, C. Vondrick, et al. Moments in time dataset: one million videos for event understanding. *TPAMI*, 2019.

[110] M. Muller, A. Bibi, S. Giancola, S. Alsubaihi, and B. Ghanem. Trackingnet: A large-scale dataset and benchmark for object tracking in the wild. In *ECCV*, 2018.

[111] S. Munasinghe, H. Gani, W. Zhu, J. Cao, E. Xing, F. S. Khan, and S. Khan. Videoglamm: A large multimodal model for pixel-level visual grounding in videos. In *CVPR*, 2025.

[112] Olmo Team. Olmo 3. Technical report, Allen Institute for AI, 2025. URL https://www.datocms-assets.com/64837/1763662397-1763646865-olmo_3_technical_report-1.pdf.

[113] OpenAI. GPT-4o mini system card, 2024. URL https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/.

[114] OpenAI. GPT-5 system card, 2025. URL https://openai.com/index/gpt-5-system-card/.

[115] V. Patraucean, L. Smaira, A. Gupta, A. Recasens, L. Markeeva, D. Banarse, S. Koppula, M. Malinowski, Y. Yang, C. Doersch, et al. Perception test: A diagnostic benchmark for multimodal video models. *NeurIPS*, 2023.

[116] L. Peng, J. Gao, X. Liu, W. Li, S. Dong, Z. Zhang, H. Fan, and L. Zhang. Vasttrack: Vast category visual object tracking. In *NeurIPS*, 2024.

[117] J. Qi, Y. Gao, Y. Hu, X. Wang, X. Liu, X. Bai, S. Belongie, A. Yuille, P. Torr, and S. Bai. Occluded video instance segmentation: A benchmark. *IJCV*, 2022.

[118] R. Qian, X. Dong, P. Zhang, Y. Zang, S. Ding, D. Lin, and J. Wang. Streaming long video understanding with large language models. In *NeurIPS*, 2024.

[119] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *ICML*, 2021.

[120] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever. Robust speech recognition via large-scale weak supervision. In *ICML*, 2023.

[121] H. Rasheed, M. Maaz, S. Shaji, A. Shaker, S. Khan, H. Cholakkal, R. M. Anwer, E. Xing, M.-H. Yang, and F. S. Khan. GLaMM: Pixel grounding large multimodal model. In *CVPR*, 2024.

[122] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, and C. Feichtenhofer. Sam 2: Segment anything in images and videos. In *ICLR*, 2025.

[123] R. Rawal, K. Saifullah, M. Farré, R. Basri, D. Jacobs, G. Somepalli, and T. Goldstein. Cinepile: A long video question answering dataset and benchmark. *arXiv preprint arXiv:2405.08813*, 2024.

[124] V. Rawte, S. Jain, A. Sinha, G. Kaushik, A. Bansal, P. R. Vishwanath, S. R. Jain, A. N. Reganti, V. Jain, A. Chadha, et al. Vibe: A text-to-video benchmark for evaluating hallucination in large multimodal models. *arXiv preprint arXiv:2411.10867*, 2024.

[125] D. Schwenk, A. Khandelwal, C. Clark, K. Marino, and R. Mottaghi. A-OKVQA: A benchmark for visual question answering using world knowledge. In *ECCV*, 2022.

[126] A. Scott, I. Uchida, N. Ding, R. Umemoto, R. Bunker, R. Kobayashi, T. Koyama, M. Onishi, Y. Kameda, and K. Fujii. Teamtrack: A dataset for multi-sport multi-object tracking in full-pitch videos. In *CVPR Workshop on Computer Vision in Sports*, 2024.

[127] S. Seo, J.-Y. Lee, and B. Han. Urvos: Unified referring video object segmentation network with a large-scale benchmark. In *ECCV*, 2020.

[128] Z. Shangguan, C. Li, Y. Ding, Y. Zheng, Y. Zhao, T. Fitzgerald, and A. Cohan. Tomato: Assessing visual temporal reasoning capabilities in multimodal foundation models. In *ICLR*, 2025.

[129] X. Shen, Y. Xiong, C. Zhao, L. Wu, J. Chen, C. Zhu, Z. Liu, F. Xiao, B. Varadarajan, F. Bordes, Z. Liu, H. Xu, H. J. Kim, B. Soran, R. Krishnamoorthi, M. Elhoseiny, and V. Chandra. Longvu: Spatiotemporal adaptive compression for long video-language understanding. *arXiv preprint arXiv:2410.17434*, 2024.

[130] A. Singh, V. Natarjan, M. Shah, Y. Jiang, X. Chen, D. Parikh, and M. Rohrbach. Towards VQA models that can read. In *CVPR*, 2019.

[131] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 2024.

[132] A. Suhr, S. Zhou, I. Zhang, H. Bai, and Y. Artzi. A corpus for reasoning about natural language grounded in photographs. In *ACL*, 2018.

[133] P. Sun, J. Cao, Y. Jiang, Z. Yuan, S. Bai, K. Kitani, and P. Luo. Dancetrack: Multi-object tracking in uniform appearance and diverse motion. In *CVPR*, 2022.

[134] Y. Tang, D. Ding, Y. Rao, Y. Zheng, D. Zhang, L. Zhao, J. Lu, and J. Zhou. Coin: A large-scale dataset for comprehensive instructional video analysis. In *CVPR*, 2019.

[135] G. Team. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.

[136] G. Team, A. Kamath, J. Ferret, S. Pathak, N. Vieillard, R. Merhej, S. Perrin, T. Matejovicova, A. Ramé, M. Rivière, et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.

[137] V. Team, W. Hong, W. Yu, X. Gu, G. Wang, G. Gan, H. Tang, J. Cheng, J. Qi, J. Ji, L. Pan, S. Duan, W. Wang, Y. Wang, Y. Cheng, Z. He, Z. Su, Z. Yang, Z. Pan, A. Zeng, B. Wang, B. Chen, B. Shi, C. Pang, C. Zhang, D. Yin, F. Yang, G. Chen, J. Xu, J. Zhu, J. Chen, J. Chen, J. Chen, J. Lin, J. Wang, J. Chen, L. Lei, L. Gong, L. Pan, M. Liu, M. Xu, M. Zhang, Q. Zheng, S. Yang, S. Zhong, S. Huang, S. Zhao, S. Xue, S. Tu, S. Meng, T. Zhang, T. Luo, T. Hao, T. Tong, W. Li, W. Jia, X. Liu, X. Zhang, X. Lyu, X. Fan, X. Huang, Y. Wang, Y. Xue, Y. Wang, Y. Wang, Y. An, Y. Du, Y. Shi, Y. Huang, Y. Niu, Y. Wang, Y. Yue, Y. Li, Y. Zhang, Y. Wang, Y. Wang, Y. Zhang, Z. Xue, Z. Hou, Z. Du, Z. Wang, P. Zhang, D. Liu, B. Xu, J. Li, M. Huang, Y. Dong, and J. Tang. Glm-4.5v and glm-4.1v-thinking: Towards versatile multimodal reasoning with scalable reinforcement learning. *arXiv preprint arXiv:2507.01006*, 2025.

[138] G. Tom, M. Mathew, S. Garcia-Bordils, D. Karatzas, and C. Jawahar. Reading between the lanes: Text videoqa on the road. In *ICDAR*, 2023.

[139] M. Tschannen, A. Gritsenko, X. Wang, M. F. Naeem, I. Alabdulmohsin, N. Parthasarathy, T. Evans, L. Beyer, Y. Xia, B. Mustafa, O. Hénaff, J. Harmsen, A. Steiner, and X. Zhai. Siglip 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *arXiv preprint arXiv:2502.14786*, 2025.

[140] L. A. Varga, B. Kiefer, M. Messmer, and A. Zell. Seadronessee: A maritime benchmark for detecting humans in open water. In *WACV*, 2022.

[141] P. Voigtlaender, S. Changpinyo, J. Pont-Tuset, R. Soricut, and V. Ferrari. Connecting vision and language with video localized narratives. In *CVPR*, 2023.

[142] F. Wang, X. Fu, J. Y. Huang, Z. Li, Q. Liu, X. Liu, M. D. Ma, N. Xu, W. Zhou, K. Zhang, et al. Muirbench: A comprehensive benchmark for robust multi-image understanding. In *ICLR*, 2025.

[143] H. Wang, C. Yan, S. Wang, X. Jiang, X. Tang, Y. Hu, W. Xie, and E. Gavves. Towards open-vocabulary video instance segmentation. In *ICCV*, 2023.

[144] J. Wang, Y. Peng, X. Yang, T. Wang, and Y. Zhang. Sportstrack: An innovative method for tracking athletes in sports scenes. *arXiv preprint arXiv:2211.07173*, 2022.

[145] P. Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai, K. Chen, X. Liu, J. Wang, W. Ge, et al. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024.

[146] P. Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai, K. Chen, X. Liu, J. Wang, W. Ge, et al. Qwen2-VL: A stronger and more general multimodal LLM. *arXiv preprint arXiv:2409.12191*, 2024.

[147] Q. Wang, Y. Shi, J. Ou, R. Chen, K. Lin, J. Wang, B. Jiang, H. Yang, M. Zheng, X. Tao, F. Yang, P. Wan, and D. Zhang. Koala-36m: A large-scale video dataset improving consistency between fine-grained conditions and video content. In *CVPR*, 2025.

[148] W. Wang and Y. Yang. Vidprom: A million-scale real prompt-gallery dataset for text-to-video diffusion models. In *NeurIPS Track on Datasets and Benchmarks*, 2024.

[149] W. Wang, Z. Gao, L. Gu, H. Pu, L. Cui, X. Wei, Z. Liu, L. Jing, S. Ye, J. Shao, et al. Internvl3.5: Advancing open-source multimodal models in versatility, reasoning, and efficiency. *arXiv preprint arXiv:2508.18265*, 2025.

[150] W. Wang, Z. He, W. Hong, Y. Cheng, X. Zhang, J. Qi, M. Ding, X. Gu, S. Huang, B. Xu, et al. Lvbench: An extreme long video understanding benchmark. In *ICCV*, 2025.

[151] X. Wang, X. Shu, Z. Zhang, B. Jiang, Y. Wang, Y. Tian, and F. Wu. Towards more flexible and accurate object tracking with natural language: Algorithms and benchmark. In *CVPR*, 2021.

[152] X. Wang, L. Jin, X. Lou, S. Wang, L. Chen, B. Jiang, and Z. Zhang. Reasoningtrack: Chain-of-thought reasoning for long-term vision-language tracking. *arXiv preprint arXiv:2508.05221*, 2025.

[153] Y. Wang, Y. He, Y. Li, K. Li, J. Yu, X. Ma, X. Li, G. Chen, X. Chen, Y. Wang, et al. Internvid: A large-scale video-text dataset for multimodal understanding and generation. In *ICLR*, 2023.

[154] Z. Wang, A. Blume, S. Li, G. Liu, J. Cho, Z. Tang, M. Bansal, and H. Ji. Paxion: Patching action knowledge in video-language foundation models. In *NeurIPS*, 2023.

[155] A. Wilf, L. Mathur, S. Mathew, C. Ko, Y. Kebe, P. P. Liang, and L.-P. Morency. Social-iq 2.0 challenge: Benchmarking multimodal social understanding. https://github.com/abwilf/Social-IQ-2.0-Challenge, 2023.

[156] B. Wu, S. Yu, Z. Chen, J. B. Tenenbaum, and C. Gan. A benchmark for situated reasoning in real-world videos. In *NeurIPS*, 2024.

[157] H. Wu, D. Li, B. Chen, and J. Li. Longvideobench: A benchmark for long-context interleaved video-language understanding. In *NeurIPS*, 2024.

[158] xAI. RealWorldQA. https://huggingface.co/datasets/xai-org/RealworldQA, 2024. Accessed: 2024-09-24.

[159] H. Xia, Z. Yang, Y. Wang, R. Tracy, Y. Zhao, D. Huang, Z. Chen, Y. Zhu, Y.-f. Wang, and W. Shen. Sportqa: A benchmark for sports understanding in large language models. In *NAACL*, 2024.

[160] J. Xiao, X. Shang, A. Yao, and T.-S. Chua. Next-qa: Next phase of question-answering to explaining temporal actions. In *CVPR*, 2021.

[161] B. Xie, S. Zhang, Z. Zhou, B. Li, Y. Zhang, J. Hessel, J. Yang, and Z. Liu. Funqa: Towards surprising video comprehension. In *ECCV*, 2024.

[162] H. Xu, S. Xie, X. Tan, P.-Y. Huang, R. Howes, V. Sharma, S.-W. Li, G. Ghosh, L. Zettlemoyer, and C. Feichtenhofer. Demystifying CLIP data. In *ICLR*, 2024.

[163] L. Xu, H. Huang, and J. Liu. Sutd-trafficqa: A question answering benchmark and an efficient network for video reasoning over traffic events. In *CVPR*, 2021.

[164] M. Xu, M. Gao, Z. Gan, H.-Y. Chen, Z. Lai, H. Gang, K. Kang, and A. Dehghan. Slowfast-llava: A strong training-free baseline for video large language models. *arXiv preprint arXiv:2407.15841*, 2024.

[165] M. Xu, M. Gao, S. Li, J. Lu, Z. Gan, Z. Lai, M. Cao, K. Kang, Y. Yang, and A. Dehghan. Slowfast-llava-1.5: A family of token-efficient video large language models for long-form video understanding. In *COLM*, 2025.

[166] C. Yan, H. Wang, S. Yan, X. Jiang, Y. Hu, G. Kang, W. Xie, and E. Gavves. Visa: Reasoning video object segmentation via large language models. In *ECCV*, 2024.

[167] A. Yang, A. Miech, J. Sivic, I. Laptev, and C. Schmid. Just ask: Learning to answer questions from millions of narrated videos. In *CVPR*, 2021.

[168] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou, C. Li, C. Li, D. Liu, F. Huang, G. Dong, H. Wei, H. Lin, J. Tang, J. Wang, J. Yang, J. Tu, J. Zhang, J. Ma, J. Xu, J. Zhou, J. Bai, J. He, J. Lin, K. Dang, K. Lu, K. Chen, K. Yang, M. Li, M. Xue, N. Ni, P. Zhang, P. Wang, R. Peng, R. Men, R. Gao, R. Lin, S. Wang, S. Bai, S. Tan, T. Zhu, T. Li, T. Liu, W. Ge, X. Deng, X. Zhou, X. Ren, X. Zhang, X. Wei, X. Ren, Y. Fan, Y. Yao, Y. Zhang, Y. Wan, Y. Chu, Y. Liu, Z. Cui, Z. Zhang, and Z. Fan. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.

[169] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[170] B. Yang, B. Wen, B. Ding, C. Liu, C. Chu, C. Song, C. Rao, C. Yi, D. Li, D. Zang, et al. Kwai keye-vl 1.5 technical report. *arXiv preprint arXiv:2509.01563*, 2025.

[171] W. Yang and Z. Huang. Poivre: Self-refining visual pointing with reinforcement learning. *arXiv preprint arXiv:2509.23746*, 2025.

[172] Y. Yang, A. Patel, M. Deitke, T. Gupta, L. Weihs, A. Head, M. Yatskar, C. Callison-Burch, R. Krishna, A. Kembhavi, et al. Scaling text-rich image understanding via code-guided synthetic multimodal data generation. In *ACL*, 2025.

[173] K. Yi, C. Gan, Y. Li, P. Kohli, J. Wu, A. Torralba, and J. B. Tenenbaum. Clevrer: Collision events for video representation and reasoning. *arXiv preprint arXiv:1910.01442*, 2019.

[174] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell. Bdd100k: A diverse driving dataset for heterogeneous multitask learning. In *CVPR*, 2020.

[175] L. Yu, P. Poirson, S. Yang, A. C. Berg, and T. L. Berg. Modeling context in referring expressions. In *ECCV*, 2016.

[176] T. Yu, Z. Wang, C. Wang, F. Huang, W. Ma, Z. He, T. Cai, W. Chen, Y. Huang, Y. Zhao, B. Xu, J. Cui, Y. Xu, L. Ruan, L. Zhang, H. Liu, J. Tang, H. Liu, Q. Guo, W. Hu, B. He, J. Zhou, J. Cai, J. Qi, Z. Guo, C. Chen, G. Zeng, Y. Li, G. Cui, N. Ding, X. Han, Y. Yao, Z. Liu, and M. Sun. Minicpm-v 4.5: Cooking efficient mllms via architecture, data, and training recipe. *arXiv preprint arXiv:2509.18154*, 2025.

[177] H. Yuan, X. Li, T. Zhang, Z. Huang, S. Xu, S. Ji, Y. Tong, L. Qi, J. Feng, and M.-H. Yang. Sa2va: Marrying sam2 with llava for dense grounded understanding of images and videos. *arXiv preprint arXiv:2501.04001*, 2025.

[178] W. Yuan, J. Duan, V. Blukis, W. Pumacay, R. Krishna, A. Murali, A. Mousavian, and D. Fox. Robopoint: A vision-language model for spatial affordance prediction for robotics. In *CoRL*, 2024.

[179] X. Yue, Y. Ni, K. Zhang, T. Zheng, R. Liu, G. Zhang, S. Stevens, D. Jiang, W. Ren, Y. Sun, C. Wei, B. Yu, R. Yuan, R. Sun, M. Yin, B. Zheng, Z. Yang, Y. Liu, W. Huang, H. Sun, Y. Su, and W. Chen. MMMU: A massive multi-discipline multimodal understanding and reasoning benchmark for expert AGI. In *CVPR*, 2024.

[180] R. Zellers, J. Lu, X. Lu, Y. Yu, Y. Zhao, M. Salehi, A. Kusupati, J. Hessel, A. Farhadi, and Y. Choi. Merlot reserve: Multimodal neural script knowledge through vision and language and sound. In *CVPR*, 2022.

[181] C. Zhang, G. Huang, L. Liu, S. Huang, Y. Yang, X. Wan, S. Ge, and D. Tao. Webuav-3m: A benchmark for unveiling the power of million-scale deep uav tracking. *TPAMI*, 2023.

[182] C. Zhang, L. Liu, G. Huang, H. Wen, X. Zhou, and Y. Wang. Webuot-1m: Advancing deep underwater object tracking with a million-scale benchmark. In *NeurIPS*, 2024.

[183] L. Zhang, J. Gao, Z. Xiao, and H. Fan. Animaltrack: A benchmark for multi-animal tracking in the wild. *IJCV*, 2023.

[184] Y. Zhang, J. Wu, W. Li, B. Li, Z. Ma, Z. Liu, and C. Li. Llava-video: Video instruction tuning with synthetic data. *TMLR*, 2025.

[185] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer, et al. Pytorch fsdp: Experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277*, 2023.

[186] G. Zheng, S. Lin, H. Zuo, C. Fu, and J. Pan. Nettrack: Tracking highly dynamic objects with a net. In *CVPR*, 2024.

[187] J. Zhou, Y. Shu, B. Zhao, B. Wu, Z. Liang, S. Xiao, M. Qin, X. Yang, Y. Xiong, B. Zhang, et al. Mlvu: Benchmarking multi-task long video understanding. In *CVPR*, 2025.

[188] L. Zhou, C. Xu, and J. Corso. Towards automatic learning of procedures from web instructional videos. In *AAAI*, 2018.

[189] Y. Zhu, C. Li, Y. Liu, X. Wang, J. Tang, B. Luo, and Z. Huang. Tiny object tracking: A large-scale dataset and a baseline. *TNNLS*, 2023.

[190] O. Zohar, X. Wang, Y. Dubois, N. Mehta, T. Xiao, P. Hansen-Estruch, L. Yu, X. Wang, F. Juefei-Xu, N. Zhang, S. Yeung-Levy, and X. Xia. Apollo: An exploration of video understanding in large multimodal models. In *CVPR*, 2025.

Appendix

The appendix includes the following sections:

- §A - Model details
- §B - Training details
- §C - Evaluation details
- §D - Additional results
- §E - Test time scaling and SlowFast encoding
- §F - Data details
- §G - Data examples
- §H - Limitations
- §I - Qualitative results

A Model details

We present additional details about image encoding, hyperparameters, and implementation choices.

Image crops. Our method of encoding images largely follows Molmo [29], including the use of overlapping crops. Unlike Molmo, we do not pad crops with black. Instead, we resize them to 378 (even if that means changing the aspect ratio), following how SigLIP 2 [139] was trained. If the number of image patches is not evenly divisible by the pooling size, the bottom and far-right image patches are pooled with a reduced number of patches.

Video frames. We use torchcodec[3] to extract frames from videos. We extract frames at S fps and the last frame. If that leads to more than F frames, we instead extract frames uniformly, including the first and last frames. For tracking, during training, we always sample videos at S fps and trim both videos and point tracks to a maximum of F frames instead. This ensures that points, which are annotated for S fps, remain aligned with the sampled frames. We include the last frame since it is typically what is shown when the video ends and, therefore, can have special importance to users. Frames are extracted based on timestamps (instead of frame indices) to handle variable fps videos.

Formatting. Videos and image tokens are always inserted first, right after the BOS token. We insert different start and end special tokens for videos, tokens from a multi-crop image, and tokens for the low-resolution single-crop version of the image. Frames are interleaved with text timestamps written as seconds to one decimal point, and multi-images are interleaved with "Image 1", "Image 2", *etc.*, labels. Text is added after the image/video tokens following the Qwen3 [169] prompt template without thinking tokens.

Pointing. Our pointing format provides points in an HTML-like format, with the coordinates stored in a compact string. For each frame or image with points, the string contains an image index (for image input, starting at 1) or a frame timestamp (for video, shown in seconds with one decimal point), followed by a list of point coordinates. The points each have an *object index*, which is unique for each distinct object being pointed at, and x and y coordinates that are normalized to be between 0 and 1000. Object indices are sequential, starting at 1. The object indices both facilitate counting, because the final object index represents the total count, and enable tracking by identifying repeating objects. Points are sorted by time/frame index and then by x and y coordinates. Values are space-separated, with semi-columns indicating a new frame/image. We elect to use this format over a format like JSON since it dramatically reduces the number of tokens needed to represent points.

An example output for a pointing and tracking task are shown below (new lines added for clarity):

```
<points coords="1 1 555 169;2 3 649 154 4 709 162;5 5 758 175 6 808 183 7 852 187">
Inline text
```

[3] https://pytorch.org/blog/torchcodec/

```
</points>

<tracks coords="0.0 1 635 522;0.5 1 606 490 2 511 124;1.0 2 515 164;1.5 2 520 168">
Inline text
</tracks>
```

Where image indices and frame timestamps are in blue, object indices are in purple, and x and y coordinates are in green. The first example points to an object in images 1, 2, and 5. The second one tracks two different objects through several frames. The "Inline text" is used to describe what is being pointed at.

Hyperparameters. Hyperparameters for the Molmo2 models are shown in Table 12. The connector MLP uses the same intermediate dimension as the LLM, so its size depends on the LLM; otherwise, they are the same across all models. All models use the SigLIP 2 So400m/14 384px ViT [139].

Implementation. Our implementation uses PyTorch with Fully Sharded Data Parallel (FSDP) 2 [185]. We use PyTorch's Scaled Dot Product Attention (SDPA), not FlashAttention [28, 27], since it does not support custom attention masks. We use `torch.compile` to improve throughput and ensure that the shapes in the LLM and ViT are static so the model can be statically compiled, which we find essential for maximizing throughput.

To improve throughput, we also utilize PyTorch's Automatic Mixed Precision (AMP) module[4], which enables most operations to run in half-precision with bfloat16 numbers. Computations for layer normalization [9] and Rotary Position Embedding (RoPE) [131] are still carried out in full precision.

When computing gradients, each GPU computes a gradient on a small mini-batch of examples, after which the gradients are averaged across all devices. We always compute the per-device gradient by dividing the total loss on that device by the *average* number of loss tokens across all devices, not the number of loss tokens on that particular device. This avoids a subtle bias that effectively up-weights examples with a small number of loss tokens (*e.g.*, with short responses)[5] [51].

During fine-tuning, mixing is done within each batch so that the batches contain examples from a variety of datasets. We truncate examples that are longer than the max sequence length. This occurs in $< 0.1\%$ of cases, usually due to videos with both subtitles and a large number of annotations. We find training to be stable, without loss spikes or NaNs.

B Training details

In this section, we provide additional details about packing, the data mixture, and other components of how Molmo2 was trained.

Packing. Our packing algorithm keeps a pool of $M = 48$ examples that have already been preprocessed and converted into a tokenized representation. If the pool is not full, examples are drawn from the training mixture and added to the pool. When the pool is full, we run a dynamic programming solver to find the optimal subset of examples that maximizes $T + I * w_i$ subject to $T \leq 16384$ and $I \leq 128$, where T is the total number of text tokens in the selected subset, I is the total number of crops, and $w_i = 30$ is a hyperparameter. During long context training, we instead use a max of 384 images and 36864 tokens. The selected examples are yielded as a single packed sequence and removed from the pool. In practice, we run the solver on a quantized version of the problem by rounding the number of tokens to the nearest multiple of 32.

Increasing M quickly leads to diminishing returns in terms of packing efficiency. We do not observe any gains from using more than 48. The algorithm is usually robust to w_i, but we observe that in some settings, if w_i is too low, the pool can become filled with examples with 128 crops, which usually cannot be packed with anything else, thereby reducing efficiency.

Implementation-wise, we add this logic into torch's *DataLoader* so that each data-worker runs this algorithm independently. This makes the algorithm easy to use, but it does add some unnecessary overhead when there

[4] https://pytorch.org/docs/stable/report/amp.html
[5] https://unsloth.ai/blog/gradient

		4B	7B	8B
Image Encoder	Params		380m	
	Dim		1152	
	MLP Dim		4304	
	Act.		GELU	
	Heads		16	
	KV Heads		16	
	Layers		27	
	Image Size		384×384	
	Patch Size		14	
	Dropout		0.0	
V/L Connector	Params	57m	80m	88m
	Image Pool Size		2×2	
	Video Pool Size		3×3	
	Pool Dim		1152	
	Pool Heads		16	
	MLP Dim	9728	100352	12288
	Act.		SwiGLU	
	Dropout		0.0	
LLM	Params	4.0b	7.3m	8.2m
	Embed	151936	100352	151936
	Dim	2560	4096	4096
	MLP Dim	9728	11008	12288
	Act.		SwiGLU	
	Heads		32	
	KV Heads	8	32	8
	Layers	36	32	36
	Theta	1m	0.5m	1m
	Dropout		0.1	
Pre-Train	Warmup ViT		2000	
	Warmup Con.		200	
	Warmup LLM		2000	
	LR ViT		6e-6	
	LR Con.		2e-4	
	LR LLM		2e-4	
	Cosine Decay		10%	
	Eps.		1e-6	
	Betas		0.9, 0.95	
	Batch Size		128	
	Sequence Length		2560	
	Steps		32k	
SFT	Warmup ViT		200	
	Warmup Con.		200	
	Warmup LLM		200	
	LR ViT		5e-6	
	LR Con.		5e-6	
	LR LLM		1e-5	
	Cosine Decay		10%	
	Eps.		1e-6	
	Betas		0.9, 0.95	
	Batch Size		128	
	Sequence Length		16384	
	Steps		30k	

Table 12 Model and training hyper-parameters, Molmo2-O-7B is a version of Molmo2 with OLMo 3 [112]. Long-context post-training used the same parameters as SFT

name	rate	visual	anno.	ex.
Image QA	22.7	2.7m	32m	2.4m
PixMo-Clocks	1.9	800k	800k	800k
Llava-665k-Multi	1.5	280k	2.5m	160k
TallyQA	1.4	130k	250k	130k
CoSyn-chart	1.3	120k	1.1m	120k
NLVR2	1.1	100k	86k	86k
VQA v2	1.1	83k	440k	83k
CoSyn-doc	1.0	71k	610k	71k
A-OKVQA	1.0	33k	34k	34k
CoSyn-math	1.0	67k	67k	67k
CoSyn-table	0.8	47k	420k	47k
DocVQA	0.7	10k	39k	39k
CoSyn-diagram	0.7	35k	300k	35k
TextQA	0.7	22k	35k	35k
Molmo2-SynMultiImageQA-chart	0.7	100k	330k	33k
ChartQA	0.6	18k	28k	28k
Molmo2-SynMultiImageQA-doc	0.6	88k	270k	28k
ST-VQA	0.6	18k	25k	25k
InfographicVQA	0.6	4.4k	24k	24k
TabWMP	0.6	23k	23k	23k
PlotQA	0.5	160k	20m	160k
AI2D	0.5	6.2k	15k	15k
Molmo2-SynMultiImageQA-diagram	0.5	45k	150k	15k
Molmo2-SynMultiImageQA-table	0.4	47k	140k	14k
CoSyn-music	0.4	12k	82k	12k
DVQA	0.4	200k	2.3m	200k
FigureQA	0.4	100k	1.3m	100k
OK-VQA	0.4	9k	9k	9k
CoSyn-chemical	0.4	8.9k	55k	8.9k
Spot-the-Difference	0.3	15k	14k	7.5k
ScienceQA	0.3	6.2k	6.2k	6.2k
Molmo2-SynMultiImageQA-music	0.3	12k	46k	4.7k
Molmo2-SynMultiImageQA-chemical	0.2	8k	23k	2.4k
Image Pointing	9.1	510k	5.5m	1.1m
PixMo-Points	4.6	220k	4.6m	530k
Molmo2-MultiImagePoint	2.0	180k	470k	470k
PixMo-Count	1.2	37k	74k	74k
CoSyn-point	1.2	68k	320k	68k
Captions/Long QA	13.6	1.2m	1.6m	1.2m
Molmo2-Cap	3.4	100k	280k	100k
PixMo-CapQa	3.1	190k	270k	190k
PixMo-Cap	2.3	710k	710k	710k
PixMo-AskModelAnything	1.9	71k	160k	71k
Molmo2-MultiImageQA	1.5	98k	73k	45k
Molmo2-AskModelAnything	1.5	43k	130k	43k
NLP	9.1	0	980k	980k
Tulu	9.1	0	980k	980k
Video Pointing	13.6	260k	500k	370k
Molmo2-VideoPoint	10.9	250k	450k	330k
AcademicVideoPoint-MeViS	1.2	1.6k	20k	20k
AcademicVideoPoint-ReVOS	0.7	3.4k	11k	11k
AcademicVideoPoint-LV-VIS	0.7	3.1k	11k	11k
AcademicVideoPoint-OVIS	0.05	600	880	880
AcademicVideoPoint-BURST	0.04	310	680	680
AcademicVideoPoint-Ref-DAVIS17	0.03	58	450	450

name	rate	visual	anno.	ex.
Video QA	18.2	2.3m	4.7m	2.4m
Molmo2-CapQA	1.6	190k	950k	190k
Molmo2-SubtitleQA	1.2	100k	470k	100k
Video Localized Narratives	1.1	53k	180k	56k
TGIF	0.9	63k	210k	63k
TVQA	0.9	120k	120k	120k
Paxion	0.9	440k	440k	440k
Moments In Time	0.9	710k	710k	710k
Kinentics	0.9	420k	420k	420k
LLaVA Academic	0.9	11k	62k	31k
Ego4D	0.9	53k	53k	53k
EPIC KITCHENS	0.7	37k	37k	37k
COIN	0.7	7.8k	30k	30k
How2QA	0.6	25k	35k	25k
ActivityNet	0.5	12k	46k	21k
FunQA	0.5	3.1k	200k	21k
CLEVRER	0.5	10k	130k	20k
STAR	0.5	3k	91k	19k
YouCook2	0.4	1.2k	18k	10k
SUTD-TrafficQA	0.4	10k	56k	10k
CinePile	0.4	9.2k	300k	9.2k
Charades STA	0.4	5.3k	12k	9.2k
QVHighlights	0.3	6.8k	7k	7k
MotionBench	0.3	5k	5k	5k
Countix	0.2	3.9k	4.4k	4.4k
NExT-QA	0.2	3.9k	34k	3.9k
Sports-QA	0.2	3.6k	56k	3.6k
IntentQA	0.2	3.2k	24k	3.2k
NewsVideoQA	0.2	2.9k	8.4k	2.9k
RoadTextVQA	0.2	2.6k	8.4k	2.6k
PerceptionTest	0.2	2k	7.4k	2k
CamaeraBench	0.1	1.4k	1.4k	1.4k
Social IQ 2	0.1	0.79k	5k	0.79k
Video Tracking	13.6	130k	800k	800k
Molmo2-VideoTrack	4.6	8k	220k	220k
AcademicVideoTrack-MeViS	2.0	1.7k	150k	150k
AcademicVideoTrack-ViCaS	1.2	15k	130k	130k
AcademicVideoTrack-ReVOS	1.2	0.7k	82k	82k
AcademicVideoTrack-TrackingNet	1.1	29k	29k	29k
AcademicVideoTrack-Ref-Youtube-VOS	0.9	3.5k	26k	26k
AcademicVideoTrack-VastTrack	0.8	46k	93k	93k
AcademicVideoTrack-LV-VIS	0.8	3.1k	38k	38k
AcademicVideoTrack-GOT-10k	0.4	9.2k	18k	18k
AcademicVideoTrack-WebUAV	0.2	3.2k	6.3k	6.3k
AcademicVideoTrack-BURST	0.07	0.28k	2.9k	2.9k
AcademicVideoTrack-LaSOT	0.06	1.1k	2.2k	2.2k
AcademicVideoTrack-TNL2K	0.06	0.88k	1.8k	1.8k
AcademicVideoTrack-WebUOT	0.05	0.84k	1.5k	1.5k
AcademicVideoTrack-LVOS V2	0.05	0.42k	1.2k	1.2k
AcademicVideoTrack-lasot	0.03	0.22k	0.45k	0.45k
AcademicVideoTrack-UW-COT220	0.03	0.21k	0.4k	0.4k
AcademicVideoTrack-LVOS V1	0.02	0.12k	0.3k	0.3k
AcademicVideoTrack-TNLLT	0.02	0.15k	0.29k	0.29k
AcademicVideoTrack-Ref-DAVIS17	0.02	0.06k	1.1k	1.1k
AcademicVideoTrack-YouTube-VIS	0.02	1.2k	1.4k	1.4k
AcademicVideoTrack-MoCA-Video	0.01	0.13k	0.4k	0.4k

Table 13 Full dataset list. Columns show sampling rates, the number of videos or images, the number of annotations, and the number of training examples built after formatting the data into message trees.

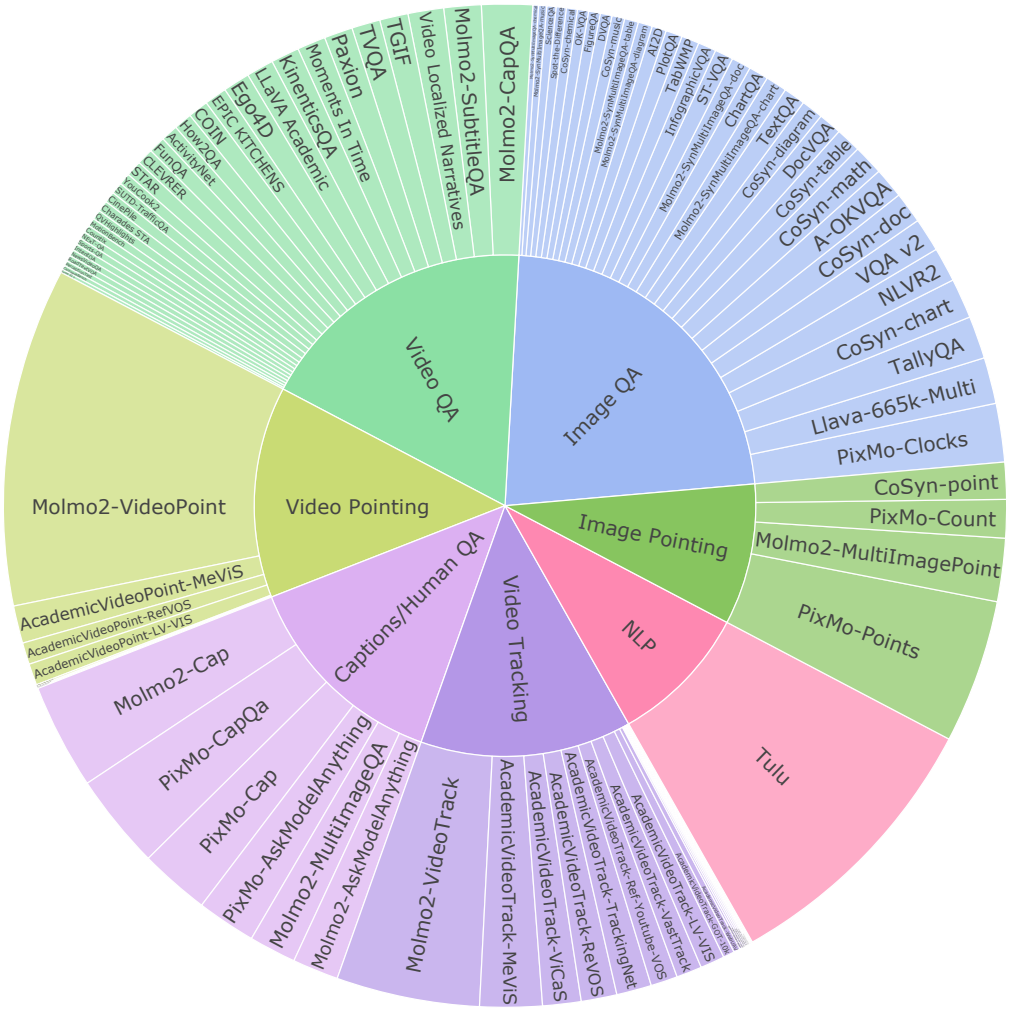


Figure 4 Molmo2 SFT mixture. Categories and datasets are shown in proportion to sampling rates in SFT mixture.

are many data workers. This could be addressed in future work through a deeper integration into torch's data-loading logic. In practice, we find that packing still does not slow down the training speed. Loading and extracting frames from videos remains, by far, the most costly part of data loading.

Pre-training. During pre-training, we use response-only dropout, *i.e.*, residual dropout on just the output tokens, of 0.1, length conditioning, and both the caption and transcript, following Molmo [29].

SFT. The full list of datasets in our SFT mixture is shown in Table 13, and visualized in Figure 4. During SFT we use regular residual dropout of 0.1.

Prompting. We use the human-written questions with long-form answers from PixMo-AskModelAnything, PixMo-CapQA, and Molmo2-AskModelAnything directly. For captioning, all multiple-choice questions, and our various grounding tasks, we use prompt templates to generate a variety of ways to prompt the model for the target output. The remaining short-answer or captioning academic datasets typically have answer styles that are poorly suited for user-facing behaviors, either because they are too terse or have other idiosyncratic quirks due to how the data was collected. For these datasets, we prompt the model with style tags (*e.g.* "short_video_answer:") so that Molmo2 adopts those answer styles only if specifically prompted to do so.

Hyperparameters. Hyperparameters for AdamW [67] are in Table 12. Following Molmo [73], during pre-training, we use a high learning rate for the connector and a long warmup for the ViT and LLM so that the first steps of training mostly train the connector. We use a cosine learning rate that decays to 10% of the

Model	Pre-train			SFT			Long-Context		
	GPUs	time	GPU hr.	GPUs	time	GPU hr.	GPUs	time	GPU hr.
4B	32	15.2	490	128	58.8	7.5k	128	25.3	3.2k
7B	64	11.3	720	128	59.3	7.6k	128	25.7	3.3k
8B	64	12.1	780	128	63.0	8.1k	128	26.0	3.3k

Table 14 Training times. Training was done with Nvidia H100 GPUs.

peak learning rate. We do not use weight decay.

Training time. We show the time and compute used for training Molmo2 in Table 14. During SFT, a high portion of the computation is from the ViT because, for videos, 9 patches in the ViT are processed for each visual token in the LLM. As a result, increasing the LLM size has a reduced effect on the training time.

Specialized models. Specialized models are pre-trained and then undergo a shorter SFT training round with a subset of our SFT data.

For the QA-specialized model, we start with an earlier version of the pre-trained Molmo2-4B checkpoint and perform SFT on video caption and video QA data, excluding image, NLP, and video pointing/tracking datasets. We only train the model for 6k steps. For the captioning-specialized model, we only use the Molmo2-Cap dataset and train the model for 5k steps. For the pointing-specialized model, we use a three-stage training pipeline in which the model is first pre-trained on image captioning for 22k steps, then further trained for 26k steps on the Molmo2 SFT mixture excluding video pointing and tracking data, and finally finetuned for 6k steps solely on video pointing data. For the tracking-specialized model, we use the same three-stage pipeline except that we finetune the model on video pointing and tracking data for 10k steps in the final stage. Finally, the image-specialized model is trained for 24k steps and a sequence length of 2560 on just the NLP, image pointing, image academic, and image datasets from the Captions/Long QA dataset groups, starting from Molmo2-4B pre-trained checkpoint. We do not do long-context post-training for any specialized models.

C Evaluation Details

Next, we provide more details about our evaluation setup.

Captioning. We evaluate video captioning quality on a set of 693 diverse videos using an F1 score designed to evaluate how accurate and detailed the captions are, similar to Molmo [29]. We selected a small number of videos across diverse categories from creative-commons licensed Vimeo[6] to ensure that the videos are disjoint from our training set, which is mostly composed of YouTube videos. The human captions of this evaluation set are collected using a protocol similar to Molmo2-Cap, but with annotators who were manually selected because they provided high-quality captions when collecting Molmo2-Cap. Each evaluation video has up to five human captions. For every model-generated caption and the human caption set, we first prompt GPT-4.1 to enumerate all distinct atomic statements. Precision is computed as the percentage of statements from the model-generated caption that were also stated in the human captions, using GPT-4.1 as a judge. Recall is computed through the opposite process, by matching statements from human captions to the model-generated captions. We average precision and recall across all videos and compute their harmonic mean to obtain our final summary metric: video caption F1.

We prompt Molmo2 and baseline models by asking for a long, detailed caption of the input video.

Human Eval. Following the best practices from [21], we use bootstrapping with 1000 rounds to get a more stable version of Elo ratings and estimate confidence intervals. We plot the Elo scores with confidence intervals in Figure 5.

To better understand the results from human preference evaluation, we also analyze (1) fine-grained task-specific Elo ratings for diagnostic purposes [82] (Table 15), (2) deterministic pairwise win rates (Figure 6); and (3) human explanations of their preference. From the task-specific results, we learn that Molmo2

[6] https://vimeo.com/creativecommons/cc0

	Overall		Captioning		QA	
Model	Score	Rank	Score	Rank	Score	Rank
API call only						
GPT-5 [114]	1031	10	1136	2	1019	11
GPT-5 mini [114]	1076	4	1086	5	1075	4
Gemini 3 Pro [45]	1082	3	1126	3	1076	3
Gemini 2.5 Pro [25]	1096	1	1148	1	1090	1
Gemini 2.5 Flash [25]	1084	2	1109	4	1082	2
Claude Sonnet 4.5 [5]	1008	12	1009	10	1008	12
Open weights only						
InternVL3.5-4B [149]	935	19	817	19	947	19
InternVL3.5-8B [149]	941	18	855	18	951	17
Qwen3-VL-4B [10]	1048	7	1052	7	1049	6
Qwen3-VL-8B [10]	1054	6	1105	5	1048	7
Keye-VL-1.5-8B [170]	952	17	957	15	950	18
GLM-4.1V-9B [137]	962	14	1013	9	956	15
MiniCPM-V-4.5-8B [176]	975	13	978	14	975	13
Eagle2.5-8B [17]	1019	11	987	13	1022	10
Open models						
PLM-3B [22]	841	21	880	17	836	21
PLM-8B [22]	853	20	761	21	863	20
LLaVA-Video-7B [184]	959	15	981	14	955	16
VideoChat-Flash-7B [79]	956	16	932	16	959	14
Molmo2 family: Open weights, Open data, Open code						
Molmo2-4B	1041	8	1004	11	1045	8
Molmo2-8B	1057	5	1049	8	1059	5
Molmo2-O-7B	1033	9	1019	9	1034	9

Table 15 Human evaluation results. Scores updated using bootstrap Elo medians from overall, captioning, and QA evaluations.

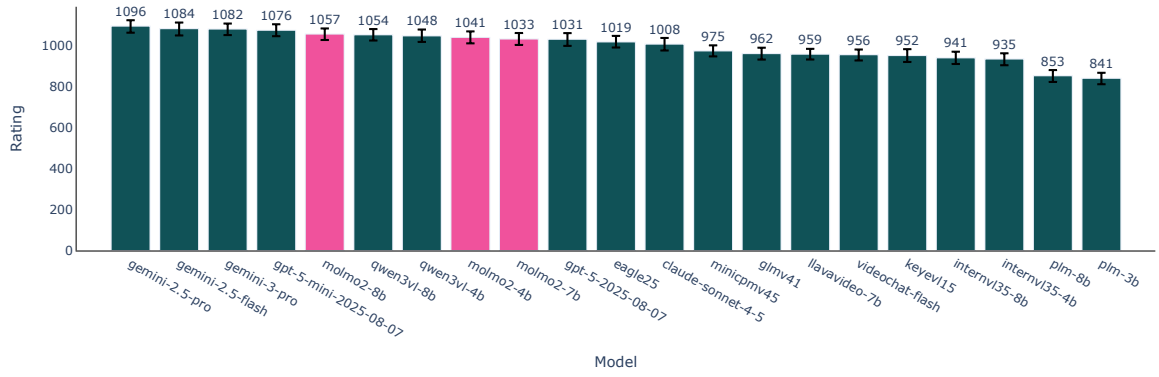


Figure 5 Elo ratings with confidence intervals

performs better than Qwen3-VL on the open-ended QA task, ranking first among open models. However, it underperforms Qwen3-VL and GLM-4.1V on captioning. Furthermore, we also examine the pairwise win rates

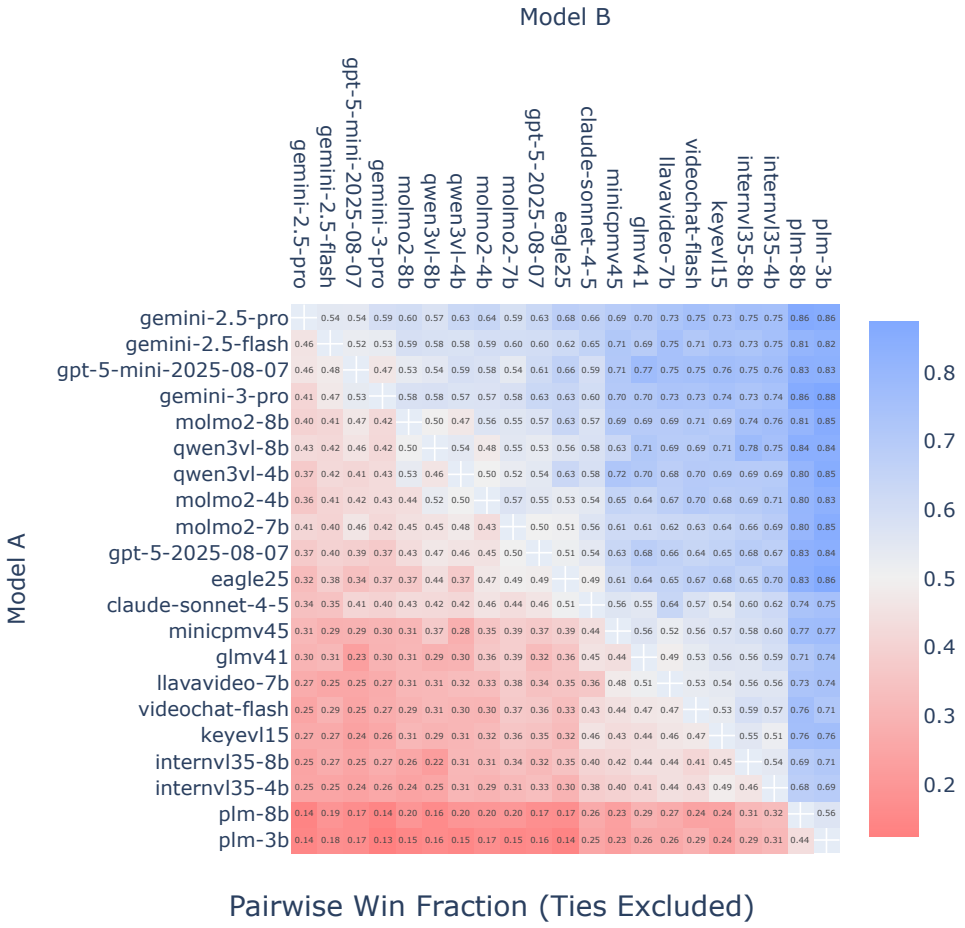


Figure 6 Pairwise win rates across all model pairs in human preference evaluation.

across all model pairs, which are deterministic. We note that Molmo2-8B's win rate against Qwen3-VL-8B is 53%, and Molmo2-4B's win rate against Qwen3-VL-4B is 51%, suggesting that Molmo2 family of models is competitive against Qwen3-VL models. Lastly, from a qualitative analysis of human annotators' explanations of their preferences, we learn that our model performs well on QA because it provides a detailed explanation to its answer when needed and a concise one otherwise, However Molmo2 falls short on captioning because it sometimes outputs repetitive or non-sensical content at the end of the caption, which we believe is due to text-repetition issues when generating extremely long output (see Section H).

Counting and Pointing. For the video counting evaluation, we preprocess 2 fps videos and clip them to random intervals under 63 seconds. In addition to exact accuracy and close accuracy, we also track models' counting accuracy by query category (Table 16) and by object count (Table 17). We find that Molmo2-8B performs the best on Action/Event and Object counting, just behind Gemini 2.5 Pro and GPT-5. Molmo2-8B also performs competitively on Animal counting, trailing slightly behind GPT-5 and Qwen3-VL-8B. Importantly, Molmo2 achieves similar accuracies to Qwen3-VL on low-count (0-10) queries while performing substantially better on high-count cases (10-60). Notably, Qwen3-VL obtains 0% accuracy in the 25-60 range, whereas Molmo2 exceeds 10%, placing it just behind Gemini 2.5 Pro.

For the video pointing evaluation, we use 2 fps videos with a maximum of 384 frames along with ground truth points and masks at 2 fps. For metrics, we compute recall, precision, F1, and valid accuracy (*i.e.*, the percentage of predictions that are parsed correctly), reporting all metrics in Table 3. In contrast to the counting task, Qwen3-VL struggles to perform meaningful pointing: Qwen3-VL-8B achieves only 1.5 F1,

	Query Catogery			
Model	Action/Event	Animal	Object	Avg.
API call only				
GPT-5 [114]	46.6	75.5	29.8	50.6
GPT-5 mini [114]	36.2	63.3	25.1	41.5
Gemini 3 Pro [45]	58.6	75.5	29.7	54.6
Gemini 2.5 Pro [25]	53.4	63.3	30.0	48.9
Gemini 2.5 Flash [25]	36.2	63.3	27.7	42.4
Claude Sonnet 4.5 [5]	26.3	53.1	24.3	34.6
Open weights only				
Qwen3-VL-4B [10]	39.7	59.2	19.5	39.4
Qwen3-VL-8B [10]	43.1	**75.5**	22.5	**47.0**
Molmo2 family: Open weights, Open data, Open code				
Molmo2-4B	**51.7**	59.2	29.1	46.7
Molmo2-8B	50.0	69.4	29.6	**49.7**
Molmo2-O-7B	50.0	63.3	**27.5**	46.9

Table 16 Molmo2-VideoCount accuracy by query category.

indicating that it rarely produces correct points. Even the strongest proprietary model shows a significant gap relative to ours: Gemini 3 and 2.5 Pro reach 20.0 and 13.0 F1, whereas Molmo2-4B and Molmo2-8B achieve 39.9 and 38.4 F1, respectively. This highlights a substantial performance advantage of Molmo2 on fine-grained spatio-temporal localization.

To evaluate the performance of baseline models on counting and pointing, we adopt the following setups. For both counting and pointing, we feed the entire videos to Gemini and Qwen3-VL models and use their default setup for video preprocessing. For GPT and Claude models, we feed the video frames to them using the same max frames and fps in our models' video preprocessing. As for the prompt, we use a general counting prompt followed by a brief format instruction across all models: "How many {label} are there? Output the integer number of the count only. The answer is:". For pointing, we first try prompting baseline models with our pointing format, but find that they struggle to follow the instruction and produce sensible outputs. We then carefully review various cookbooks for the baseline models where available, and design prompts with the HH:MM:SS format for timestamps and the bounding box format (which we then calculate the center's coordinates and use those for evaluation). We present the prompts used in video pointing evaluation for models with video and image inputs in prompt 1 and 2, respectively.

```
You are a video-analysis assistant that points to unique target objects in the video at 2FPS.

Goal:
Point to the timestamp and spatial coordinates of target objects, actions, or events in the input
    video.
- timestamp (as a string in ‘HH:MM:SS‘ format, where the second can be to the closest 0.5 seconds e.
    g. ‘00:01:23.5‘)
- x_min, y_min, x_max, y_max (integer coordinates normalized to a 0-1000 scale)

Rules (strict):
- For actions/events spanning some time, pick the most representative / clear timestamp.
- Each instance should be a separate spatial-temporal point in "results".
- Do NOT point to the same object more than once.
- Return only valid JSON, without markdown code blocks, explanations, or extra text.

Output format (strict JSON):
{
  "results": [
    {
```

	Object Count						
Model	0–5	5–10	10–15	15–20	20–25	25–60	Avg.
API call only							
GPT-5 [114]	64.4	34.1	31.3	16.2	11.1	10.5	27.9
GPT-5 mini [114]	55.7	28.2	25.0	10.8	6.3	10.5	22.8
Gemini 3 Pro [45]	69.5	34.1	24.1	16.2	14.3	12.5	28.5
Gemini 2.5 Pro [25]	61.5	31.3	31.5	15.7	17.5	13.0	28.4
Gemini 2.5 Flash [25]	56.9	31.0	27.5	19.2	9.8	3.5	24.6
Claude Sonnet 4.5 [5]	48.0	24.7	20.3	14.9	15.9	5.4	21.5
Open weights only							
Qwen3-VL-4B [10]	56.9	17.6	21.3	2.7	3.2	0.0	16.9
Qwen3-VL-8B [10]	63.8	30.6	15.0	6.8	6.3	0.0	20.4
Molmo2 family: Open weights, Open data, Open code							
Molmo2-4B	58.0	31.8	30.0	24.3	**9.5**	**12.3**	**27.7**
Molmo2-8B	**64.4**	**32.9**	26.3	**25.7**	7.9	7.0	27.4
Molmo2-O-7B	60.9	32.9	**27.5**	16.2	6.3	8.8	25.4

Table 17 Molmo2-VideoCount accuracy by object count.

```
      "timestamp": <str>, ‘HH:MM:SS‘ format
      "x_min": <int>,
      "y_min": <int>,
      "x_max": <int>,
      "y_max": <int>
    },
    ...
  ]
}

Target: {label}
```

Listing 1 Video pointing prompt for baselines with video inputs

```
You are a video-analysis assistant that points to unique target objects in the video, represented as
    a sequence of image frames at 2FPS.

Goal:
Point to the timestamp and spatial coordinates of target objects, actions, or events in the input
    video frames at 0.5 second
    intervals.
- timestamp (as a string in ‘HH:MM:SS‘ format, where the second can be to the closest 0.5 seconds e.
    g. ‘00:01:23.5‘)
- x_min, y_min, x_max, y_max (integer coordinates normalized to a 0-1000 scale)

Rules (strict):
- For actions/events spanning some time, pick the most representative / clear timestamp.
- Each instance should be a separate spatial-temporal point in "results".
- Do NOT point to the same object more than once.
- Return only valid JSON, without markdown code blocks, explanations, or extra text.

Output format (strict JSON):
{
"results": [
    {
    "timestamp": <str>, ‘HH:MM:SS‘ format
    "x_min": <int>,
```

```
    "y_min": <int>,
    "x_max": <int>,
    "y_max": <int>
    },
    ...
]
}

Target: {label}
```

Listing 2 Video pointing prompt for baselines with image inputs

Tracking. We explain the tracking evaluation setup used for Tables 4–5. Across all benchmarks, segmentation metrics are computed at the original video frame rate, while point-based metrics are evaluated at 1 fps and marked as correct if they fall inside the mask. For baselines, we evaluate specialized open segmentation models that output a single foreground mask per frame and report their segmentation quality. When a model can produce discrete points per object (e.g., VLMs), we additionally report its point-based metrics. We found that API models and generic VLMs are incapable of producing accurate point tracks, as shown in the video pointing task (Table 3), but their grounding performance improves substantially when prompted to output bounding boxes instead. Thus, for these models, we predict bounding boxes at 1-second intervals, use the boxes to prompt SAM 2 to generate segmentation masks, and take the box centers as representative points for point-based metrics. Our model, instead, can predict discrete point tracks with explicit IDs, and their points are directly fed to SAM 2 to obtain segmentation masks.

For metrics, we report their average $\mathcal{J}\&\mathcal{F}$ over all objects and frames as a standard metric for segmentation quality. The Jaccard index $\mathcal{J}$ measures region overlap between predicted and ground-truth masks via intersection-over-union (IoU). The boundary F-score $\mathcal{F}$ measures how well predicted and ground-truth object contours align. **Point F1** is computed similarly to the video counting task but at 1 fps, and captures frame-wise detection performance. Since Point F1 is insensitive to identity swaps when the number of objects remains constant, we also report **HOTA** [97] (HOTA = $\sqrt{\text{DetA} \times \text{AssA}}$) to measure tracking quality, which jointly scores detection accuracy (DetA) and association accuracy (AssA). While originally designed for bounding box tracking, where similarity is measured via IoU, we adapt HOTA to point-based tracking by defining similarity as binary: a predicted point matches a ground-truth object if it falls within the object's segmentation mask. DetA then measures whether points are placed in correct masks, while AssA measures whether consistent object IDs are maintained over time based on their presence in the mask and penalizes identity switches if swapped. Since baseline models do not output stable track IDs but only counts, HOTA is only reported for Molmo2 that can perform tracking reliably.

Table 4 presents comprehensive results across all academic benchmarks and their splits. We see Molmo2 substantially outperforms API-based and open-source VLMs by a wide margin, suggesting the existing VLMs are not well-suited for object tracking tasks. Specialized open models that directly generate segmentation also fall behind our approach, indicating their inability to effectively ground object semantics despite being specifically trained for tracking. The most directly comparable baseline is VideoMolmo [3], another video language model trained for point grounding in videos. While specialized models perform on par or outperform our model on Ref-Davis, which involves single objects with simple text queries, our model excels in more complex scenarios beyond basic tracking, where it significantly outperforms multi-object tracking supported in MeViS [31] and reasoning-intensive tasks in ReasonVOS [166].

Lastly, we report the performance on our proposed benchmark Molmo2-Track in Table 5, further broken down by video domains. Overall, Molmo2 comes out on top, outperforming other VLMs and even the specialized open video models. Across the board, API-based and open-source VLMs, including Molmo and VideoMolmo [3], struggle to count and track consistent objects throughout videos, as indicated by their low F1 and HOTA scores. Interestingly, the Molmo variants and specialized models achieve a high segmentation score ($\mathcal{J}\&\mathcal{F}$), though we observe that for cluttered scenes–such as Pedestrians, Sports, and Dancers–models generate large, coarse masks covering entire people rather than precisely localizing individual objects. This results in high region overlap that inflates $\mathcal{J}\&\mathcal{F}$ while failing to accurately ground and track specific objects, as reflected in the substantially lower F1 and HOTA scores. This highlights the importance and necessity of our point-based F1 and identity-aware HOTA metrics, which more directly measure a model's ability to

precisely ground and track the correct objects.

D Additional results

In this section, we present several additional evaluations.

D.1 Additional model ablations

Pretrain	Video QA	Molmo2 Video Cap.	Image QA	Image Pointing
With pointing	66.8	31.8	80.9	73.0
No pointing	65.9	31.3	80.1	71.8

Table 18 Pre-training ablations. Columns show the average of our 12 video benchmarks, using validation sets for EgoSchema, PerceptionText, and MLVU, video captioning F1, the average of the 11 image benchmarks using validation sets for InfoQA, DocQA, ChartQA, VQA v2, and AI2D, and the average score in Point-Bench.

Pre-traing ablation. We also present an ablation without image-pointing pre-training in Table 18. This model is only trained on image captioning and NLP data. For the SFT stage, it uses 2x the sampling rate for the image pointing datasets and 28k steps of training instead of 25k to compensate for the fact that the image pointing data is not seen during pre-training. We observe a small decrease in the benchmarks in this setting, even for those not related to image pointing. We hypothesize that pointing pre-training simplifies the SFT stage for the model since it no longer needs to learn the basic pointing format and task, allowing for more focus on the non-pointing tasks.

D.2 NLP Benchmarks

Model	MMLU [50]	GSM8K [24]	ARC-C [23]	MBPP+ [8]
Qwen3-4B [169]	72.2	87.8	83.3	59.5
Qwen3-8B [169]	76.8	89.8	88.3	62.2
OLMo3-7B-Instruct [112]	69.1	90.1	72.2	60.2
Molmo2-4B	72.2	86.6	89.3	56.2
Molmo2-8B	76.6	89.7	89.6	57.5
Molmo2-O-7B	64.1	89.0	79.9	55.7

Table 19 Results on selective NLP benchmarks, including MMLU for general knowledge QA, GSM8K for math, ARC-C for reasoning, and MBPP+ for coding tasks.

We evaluate Molmo2 on selective NLP benchmarks covering general knowledge QA, math, reasoning, and coding tasks and report their results compared to the base language models Qwen3 in Table 19. We run evaluations for all models following OLMo 3's evaluation protocol, except for OLMo3-7B-Instruct's MMLU and MBPP+ numbers, which we take directly from OLMo3's model card. We find that Molmo2 achieves comparable numbers on the general knowledge QA and math benchmarks, MMLU and GSM8K, but suffers from some drops in coding on the MBPP+ coding benchmark [8]. Interestingly, both Molmo2-4B and Molmo2-8B perform slightly better than their respective base language models in the ARC Challenge multiple-choice evaluation.

E Test time scaling with 128-frame model

In this section, we consider whether it is possible to scale the number of frames past 128 during inference without long-context training. We also test an approach using SlowFast [164] to provide the model with a mix of high and low-resolution frames during inference, or during both training and inference.

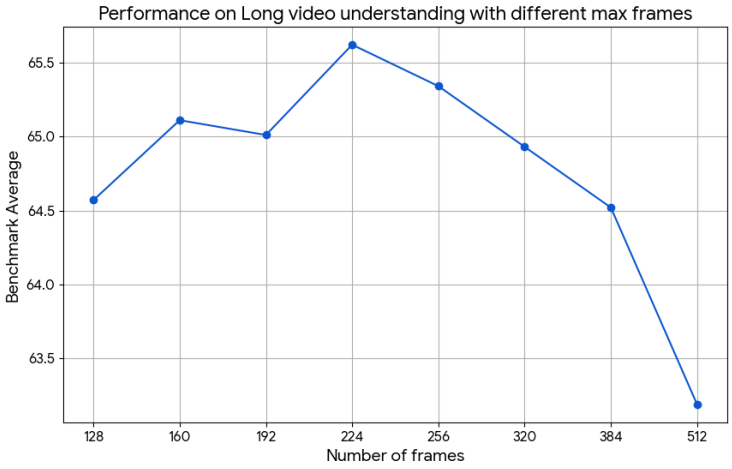


Figure 7 Long video benchmark results with different max frames, the average of our six long video benchmarks.

Model	VTok	Video-MME	Video-MME-Sub	LongVideoBench	MLVU	LVBench	VideoEvalPro	Short QA avg	Long QA avg
128 frames	10.6k	68.8	74.3	65.9	74.5	49.6	54.3	**69.8**	64.6
pool4, 216 frames	11k	68.9	**75.0**	64.3	75.7	48.9	54.9	68.8	64.6
pool5, 332 frames	10.6k	69.1	74.2	64.2	**76.5**	50.6	56.9	68.4	65.2
128 frames + SF-periodic	10.7k	68.1	74.5	64.2	74.5	48.3	53.5	69.6	63.9
128 frames + SF-diff	10.7k	68.4	74.1	64.7	75.7	48.7	54.8	69.6	64.4
128 frames + SF-query	10.7k	68.9	73.9	**66.6**	76.2	**51.5**	**57.2**	69.6	**65.7**
128 frames + SF-tr-0.1	10.7k	69.1	74.3	65.4	75.0	48.6	54.3	69.8	64.4
128 frames + SF-tr-0.1 + SF-query	10.7k	68.9	74.3	65.5	75.4	51.5	57.1	69.8	65.5
224 frames	18.6k	**69.2**	74.6	66.1	76.4	50.7	56.7	69.7	65.6

Table 20 Molmo2-8B with test time scaling / SlowFast (SF) encoding SF-query boosts long video understanding and matches using 224 frames while using ~ 43% fewer visual tokens. Training without SF and then using SF-query marginally beats training with SF-tr-0.1 on long video understanding tasks. All SlowFast models use a max of 368 frames. VTok denotes max vision tokens. SF-tr-0.1 denotes using SlowFast 10% of the time in training.

Increasing max frames. At test time, we scale the maximum number of frames for better long video understanding. We evaluate Molmo2-8B after the SFT stage, but before long-context training, with 160, 192, 224, 256, 320, and 512 max frames and report the average on the val sets of our six long video understanding benchmarks in Figure 7. Molmo2 has the best performance with 224 frames for long video benchmarks. For short video understanding benchmarks, the average is 69.8 for 128 frames and 69.7 for all other settings as shown in Table 20.

Keeping Vision tokens fixed. However, increasing the maximum number of frames also increases the number of vision tokens fed into the model, which raises compute cost and may not be feasible on GPUs with limited memory. With the default setting of max 128 frames, the maximum number of vision tokens is $83 * 128 \sim 10.6k$. We therefore evaluate alternative test-time strategies that keep the number of max vision tokens close to $10.6k$. Specifically, we evaluate different pooling strategies in the vision-language connector - 4×4 pooling with 216 frames and 5×5 pooling with 332 frames. The 5×5 pooling setting improves long video understanding by accessing more frames; however, both settings regress on short video understanding (Table 20).

SlowFast encoding. Since we find that our model can generalize to different pooling sizes at test time, we further explore a SlowFast video strategy [164]. We build on the interleaved SlowFast variant used in [165, 170, 129], which dynamically allocates computational resources across frames by varying their spatial pooling in the Molmo2 connector, with each frame represented exactly once – either in the slow or the fast pathway. Frames are categorized as slow or fast based on a periodicity parameter p: every p-th frame is designated as a slow frame, while the remaining frames are fast frames. We refer to this approach as *Slowfast-periodic*. Note that $p = 1$ reduces to the default setting. Slow frames use the default pooling size

of 3×3, whereas fast frames use 9×9 pooling. We use four different periodicities $p \in \{1, 2, 3, 4\}$ with corresponding max frames $M \in \{128, 224, 300, 368\}$. The max frame M for each periodicity is chosen such that the maximum number of vision tokens input to the LLM is approximately $10.6k$. $10.6k$ is the maximum number of vision tokens used in the default setup of Molmo2. When processing a video with SlowFast encoding, after we sample F_t frames, p is selected to maximize the tokens in the slow pathway. For example, when $F_t \leq 128$, we use $p = 1$ and all the frames are in the slow pathway, or when $128 < F_t \leq 224$, we use $p = 2$ and every other frame is in the slow pathway. In practice, that leads to stepwise changes in selected p as the number of frames ranges from 1 to 368.

We explore two strategies to score the frames' relevance for inclusion in the slow pathway. First, we embed both the query and all the frames using SigLIP 2 [139] and calculate per frame cosine similarity scores. Second, we calculate the average of the absolute similarity difference of the embedded frames with their neighboring frames. In either strategy, we use the per-frame score to select the relevant frames for the slow pathway. Our formulation when selecting F_s slow pathway frames from F_t sampled frames is to include both frames that globally have the highest scores and frames that have high scores in their local neighborhoods. To select locally high scoring frames, we first select $F_s/2$ frames by choosing the single highest scoring frame from temporally ordered groups of size $F_t \div F_s/2$. To select globally relevant frames, we select the remaining $F_s/2$ frames that have the highest scores from all the remaining frames. Additionally, we don't use score based selection and use Slowfast-periodic when the frames per second F_r is high. This follows the intuition that frame selection is useful when selecting amongst sparser frames for long videos with multiple scenes, but not for shorter videos that get densely sampled and tend to have only one scene. In practice, we fall back to Slowfast-periodic when $F_r \geq 2$.

With Slowfast-periodic, the model regresses on the long video understanding, contrary to the finding in [164]. Using the frame difference improves over using periodic sampling, but still lags behind the default setting. However, using the query to select frames for the slow pathway achieves the best performance. It provides a boost to long video understanding with minor regression in short video understanding. It closes the gap to the optimal setting of using 224 frames while having ~ 43% fewer visual tokens (Table 20).

Training with SlowFast. Due to the improvement on long video understanding tasks using SlowFast encoding in the training-free regime, we explore training with SlowFast. We report results for training in a combined single stage starting from the image captioner. We keep the max frames the same 128 and sample using the SlowFast setup with a probability P_{sf} while randomly sampling different $p \in 2, 4, 8$. We use the default sampling with a probability if $1 - P_{sf}$ and use $P_{sf} = 0.1$. When training with a SlowFast setup, we randomize the slow frames. Concretely, to select F_s frames from F_t sampled frames, 1 frame in ordered groups of size $F_t \div F_s$ is selected randomly. Even though the max frames is not increased, the goal is to familiarize the video model with the SlowFast encoding similar to score-based Slow frame selection, but without increasing the training cost by requiring the use of more frames. At test time, we evaluate with and without the query based SlowFast setup described above. Surprisingly, training without SF and then using the query to select Slow frames beats training with SF 10% of the time as shown in Table 20. This suggests Molmo2 can frame using 9×9 pooling even though such frames were not seen during training.

F Dataset details

In this section, we provide additional details about our data collection methodology.

F.1 Dataset statistics

Pointing. We report the statistics on the Molmo2-VideoPoint training and validation sets. Overall, the Molmo2-VideoPoint dataset contains diverse pointing queries across seven categories (Figure 8). There are more queries in Action/Event, Object, and Referring expression, as we expect these to be harder for the model to learn. We also see that the distribution is skewed towards low-count examples with 0 to 5 counts (Figure 8 and 10). We mitigate this bias by upsampling medium- and high-count examples during training, and plan to collect more high-count examples in the future. Similarly, the distribution of frames annotated per query is also heavily skewed to the left (Figure 9).

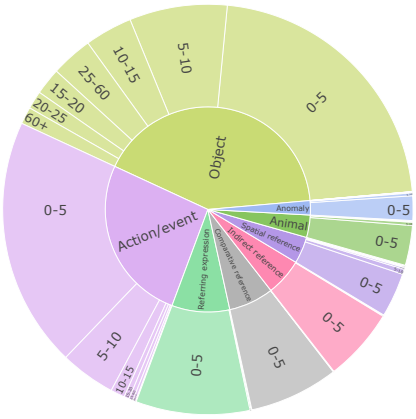


Figure 8 The distribution of categories and counts across pointing queries in **Molmo2-VideoPoint**.

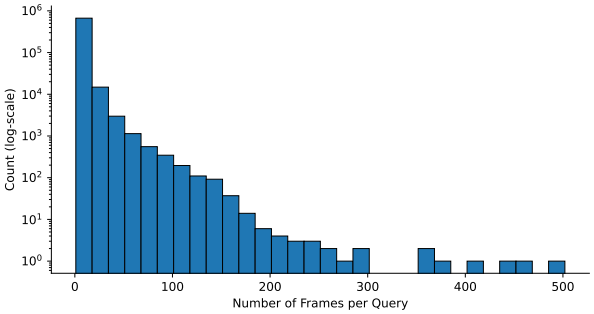


Figure 9 The distribution of annotated frame count per query in **Molmo2-VideoPoint**.

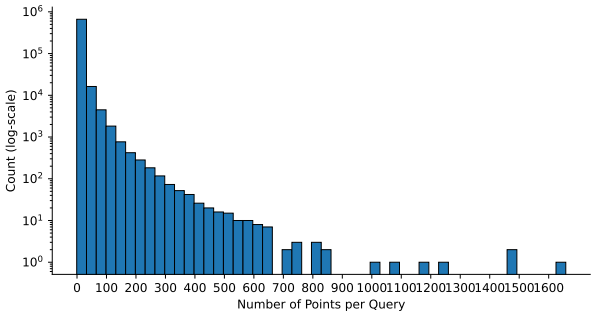


Figure 10 The distribution of annotated point count per query in **Molmo2-VideoPoint**.

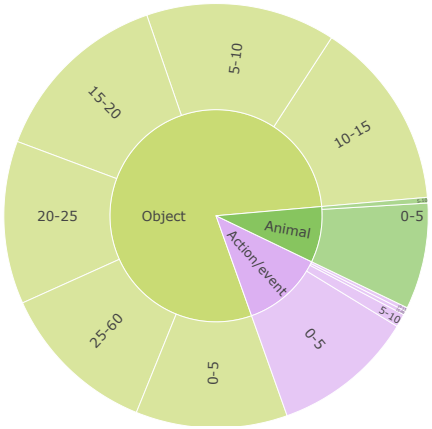


Figure 11 The distribution of categories and counts across queries in the **Molmo2-VideoCount** evaluation.

Figure 12 The distribution of categories and counts across queries in the **Molmo2-VideoPoint** evaluation.

For the validation sets used in Molmo2-VideoCount and Molmo2-VideoPoint evaluations, we carefully build them by (1) collecting double annotations on some queries and selecting high-confidence examples where two different annotators provide the same answer; and (2) sampling queries across diverse categories and counts (Figure 11 and 12). For video counting, we mostly sample queries from the object category, as there are significantly more high-count examples in this category than in others (Figure 11). For video pointing evaluation, we intentionally pick queries in the more difficult categories – referring expression and indirect reference (Figure 12) – orthogonal to the ones in the counting evaluation, so that we have a comprehensive evaluation of our model's counting and pointing capabilities.

Tracking. We report statistics on the videos and text queries in Molmo2-VideoTrack and the Molmo2-Track benchmark. The two datasets have a total of 8k video clips, with 6.6k for training and 1.3k for evaluation. Both datasets provide segmentation masks, text queries, and metadata for each video. On average, there are 6.08 annotated objects per video, and the videos are up to 2 minutes long, with most being around 10-30 seconds. The distribution of video durations is shown in Figure 13.

Our dataset contains a total of 29k diverse text queries covering a wide variety of categories, bringing an average of 1.33 text queries per video. The distribution of categories is detailed in Figure 16 and Figure 17. Multi-object tracking is a primary focus in the tracking capabilities of Molmo2, so we strived to find text queries that describe many objects within a video. The dataset has an average of 3.31 objects described per text query, with many queries describing far more than that. The distribution is shown in Figure 14. Each text query is on average 8.21 words long, but there is a wide range. The exact distribution across all text queries is shown in Figure 15.

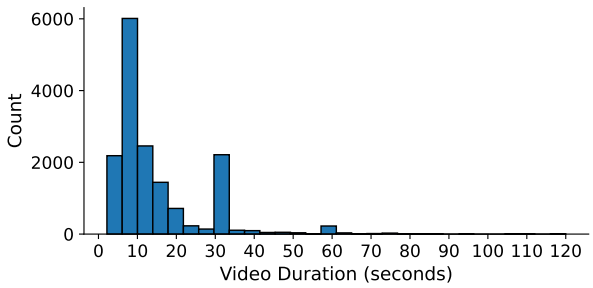


Figure 13 Distribution of video clip duration in **Molmo2-VideoTrack** and **Molmo2-Track**.

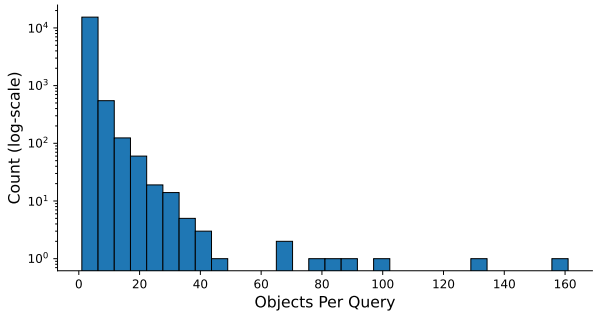


Figure 14 Distribution of objects described by text queries in **Molmo2-VideoTrack** and **Molmo2-Track**.

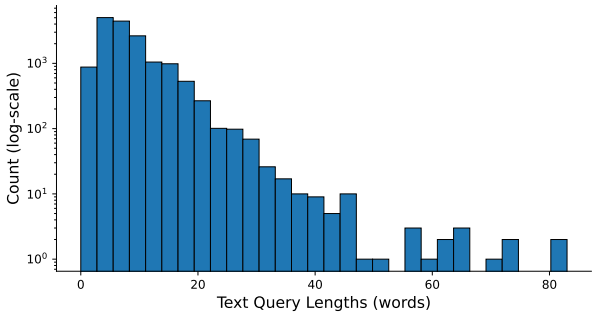


Figure 15 Distribution of text query lengths in **Molmo2-VideoTrack** and **Molmo2-Track**.

F.2 Data collection

Here, we detail how we collect videos and synthesize annotations for most of Molmo2 video datasets.

Video collection for Molmo2-Cap. We first source videos less than 3 minutes from multiple large-scale datasets [180, 147, 153, 184] and YouTube videos searched with keywords used in MetaCLIP [162] to form a

pool of over 10M videos.

Then, we perform one step of filtering based on the informativeness of the video: we first discard the audio track and uniformly sample the video at 1 fps; Then the sampled frames are encoded using H.264; The total size of the resulting encoded stream (in bits) is divided by the product of the video duration and spatial resolution (duration $\times$ W $\times$ H) to obtain a normalized video informativeness score. After collecting scores for all videos in the pool, we discard those whose score falls below (mean - 1 standard deviation), effectively removing videos with unusually low visual or temporal diversity.

After this filtering, we conduct a diversity-based sampling to obtain a final set of videos for human annotation: for each remaining video, we uniformly sample 5 frames and apply SAM 2 [122] to segment each frame, computing the average number of segments as a proxy for visual complexity. We further use Molmo to caption each sampled frame and follow MetaCLIP's processing pipeline to extract a set of keywords that characterize its semantic content. To select a diverse subset, we perform a greedy sampling procedure that aims to maximize the entropy of both the segment-count distribution and the keyword distribution. At each step, we score all candidate videos using a two-stage ranking: (1) we compute a "what-if" entropy gain for the keyword distribution if the candidate were selected, and rank candidates accordingly; (2) we compute a density-based score that favors videos contributing to underrepresented segment-count regions. The final score is obtained by summing the two ranks, and we select the top-ranked candidate. For efficiency, we approximate this process by scanning the pool in chunks of 1,000 candidates at a time, rather than evaluating the entire pool at each iteration. This procedure yields a video subset that is both semantically diverse and visually varied, providing a strong foundation for high-quality human annotations. Finally, we set the sampling ratio to be 1% and obtained around 100k videos.

Video and synthetic annotation collection for Molmo2-CapQA, -SubtitleQA, -VideoPoint, and -AskModelAnything. We first source 500k videos with Creative Commons license from YT-Temporal [180] and YouTube keyword search. Then we use a video captioner trained on Molmo2-Cap to caption these videos. In particular, we segment each video into multiple scenes and caption each scene instead of the entire video to encourage detailed descriptions. Since model-generated captions can sometimes be low-quality, we apply a heuristic rule-based filter to remove captions with repetition patterns. The final set of videos and synthetic captions is used to curate Molmo2-CapQA, -SubtitleQA, and -VideoPoint datasets.

For Molmo2-CapQA and Molmo2-SubtitleQA, we prompt an LLM to generate both the question and the answer. For Molmo2-VideoPoint, we prompt an LLM to generate the queries and solicit human answers. For Molmo2-AskModelAnything, we elicit questions from human annotators and generate the corresponding answers using an LLM with human feedback.

F.3 Data annoation

Molmo2-Cap. To obtain clips for the first-stage captioning, we develop an algorithm to split a video into clips of variable lengths between 10 and 30 seconds based on their information density so that a more informative clip has a shorter duration. This algorithm minimizes the highest information density of a video clip across all clips. Overall, videos are split into 4-5 clips on average. We then deploy the video-description task to online crowdworkers (see Figure 21 for the task interface). For each full video, workers are first shown a sequence of shorter clips split by our algorithm from the original video with audio muted. At the top of the interface, we provide instructions to guide their descriptions. For each clip, workers verbally describe what is happening on the screen, and their speech is automatically converted to text via real-time transcription. They then edit the transcript to correct recognition errors before submitting it. After completing all clips, workers are asked to provide a comprehensive description of the full video (see Figure 22).

Molmo2-VideoPoint. For each video, we design several visual questions that require workers to answer using evidence from a single or several frames (see Figure 23 for the task interface). Crowdworkers first watch the full video clip without audio. For each question, they capture screenshots from the video at the moments when the relevant content is visible. On the screenshot, workers annotate points on object instances that satisfy the question, and we record both the video timestamp and the (x, y) coordinates of all points. Then they answer the corresponding questions in a required format. Workers could mark a question as Unanswerable (e.g., if the content is missing or ambiguous) or flag that they are unsure about their answer. This process is repeated

for all questions associated with the video.

To collect annotations for anomaly identification queries in Molmo2-VideoPoint, we first need to construct a dataset of generative videos exhibiting visual defects. We begin by leveraging two publicly available datasets: the ViBe dataset [124] and the Broken Video Detection Dataset [85]. The Broken Video Detection Dataset provides high-quality, frame-level annotations of defective regions, allowing us to directly incorporate its pixel-accurate defect masks. From the ViBe dataset, we selectively retain only videos labeled as Vanishing Subject, Physical Incongruity, or Temporal Dysmorphia. These categories correspond to defects intrinsic to the generated video itself rather than issues arising from ill-posed or misleading prompts, ensuring our dataset focuses on model-induced visual failures. To complement these sources with realistic user prompts, we sample 2,000 human-written prompts from the VidProM dataset [148]. For each prompt, we generate videos using 10 T2V models and manually filter the outputs to retain only those containing clear and salient defects. This step introduces diversity in both content and failure types and reflects real-world usage patterns of contemporary text-to-video systems. In total, our final training set for generative video anomaly pointing consists of 10k videos, covering a broad range of defective generations produced by around 25 T2V models.

Molmo2-VideoTrack. Directly reusing the Molmo2-VideoPoint annotation strategy for tracking is infeasible, as it would require point annotations on every sampled frame. One could use off-the-shelf tracking models, such as Co-Tracker [63] or SAM 2 [122], with point prompts; however, we found them to yield incomplete or unstable trajectories and are therefore not reliable sources for generating accurate training data for tracking. We thus resort to existing human-annotated tracks and focus on expanding coverage to video domains and object categories underrepresented in standard training datasets.

As our base pool, we use a set of videos in video object segmentation (VOS) datasets: SAM-V [122], VIPSeg [108], MOSE [32], and MOSEv2 [33], which are not as densely supported in existing academic video track datasets. We discard videos that are shorter than 3 seconds or that contain fewer than three object tracks. We additionally decontaminate videos in MOSE [32] with respect to the MeViS validation set [31]; we sample 8 frames per video, extract CLIP ViT-L/14 features [119], and remove any videos whose maximum pairwise frame similarity exceeds 0.95. We then extract points from segmentation masks by computing an alpha-weighted score that combines centroid distance and distance to mask boundaries, which keeps the points near the center while minimizing flickering.

We further extend our pool with datasets that provide video object tracks in the form of bounding boxes. These datasets span diverse domains and challenging multi-object scenarios with occlusion, including pedestrians, dancers, autonomous vehicles, animals, athletes, and UAV footage. Unlike in segmentation tracks, naively sampling a (center) point from a bounding box does not guarantee that the point lies on the object. Thus, we convert each bounding-box track into a segmentation task to obtain reliable point tracks. We prompt SAM 2 with the first available bounding box for an object to generate a mask tracklet and propagate this segmentation through the rest of the video. We re-prompt SAM 2 with a new box if the predicted mask has low IoU with the ground truth bounding box or if more than 20% of the mask is outside the bounding box. We filter out object tracks whose predicted segmentation masks have an average IoU below a threshold 0.5 across all frames. We then apply the same point-sampling procedure on these generated segmentation masks to obtain point tracks. This process is depicted in the first panel of Figure 18, and the annotator interface for this step is shown in Figure 19.

Text descriptions for these tracks are acquired with human annotators. The annotation procedure is illustrated in the second panel of Figure 18, where human annotators are given a video and its list of object tracks and are asked to select one or more objects to write text queries for. The query should describe the selected objects only. The process is repeated N times per video, while ensuring that the set of selected objects is unique for each query. A separate validation round performs quality checks on the annotated text queries. After this filtering, we retain approximately 70% of the queries on average. This process yields both our training set and the Molmo2-Track benchmark. The annotator interface for validation is shown in Figure 20.

Table 21 summarizes the dataset statistics, and Figures 16 and 17 break down the distribution of queries and objects per semantic category for both training data and Molmo2-Track. The segmentation datasets provide general object tracking across diverse categories, while the bounding-box datasets contribute domain-specific tracking scenarios. Together, these complementary data sources yield a large-scale and diverse corpus for object tracking.

Data Source	Type (Ann.)	# Clips	# Tracks	# Queries	Avg # Obj/Q
VIPSeg	General (Segm)	675	2,150	5,466	2.65
SAM-V	General (Segm)	1,090	2,282	2,537	1.43
MOSEv2	General (Segm)	463	1,107	1,168	2.08
MOSE	General (Segm)	337	863	880	1.91
TeamTrack	Sports (Bbox)	154	899	1,158	2.13
SoccerNet	Sports (Bbox)	610	4109	4420	6.60
SportsMOT	Sports (Bbox)	396	2,150	2,420	4.48
BDD100K	Auto. Driving (Bbox)	450	1,810	1892	3.10
APTv2	Animals (Bbox)	401	1,051	1,132	2.68
AnimalTrack	Animals (Bbox)	52	413	542	3.59
BFT	Animals (Bbox)	30	214	364	2.38
UAV-MOTD	UAV (Bbox)	142	426	437	3.43
SeaDrones	UAV (Bbox)	79	368	408	2.25
MOT20	Person (Bbox)	147	603	643	2.68
PersonPath	Person (Bbox)	1,146	2,383	2,502	1.86
DanceTrack	Dancers (Bbox)	704	3,199	3,735	4.07
Total	**All**	**6,624**	**25,437**	**29,704**	**3.38**

(a) Statistics for the Molmo2-VideoTrack dataset.

Data Source	Type (Ann.)	# Clips	# Tracks	# Queries	Avg # Obj/Q
APTv2	Animals (Bbox)	188	331	332	1.57
PersonPath	Person (Bbox)	487	958	992	1.58
SportsMOT	Sports (Bbox)	323	825	838	4.03
DanceTrack	Dancers (Bbox)	360	885	905	3.11
SAM-V	Misc (Segm)	28	63	80	1.21
Total	**All**	**1,386**	**3,062**	**3,147**	**2.66**

(b) Statistics for the Molmo2-Track benchmark

Table 21 Distribution of tracking dataset for Molmo2-VideoTrack (train) and Molmo2-Track (benchmark). We report the number of unique video clips, unique tracks, total queries, and average number of objects per query (Avg # Obj/Q) for each dataset. Type indicates video category; Ann. indicates original tracking annotation format (Segm: segmentation masks, Bbox: bounding boxes).

Academic-VideoTrack. We additionally construct an Academic-VideoTrack dataset by aggregating existing academic VOS datasets and bounding-box tracking datasets with referring expressions. Similar to the bounding-box processing for Molmo2-VideoTrack, we convert bounding-box tracks into segmentation mask tracklets by running them through the same pipeline (bounding-box–prompted SAM 2 followed by propagation and IoU-based filtering).

We also accommodate datasets with non-exhaustive labels, where objects mentioned in the text queries lack corresponding tracks despite appearing in the video. Since these missing objects cannot be used directly for general multi-object tracking, we repurpose them for the "single-point" task (Section 3), where the model receives a single point on the target object with the associated query and generates its track. This allows us to augment non-exhaustive tracking datasets to our training data and have the model be exposed to diverse, challenging tracking scenarios.

Table 13 shows the detailed composition of the Academic-VideoTrack dataset used for training.

Molmo2-AskModelAnything. For each video, we first ask crowdworkers to watch the clip without audio and

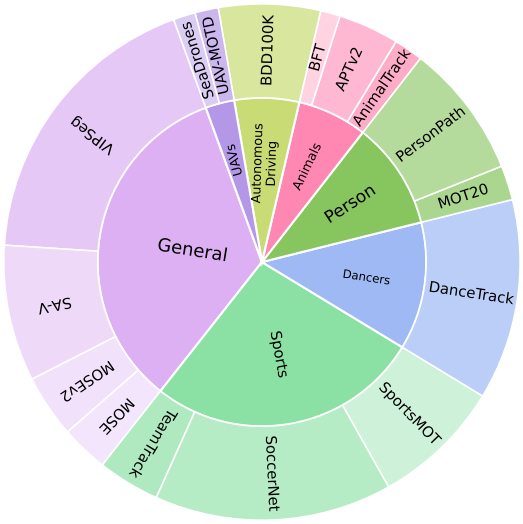


Figure 16 Molmo2-VideoTrack dataset

Figure 17 Molmo2-Track benchmark

write questions in English that require non-trivial visual reasoning, such as temporal understanding, reading on-screen text details, or identifying fine-grained visual details. We discourage questions that were too vague, too easy or low-level, subjective with no clear ground-truth answer, dependent on unverifiable information such as names or identities, or simple counting questions, which we do not collect for this task. We then feed the full video caption together with the worker's question into a backend language model, which produces an initial answer. Workers are then instructed to slightly edit the question to form a valid query and to carefully edit the model answer to form a final answer. Once they are satisfied, they submit the final Q&A pair, which we used as our annotation (see Figure 24 for the task interface).

G Data examples

Here, we present qualitative examples from the Molmo2 datasets. For datasets, we show **randomly** selected examples. Prompts are in bold, and the target output text is below. Videos are shown using a small number of sampled frames. Examples can be found in:

- Molmo2-Cap: Figure 25
- Molmo2-AskModelAnything: Figure 26
- Molmo2-CapQA: Figure 27
- Molmo2-SubtitleQA: Figure 28
- Molmo2-VideoPoint: Figure 29
- Molmo2-VideoTrack: Figure 30
- Molmo2-MultiImageQA: Figure 31
- Molmo2-SynMultiImageQA: Figure 32
- Molmo2-MultiImagePoint: Figure 33

H Limitations

Here we discuss some of the limitations of the Molmo2 models.

Closed image ViT. Even with OLMo 3 as the LLM, our models still utilize a closed-data SigLIP 2 image

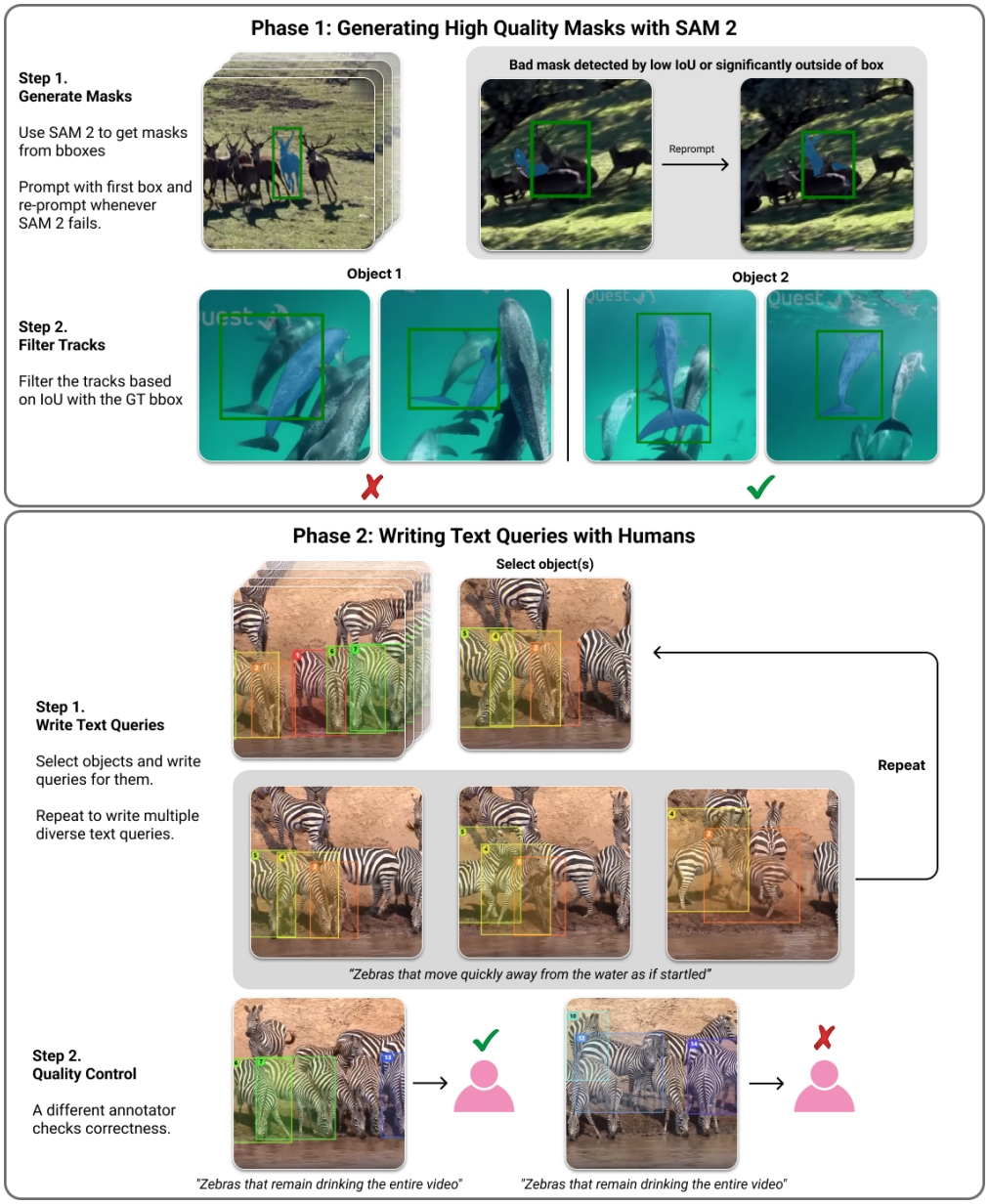


Figure 18 Overview of the annotation pipeline for **Molmo2-VideoTrack** and the **Molmo2-Track** benchmark.

encoder [139]. We chose to use SigLIP 2 because there are currently no competitive open-data encoders. We call upon the open-source community to explore such alternatives in future work.

Use of closed LLMs. We use closed text-only LLMs for data generation, as is common practice [89]. This reduces the transparency of our data collection pipeline. However, we believe that future open LLMs will become sufficiently proficient to be used in place of closed ones to reproduce this dataset in a fully open manner. It is still important that we avoid using closed *VLM*s, which would create a circular dependency (training our VLMs would require first building a VLM to generate the training data) and therefore cannot lead to a fully open system in the same way.

Video grounding repeating points. For both video tracking and pointing, we sometimes observe that the model produces degenerate outputs, such as a long line of points on one frame or the same point for every frame. This is particularly common when pointing to high-frequency objects or on long videos, so this could likely be mitigated by sourcing more training data to better cover these cases. We also observe the issue is

less common in specialized models, so we hypothesize that there might be some interference between the tasks in the joint training mixture which leads to this behavior.

Video grounding. Video grounding is less consistent than image grounding. Our metrics reflect this, with none of the models we tested reaching more than 40% on either our counting or pointing metrics, while image models often achieve 70-90% on image grounding metrics like PointBench.

We believe this is partly due to the inherent complexity of the task. Video grounding typically requires looking at much more visual content, and pointing at more things, than image grounding. Video grounding also requires re-identification, meaning understanding whether two objects in two different frames are the same object or not, which can be challenging. We also think that the lower resolution typically used when processing long videos, and the fact that the vision encoders are often not pre-trained on videos, could be contributing factors.

Long video grounding. Grounding has limited support for long (3 minutes+) videos because our grounding training is limited to that length. Handling longer videos is complicated by the fact that we would have to lower the fps when sampling frames to < 2. This would result in our annotations, which are always at 2 fps, not being aligned with the selected frames. A possible solution is to customize how frames are sampled in these cases to ensure that all grounding annotations are selected.

Point tracking. Molmo2's generated tracks will sometimes change the location of its output point on the target object. This is likely because our tracking data generation pipeline does not always ensure that the point is consistently placed within the target object for every frame. Future improvements in generating points from bounding box or segment mask data could mitigate this issue.

Captioning. We observe that Molmo2 can sometimes generate repeating text when generating a very long video caption using greedy decoding. This is a known issue with LLMs [52], including Qwen3[7]. However, we also think that the limited captioning training data contributed, as well as the high length of the captions (we observe that this typically occurs after generating thousands of tokens). We do not observe this behavior for other tasks.

I Qualitative results

We show qualitative examples from Molmo2-8B. Each figure shows a query, the response from the model, and selected frames from the input video. The returned points are annotated with pink dots. Successful examples are shown in Figure 34 and Figure 35. We also show some failure cases in Figure 36.

[7] https://huggingface.co/Qwen/Qwen3-4B Best Practices

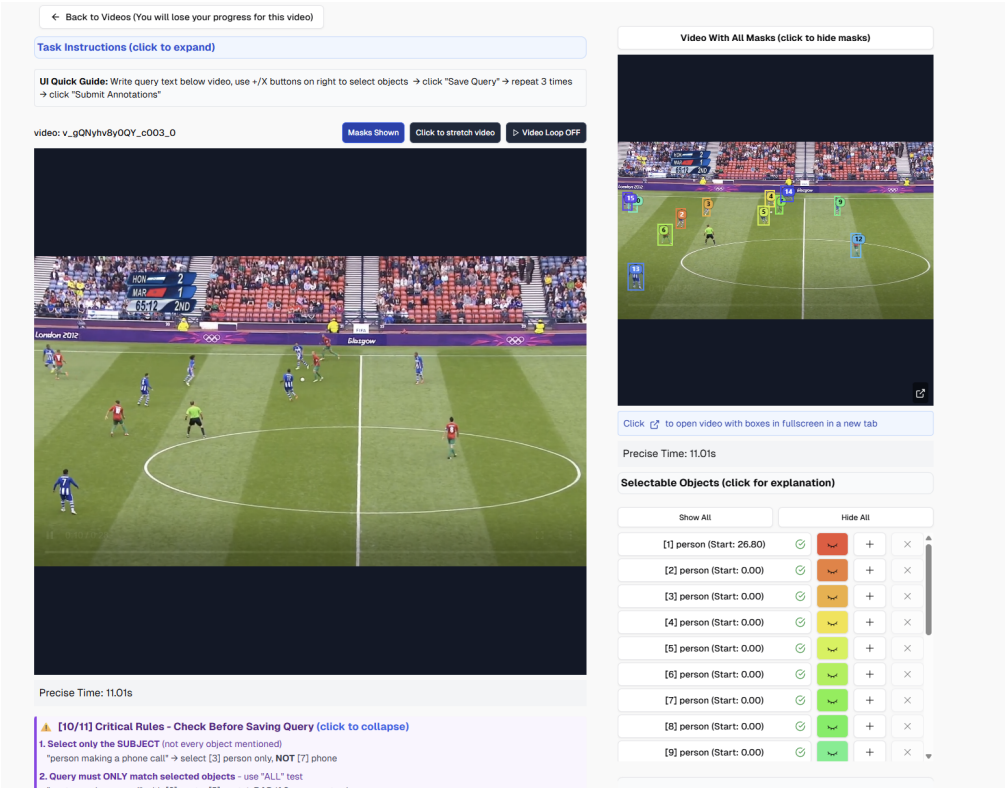


Figure 19 Crowdworkers annotating object text queries.

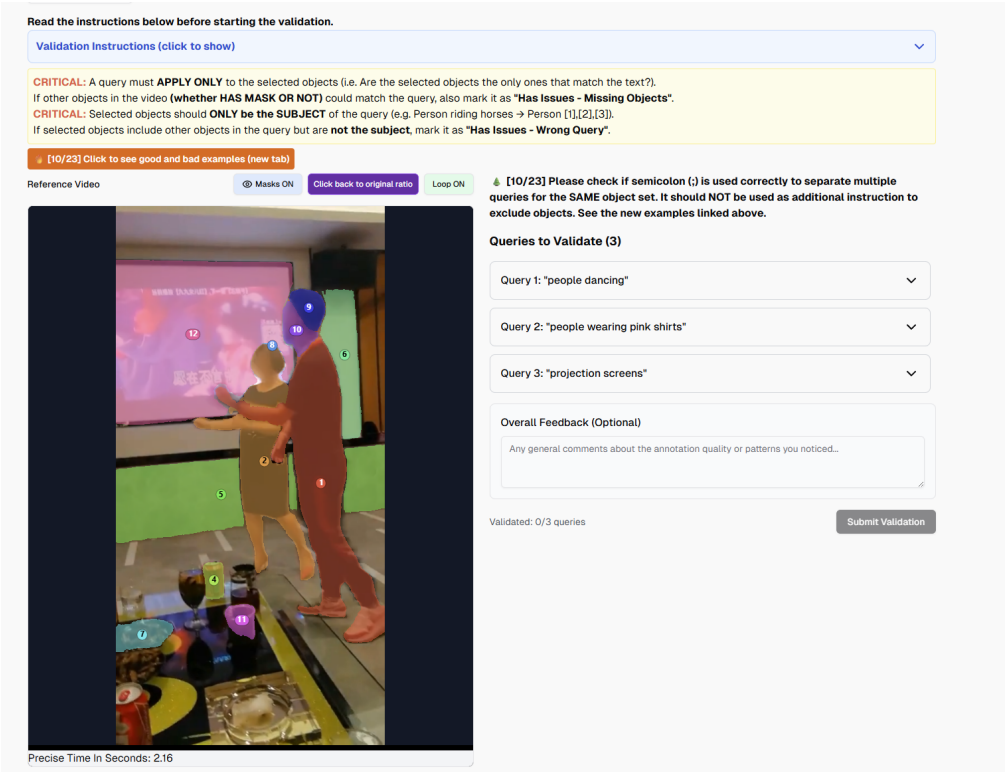


Figure 20 Crowdworkers validating object text queries.

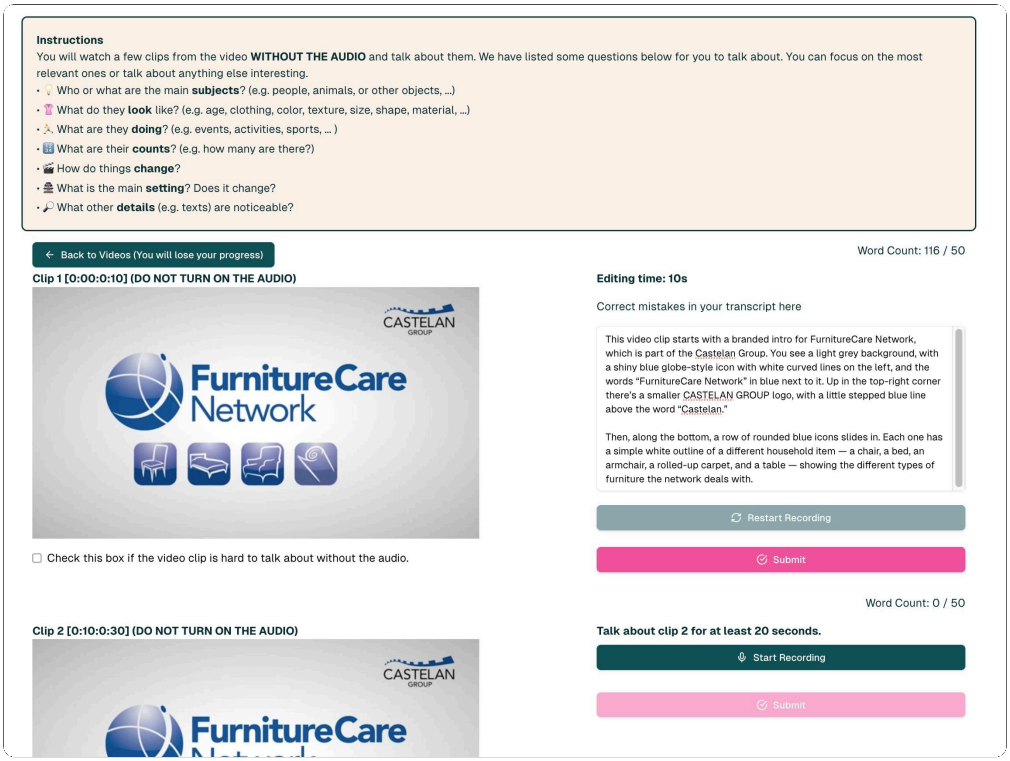


Figure 21 Video clip captioning interface. Crowdworkers are instructed to annotate captions for video clips in sequence.

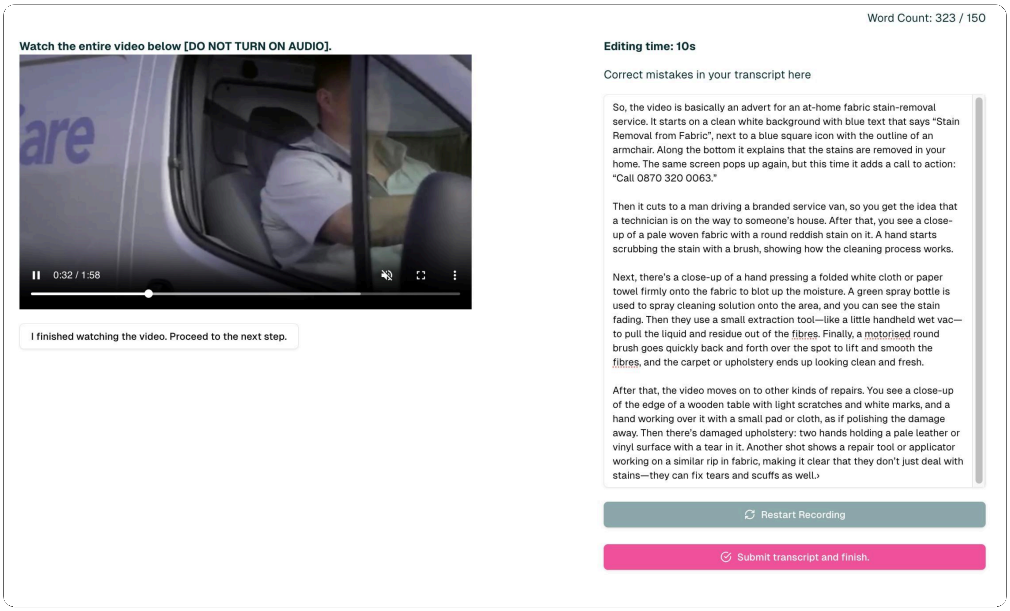


Figure 22 Video captioning interface. Crowdworkers are instructed to annotate captions for complete videos.

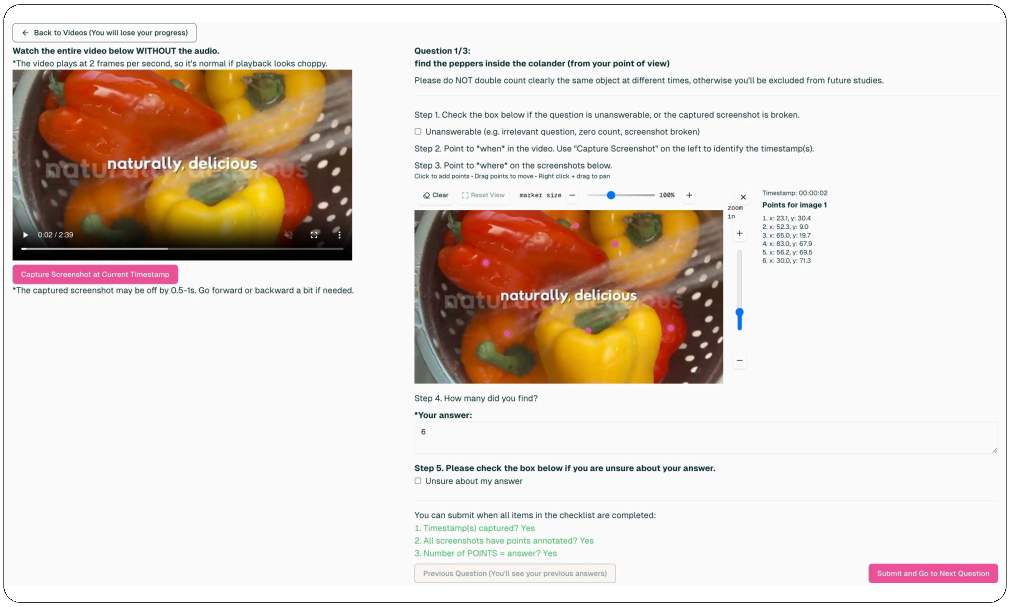


Figure 23 Video pointing interface. Crowdworkers are instructed to annotate points for object instances to answer visual questions.

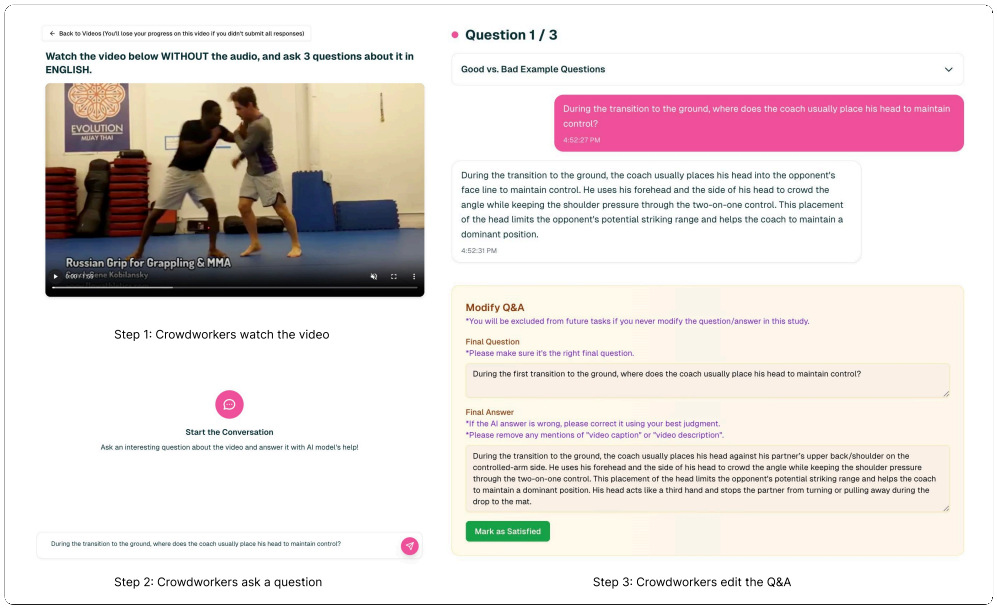


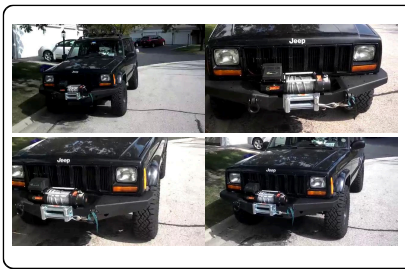
Figure 24 AskModelAnything interface. Crowdworkers are instructed to ask model non-trivial visual questions and finalize Q&A.

Describe this video in detail

A live newsroom segment unfolds under the banner of a nightly program, opening with a large studio screen that reads "In TV Tonight" as a female anchor in an olive green dress steps into frame. She holds a white piece of paper and a pen in her right hand, wears a gold necklace with a red accent at the front, and addresses the camera from the left side of the set. The NTV logo appears in the upper right corner as a teardrop with a colored swirl, and a time display reads 21:30:45 while a black…

Caption

Inside a large wide-body airliner, likely a 747, an empty twin-aisle cabin appears bright and clean: a red carpet runs between long rows of seats upholstered in combinations of red with tan, gray, or white, many topped with red headrest covers and matched with red seatbelts. Overhead bins remain closed beneath a white ceiling with a blue section down the center, and panels and compartments show a mix of white and blue; windows sit behind white shades that let in bright light. A smiling flight attendant of Asian descent with long, curled dark hair walks down the aisle toward the…



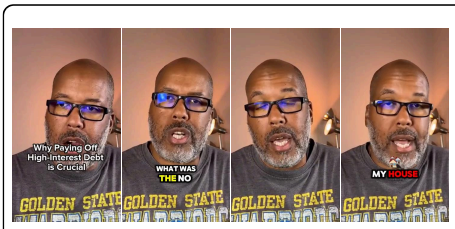
Generate a long caption about this video.

A black Jeep Cherokee XJ sits in a residential setting, first shown full and head-on, its boxy body gleaming with a slight gloss that reflects trees and sky. The vehicle stands noticeably lifted on large, aggressive-tread black tires mounted to matte black wheels, the front end dominated by a heavy-duty black steel bumper carrying a centrally mounted winch. The "Jeep" name appears in white letters at the very front, and again on the hood, framed by a black grille…

Write a long caption

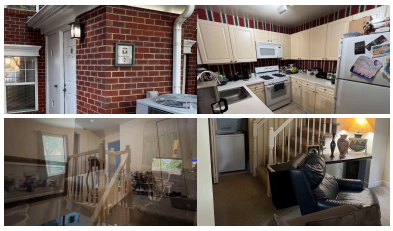
A vibrant, child-friendly teaching graphic fills the screen. Toward the top left sits a xylophone on a black, center-lit surface, its light tan, wood-like body holding light-colored metal bars that form tall-to-short rectangular keys from left to right. A mallet with a blue ball head is visible. Above the instrument, white note letters read "C, D, E, F, G, A, B, C" (at one point extended to include "D"). Just beneath the xylophone, the solfege appears in blue text as "Do, Re, Mi, Fa, Sol, La, Si, Do." The lower half shows two smiling cartoon buses against a light blue background: a bl…

Figure 25 Random examples from Molmo2-Cap. Prompts are generated from our captioning prompt templates.



What team is written on the shirt this man is wearing?

The man is wearing a shirt with "Golden State Warriors" on it.



what number is on the plaque in front of the house

The number on the plaque in front of the house is 2.

What message or emphasis do the gestures convey?

The gestures convey strong emphasis and conviction in what the man is communicating. His hand movements appear deliberate and purposeful, suggesting he is making important points that he wants to drive home to his audience.

The specific nature of his gestures reveals several layers of meaning:

Authority and clarity - When he cuts through the air with his hands and raises his palm forward in a stopping or presenting motion, he seems to be punctuating key ideas and ensuring his points land with precision.…

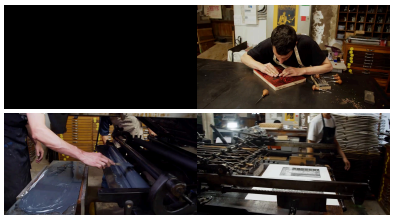
What color shorts is the man carrying the bushel of green plants on his shoulders wearing?

The man carrying the large bundle of green vegetables on his head is wearing bright yellow shorts as he walks through the waterlogged lane.

Figure 26 Random examples from Molmo2-AskModelAnything.

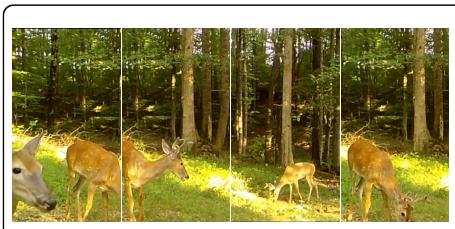
How many digits are in the case number shown on the report page? Short answer:

11



What human relationship is depicted by the small sculpture above the building's door?
Only return the correct answer option.
A. Two men shaking hands.
B. A woman holding a child.
C. A couple embracing.
D. A soldier carrying a wounded comrade.

B. A woman holding a child.



What specific action does the deer closest to the camera perform?
Only return the correct answer option.
A. It stamps a hoof.
B. It licks its lips.
C. It lies down.
D. It shakes its head vigorously.

B. It licks its lips.

In the opening advertisement, which way does the scrolling "listen to Dr. Tony Evans" text move?
Only return the correct answer option.
A. downward
B. left
C. upward
D. right

C. upward

Figure 27 Random examples from Molmo2-CapQA.

Figure 28 Random examples from Molmo2-SubtitleQA.

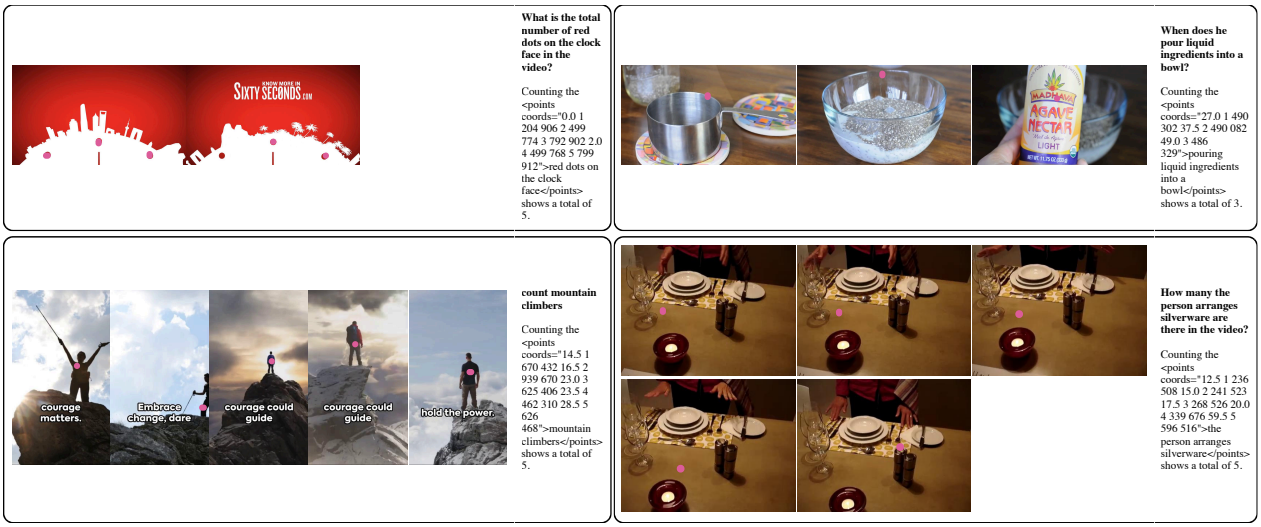


Figure 29 Random examples from Molmo2-VideoPoint. Points are shown in pink, output text follows Molmo2's point formatting.

Figure 30 Random examples from Molmo2-VideoTrack. Points are shown in different colors that are shared between the same objects, output text follows Molmo2's point formatting.

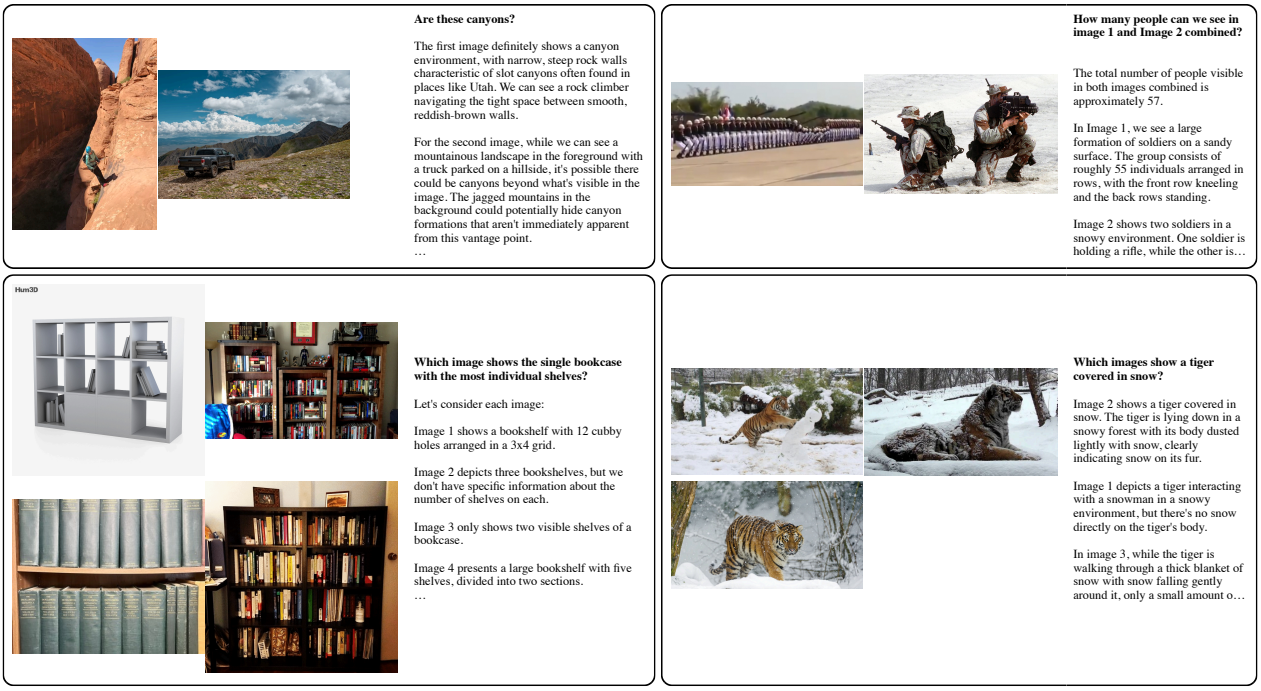


Figure 31 Random examples from Molmo2-MultiImageQA.

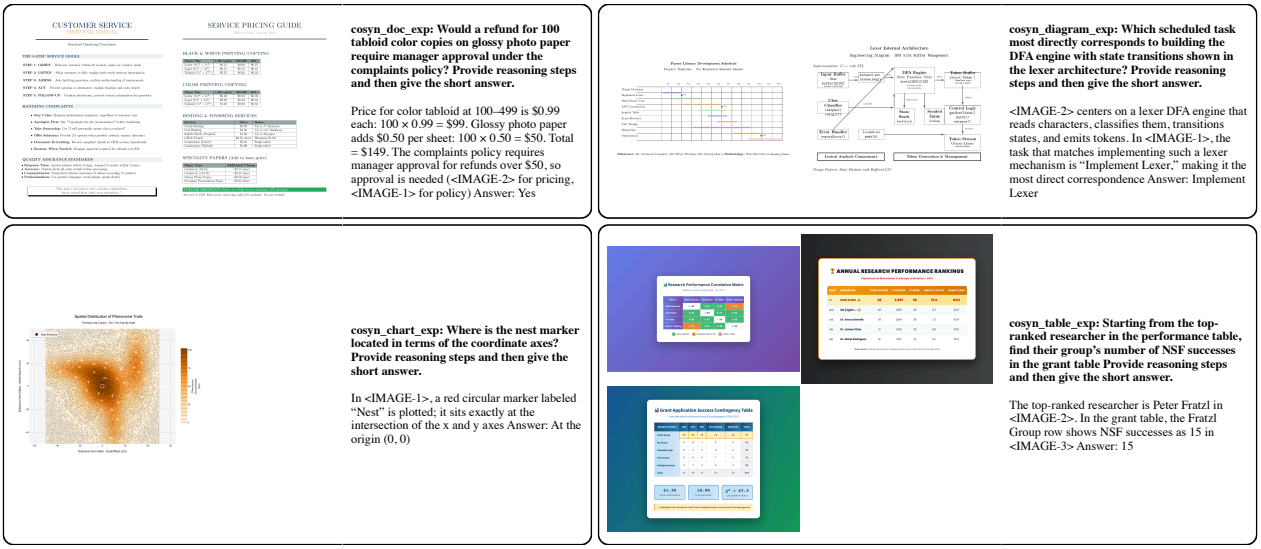


Figure 32 Random examples from Molmo2-SynMultiImageQA.

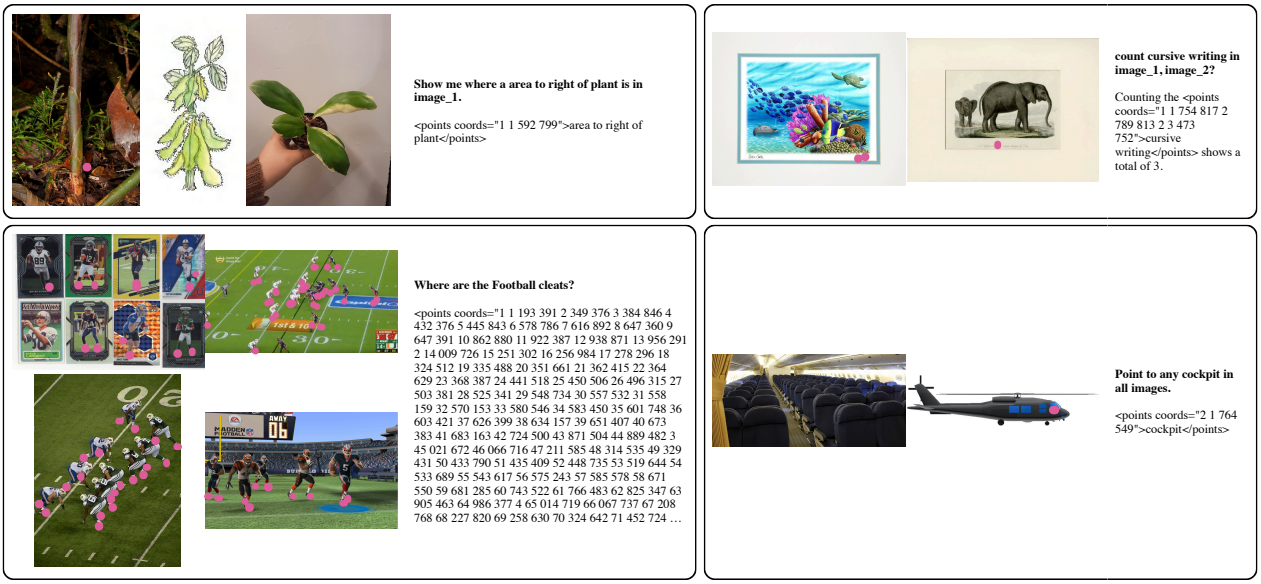


Figure 33 Random examples from Molmo2-MultiImagePoint. Points are shown in pink, output text follows Molmo2's point formatting.

Q: Describe this video.
A: Under bright stadium lights at night, a packed crowd fills the stands as a college baseball game unfolds on a lush green field with crisp white baselines and a brown infield. The home team wears white pinstripes with blue helmets and blue socks, while the visiting team is in maroon jerseys with gray pants and black helmets...

The sequence shifts to a close-up of a player in a white pinstriped jersey with blue accents and a blue cap, the number "21" visible on his back, his right arm extended forward and his left arm bent at the elbow, a black glove on his left hand. Behind him, a green wall with a yellow stripe and a white railing frames the scene, with a few spectators visible in the stands...

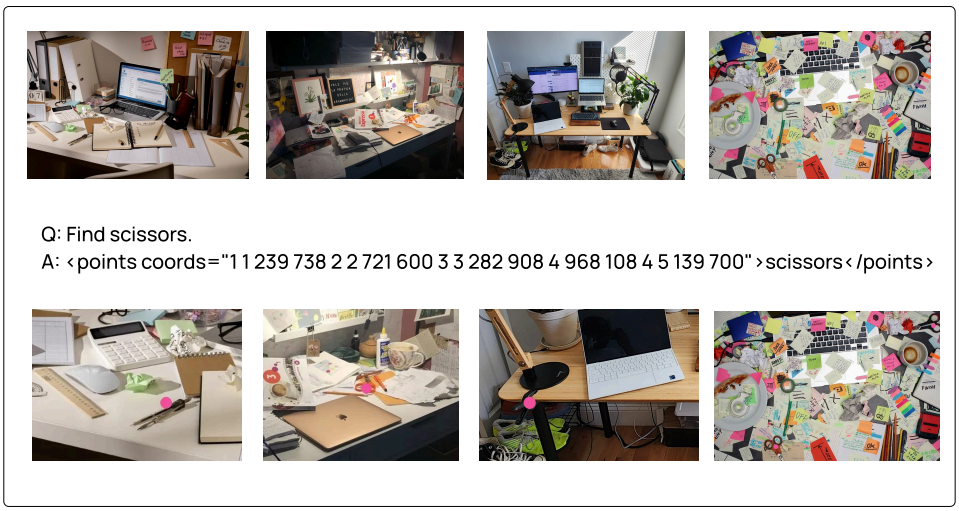
The celebration erupts on the field. Players in white pinstripes with blue helmets and blue socks converge in a jubilant pile, jumping and hugging amid the cheers of the packed stadium....

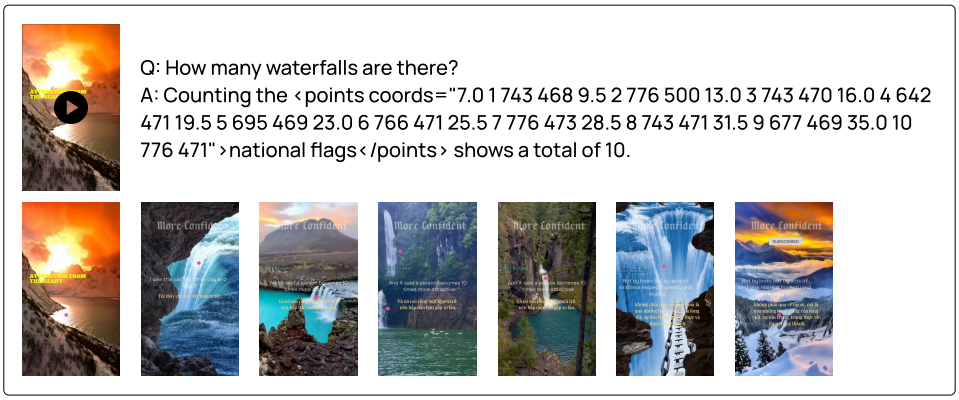
Q: How many national flags are there in the video?
A: Counting the ‹points coords="7.0 1 743 468 9.5 2 776 500 13.0 3 743 470 16.0 4 642 471 19.5 5 695 469 23.0 6 766 471 25.5 7 776 473 28.5 8 743 471 31.5 9 677 469 35.0 10 776 471"›national flags‹/points› shows a total of 10.

Q: Track the horse.
A: ‹tracks coords="0.0 1 381 528 0.5 1 340 562 1.0 1 300 507 1.5 1 431 569 2.0 1 465 535 2.5 1 498 514 3.0 1 431 410 3.5 1 292 472 4.0 1 354 493 4.5 1 531 347 5.0 1 656 514 5.5 1 617 340 6.0 1 398 264"›horse‹/tracks›

Figure 34 Qualitative examples of captioning, counting, and tracking from Molmo2-8B

Figure 35 Qualitative examples of pointing and QA from Molmo2-8B





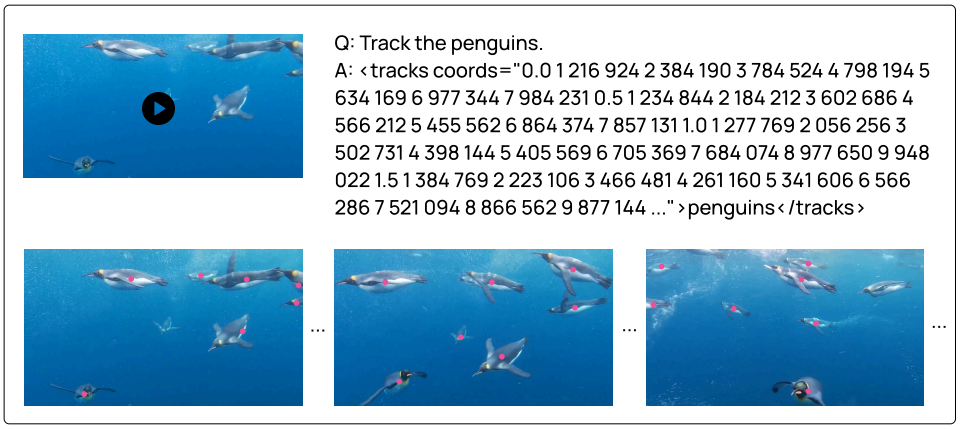


Figure 36 Qualitative failure cases from Molmo2-8B. The model identifies false positives in the first two examples and misses several of the penguins in the bottom example.